A Framework for Identification of Similarities between Multiple Algorithms

A THESIS SUBMITTED TO
THE SCIENCE AND ENGINEERING FACULTY
OF QUEENSLAND UNIVERSITY OF TECHNOLOGY
IN FULFILMENT OF THE REQUIREMENTS FOR THE DEGREE OF
DOCTOR OF PHILOSOPHY

A. A. Madhushika Madara Erangani Karunarathna

Supervisors: Professor Yu-Chu Tian and Professor Colin Fidge

School of Electrical Engineering and Computer Science
Science and Engineering Faculty
Queensland University of Technology

March 2015

Copyright in Relation to This Thesis

© Copyright 2015 by A. A. Madhushika Madara Erangani Karunarathna. All rights reserved.

Statement of Original Authorship

The work contained in this thesis has not been previously submitted to meet requirements for an award at this or any other higher education institution. To the best of my knowledge and belief, the thesis contains no material previously published or written by another person except where due reference is made.

QUT Verified Signature

Signature:

Date: 23/03/2015

To my loving parents.

Abstract

Algorithms define the step-by-step procedure used to perform computations in a computer system. Over the last half century, numerous algorithms have been developed to solve various computing problems. For better computing efficiency, how to improve the performance of computing algorithms has continuously been a widely discussed topic in various research fields. Many research works have investigated this performance improvement problem in terms of different parameters such as execution speed, memory consumption, and power and area cost. With the significant performance improvements made through specific software/hardware systems designed for a specific set of algorithms, investigating similar functional units in multiple algorithms has become a major research interest. Identification of such common operations significantly helps in boosting the performance of the whole set of algorithms by improving their execution. However, relations between the algorithms in terms of their internal operations is generally non-trivial to understand by simply looking at what functionality they perform and/or what results they produce. In many research approaches, such operations are selected manually based on the designer's knowledge about the algorithms, leading to a significant consumption of human effort and time. Existing automated analysis methodologies only consider a single algorithm at a time, and thus limit the effectiveness of the results to only one algorithm. To overcome the weaknesses of the existing solutions, this thesis presents an automated approach to the identification of similarities among a set of algorithms. A framework is presented with new static analysis strategies and an algorithm classification approach. The main contributions of this thesis include:1) A new static analysis method to convey the execution frequency in an operation of a given algorithm, 2) A novel algorithm clustering method to cluster closely related algorithms together statically based on their common operations, 3) A heuristic weighting system for choosing operations for different target applications, and 4) A customisable framework to identify characteristic operations for multiple algorithms and adapt resultant operations according to the property required by different target applications. These contributions would

support efficient developments in different application areas depending on common operations of multiple algorithms. In this thesis three such application areas have been considered: creation of special-purpose instruction sets for multiple algorithms, design of library functions common to multiple algorithms and identification of common “hot-spots” of multiple algorithms. Our results show that it is possible to identify relationships between algorithms in an automated way and to effectively cluster algorithms by the significant operations that they have in common.

Keywords

Algorithm analysis, Algorithm clustering, Parameter weighting system, Algorithm similarities, Special-purpose operations

Acknowledgments

Foremost, I would like to express my sincere gratitude towards my principal supervisor Professor Yu-Chu Tian and my Associate Supervisor Professor Colin Fidge, for their continuous support given during my PhD study. I am grateful for their patience, motivation, enthusiasm, and immense knowledge which strengthened me to make this dream a reality. Without their encouragement and effort, my research and thus this thesis would not have been accomplished. Further, I would like to extend my gratitude towards Dr Ross Hayward for his generous support, positive motivation and important technical contribution.

I would like to acknowledge the academic and technical support given by other staff members of QUT at various workshops, lectures and other gatherings during my PhD candidature. I am thankful for the financial support provided by QUT and the Australian government by awarding International Postgraduate Research Scholarship (IPRS), Vice-Chancellor's (VC) Top-Up Scholarship and Australian Postgraduate Award (APA) scholarships. Also I would like to thank Ms. Diane Kolomeitz for proofreading my thesis.

Further, I would like to thank my colleagues, Rajitha Navarathna, Dhanushka Krishnajith, Palmo Thinley, Lapas Pradittasnee and Harish Ramani for their encouragements, time and support.

Finally, I would like to express my heartiest gratitude towards my parents, A. A. Karunarathna and Sepalika Baddrangani, and my brother, Isuru Karunarathna, for the unconditional support given throughout my life. I would also like to thank from the depths of my heart, to my husband Anuruddha Ratnayake, for being by my side through the good times and the bad as well.

Table of Contents

Abstract **vii**

Keywords **ix**

Acknowledgments **xi**

1 Introduction **1**

1.1 Background 2

1.2 Motivation 4

1.3 Problem Statement 6

1.4 Significance of the Research Problem 8

1.5 Contributions of this Research 9

1.6 Evaluation Criteria 10

1.7 Organisation of the Thesis 10

1.8 Publications Arising from this Research Work 10

2 Literature Review **13**

2.1 Relationships between Algorithms 13

2.2 Algorithm Analysis 16

2.3 Static Analysis Approaches 18

2.4 Dynamic Analysis Approaches 21

2.5 Summary of Algorithm Analysis Related Work 22

2.6 Algorithm Clustering 23

2.7 Algorithms Investigated in this Thesis 25

2.7.1 Exact String Matching Algorithms 25

2.7.2 Cryptographic Algorithms 25

2.7.3 Sequence Alignment Algorithms 26

2.7.4 Sorting Algorithms 28

3 A Customisable Framework for Identification of Similarities between Multiple Algorithms 29

3.1 Overview of Common Operation Identification Framework 29

3.2 Overview of Steps Involved in the Framework 34

3.3 Generation of Atomic Instruction Sequences 35

3.4 Static Frequency Analysis Steps 35

3.4.1 Sequential Code Analysis 35

3.4.2 Nesting Level Analysis 37

3.5 Operation Property Analysis 37

3.6 Intermediate Results 38

3.7 Algorithm Clustering Approach 38

3.8 Instruction Sequence Merging and Commonality Factor Assignment 39

3.9 Characteristic Operation Selection 40

4 A Static Analysis Approach for Analysing Run-time Execution Frequency of Operations 41

4.1 Background of Characteristic Operation Identification 41

4.2 Basic Principles of Frequency Analysis Strategies 43

4.2.1 Basic concepts 43

4.2.2 Insights for the Frequency Analysis Strategies 45

4.3 Fundamental Design of Analysis Steps 46

4.3.1 Basic Static Frequency Counting 46

4.3.2 Finding the Nesting Levels of Operations 49

4.3.3 Block Level Analysis 50

4.4 Development of Final Analysis Strategies 51

4.4.1 Atomic Operation Generation 52

4.4.2 Sequential Code Analysis 55

4.4.3 Nesting Level Analysis of Instruction Sequences 59

4.5 Experimental Results 65

4.5.1 Results obtained for cryptographic algorithms 67

4.5.2 Results obtained for exact string matching algorithms 73

4.6 Summary of Static Analysis Strategies 78

5 Algorithm Clustering Methodology 83

5.1 Basic Concepts of the Algorithm Clustering Approach 84

5.2 Development of Algorithm Clustering Method 85

5.2.1 Characteristic Operation Identification 86

5.2.2 Initialising the Boolean Data Table 86

5.2.3 Vector Representation 87

5.2.4 Distance Calculation 87

5.2.5 Centre and Member Points Selection 88

5.2.6 Clustering 89

5.3 Experimental Results of the Algorithm Clustering Approach 91

5.3.1 Characteristic Operation Identification 92

5.3.2 Vector Representation 94

5.3.3 Distance Calculation 94

5.3.4 Centre and Member Points Selection 94

5.3.5 Clustering 95

5.4 Dynamic Analysis Based Evaluation of Clusters 97

5.4.1 Empirical analysis results 97

5.4.2 Special purpose instruction results 99

5.5 Evaluation of the Clustering Approach in Multiple Sets of Algorithms 107

5.6 Discussion 111

6 Heuristic Weighting System for Adaptation of Operations Targeting Different Applications 113

6.1 Concepts of the Weighting System 115

6.1.1 Determining requirements and specifications of target applications . . . 116

6.1.2 Analysing algorithms to measure properties of operations 119

6.1.3 Calculating weights and priorities 120

6.2 Case study 1: Generating Library Functions 121

6.2.1 Software re-usability 121

6.2.2 Experimental set-up 122

6.2.3 Resultant characteristic operations 122

6.2.4 Performance gain of the Library Functions 124

6.3 Case study 2: Custom Instruction Generation 126

6.3.1 Selection of Custom Instructions for an ADSP 132

6.3.2 Custom instruction generation experiment 133

6.3.3 Performance gain of the custom instructions 136

6.4 Case Study 3: Finding “Hot Spots” of a Set of Slgorithms 147

7 Conclusions **153**

7.1 Summary of Contributions 153

7.1.1 Static based analysis strategies 154

7.1.2 Classification method 155

7.1.3 Weighting system 156

7.1.4 Concluding remarks 157

7.2 Limitations and Future Work 157

List of Figures

3.1 Abstract Overview of the Framework 30

3.2 Static Frequency Analysis Approach 31

3.3 Clustering Approach Overview 32

3.4 Weighting System Overview 33

3.5 Overview of the Framework 34

3.6 Overview of the tools used in the steps of the framework 36

4.1 Example of two sequences in which data flow is different semantically 46

4.2 Example of syntactically similar but semantically different instruction sequences 48

4.3 Loop segment in a control flow graph. 50

4.4 Example of a false-positive instruction sequence 53

4.5 Sample assembly instruction code segment belonging to two basic blocks and all their possible instruction sequences 54

4.6 Syntactic and semantic matching of two instruction sequences 56

4.7 Semantic matching of two instruction sequences. 57

4.8 Sequential code analysis process 59

4.9 Calculation of the execution probability of conditional statements (the probability of execution of a root statement is considered as 1). 60

4.10 Implementation of tracking the open brackets and push brackets on the bracket stack 61

4.11 Demonstration of conversion of source code by the pretty printer 62

4.12 Example of stacking the brackets in the *Nesting Level Analysis* 63

4.13 Implementation of tracking the closing brackets and calculating nesting levels accordingly 64

4.14 Nesting level assignment of a loop and a conditional statement from MD5 hash algorithm of *polarSSL* library (the default nest level is 1). 65

4.15 Nesting level assignment of a sequence of instructions according to corresponding high level source code, ... 66

4.16 Nesting level analysis process. 66

4.17 Execution Frequency of AES algorithm with CBC mode and MD5 hash function 72

4.18 Execution frequency of AES algorithm with CTR mode and SHA1 hash function. 73

4.19 Execution frequency of BLOWFISH algorithm with CBC mode and MD5 hash function. 74

4.20 Execution frequency of BLOWFISH algorithm with CTR mode and SHA1 hash function. 75

4.21 Execution frequency of CAMELLIA algorithm with CBC mode and SHA1 hash function. 76

4.22 Execution frequency of CAMELLIA algorithm with CTR mode and MD5 hash function. 77

4.23 Execution frequency of DES algorithm with CBC mode and MD5 hash function. 78

4.24 Execution frequency of DES3 algorithm with CBC mode and SHA1 hash function. 79

4.25 Frequency of operation $p[i]$ for pattern size = 100. 80

4.26 Frequency of operation $p[i] == m$ for pattern size = 100. 80

4.27 Dynamic frequencies of operation for pattern size is 100. 81

4.28 Zoom-in of the lower group of algorithms from Figure 4.27 (pattern size is 5). . 81

5.1 Overview of the application clustering method 86

5.2 Close points to A as per distance constant DC 88

5.3 Two groups with the same number of close points 89

5.4 Resultant algorithm clusters 96

5.5 Execution frequencies of the primitive operations of algorithms (*pattern size = 100)* 99

5.6 Cluster of AG, BM, TMB and ZT algorithms 99

5.7 Cluster of MP, KMP and AC algorithms 100

5.8 Cluster of CL and GG algorithms 100

5.9 Cluster of HP and RT algorithms with the NSN algorithm 101

5.10 Xtensa Xplorer 103

5.11 Cycle count comparison of the original vs new ISA 106

5.12 Instruction count comparison of the original vs new ISA 106

5.13 Energy consumption comparison of the original vs new ISA 107

5.14 Performance gain comparison of the original vs new ISA 107

5.15 Resultant algorithm clusters produced from different application domains . . . 110

6.1 Example weights for properties of operations, targeting different applications . 118

6.2 High level implementation of Big and Little Endian manipulations. 124

6.3 Assembly level implementation of Big and Little Endian manipulations. 126

6.4 Big Endian implementation and corresponding assembly code. 127

6.5 Number of calls to chosen cryptographic operations. 128

6.6 Processor cycle and instruction count variations of the original ISA vs the new ISA for the cryptographic algorithms 139

6.7 Energy and performance gain of the original ISA vs the new ISA for the cryptographic algorithms. 141

6.8 Processor cycle and instruction count variations of the original ISA vs the new ISA for the MD5 based algorithms 143

6.9 Energy and performance gain of the original ISA vs the new ISA for the MD5 based algorithms 144

6.10 Processor cycle and instruction count variations of the original ISA vs the new ISA for the SHA1 based algorithms 146

6.11 Energy and performance gain of the original ISA vs the new ISA for the SHA1 based algorithms 148

6.12 Execution frequency of characteristic operations of the *SmithWaterman* algorithm as per Table 6.12 150

6.13 Execution frequency of characteristic operations of the *NeedlemanWunsch* algorithm as per Table 6.12 151

6.14 Execution frequency of characteristic operations of the *CV Tree* algorithm as per Table 6.12 151

List of Tables

2.1 String matching algorithms analysed. 26

2.2 List of Cryptographic algorithms used in the case study 27

2.3 Algorithms used in the case study in Section 5.4.2. 27

4.1 Control flow analysis results for thirteen string matching algorithms (the omitted elements are 0), showing the frequency of occurrences of loops. 52

4.2 Valid Operation Generation Execution Performance 55

4.3 Sequential code analysis results for the Colussi, Boyer-Moore and Horspool algorithms (A: Immediate value; X, Y: Registers; a, b: Stack addresses; m, n, i, j: Integer values; p: Integer array). 58

4.4 Code segments appearing up to three nesting levels: loop nesting analysis results for the Boyer-Moore, Zhu-Takaoka and Turbo Boyer-Moore algorithms (A, B: Immediate values; X, Y: Registers; a, b, c, d, e: Stack addresses; al, dl: 8-bit Register values; m, n, i: Integer values; p: Integer arrays). 67

4.5 Code segments appearing up to two nesting levels: loop nesting analysis results for the Boyer-Moore, Zhu-Takaoka and Turbo Boyer-Moore algorithms (A, B: Immediate values; X, Y, Z: Registers; a, b, c, d: Stack addresses; al: 8-bit Register values; m, i: Integer values; p, q: Integer arrays). 68

4.6 Static frequencies found for some instruction sequences of several cryptographic algorithms. 69

4.7 Nesting Level Measurements obtained for some instruction sequences of several cryptographic algorithms. 70

4.8 Top characteristic operations resulting from the analysis. 71

4.9 Candidate set of special-purpose operations. 73

5.1 String matching algorithms analysed. 91

5.2 Characteristic operations found in the string matching algorithms 93

5.3 Euclidean distance matrix of the case study 95

5.4 Initial algorithm groups 96

5.5 Special purpose operations of each cluster 102

5.6 Selected custom instructions for the evaluation 103

5.7 Evaluation results comparison for the string matching algorithms set 105

5.8 Algorithms used in the case study in Section 5.5. 108

5.9 Characteristic operations found in the groups of selected algorithms 109

5.10 Euclidean distance matrix of the case study in Section 5.5 110

6.1 Rankings of operations specifications of the considered target applications. . . 117

6.2 Performance Parameter Factors 124

6.3 Top prioritised sequences - part 1 125

6.4 Top prioritised sequences - part 2 130

6.5 Final set of Operations to implement library functions 131

6.6 Performance Parameter Factors for ADSP design 132

6.7 Top priotorised sequences from ADSP design experiment 135

6.8 Top prioritised sequences obtained with modified parameters 137

6.9 Evaluation results comparison of custom instructions generated for all cryptographic algorithms 138

6.10 Evaluation results comparison of custom instructions generated for MD5 hash based algorithms 142

6.11 Evaluation results comparison of custom instructions generated for SHA1 hash based algorithms 145

6.12 Top characteristic operations resulting from the analysis of *sequence alignment* algorithms 149

Chapter 1

Introduction

Algorithms lie at the heart of computer science. They define the step-by-step procedure to perform calculations and computations in a computer system. Over the past five decades, numerous algorithms have been introduced to solve common human problems [Levitin, 2007]. Many of them are related to one another, or derived from one another, in different ways. However, relationships between the algorithms in terms of their internal operations can be non-trivial to understand by looking at what functionality they perform or what results they produce. Identification of such common operations significantly helps in boosting the performance of the whole set of algorithms by improving their execution. Cross-product, normalisation, dot-product, MAC (Multiply-Accumulator), and Filters are a few well-known special operations used in many application domains, such as DSP (Digital Signal Processing), computer graphics, image processing, etc. The aim of our work is to identify such similarities between sets of algorithms. These studies would support efficient developments in different application areas that depend on common operations of multiple algorithms. In this thesis, three such application areas will be considered: creation of special-purpose instruction sets for multiple algorithms, design of library functions common to multiple algorithms and identifying common "hot-spots" of multiple algorithms.

This study is conducted by developing a methodology to identify common *characteristic operations* for a given set of algorithms. *Characteristic operations*, also known as "special-purpose operations" or "basic operations", are the most critical set of operations of an algorithm, and define its run-time execution behaviour [Sedgewick and Flajolet, 2013]. Our analysis considers multiple algorithms at a time and retrieves such common operations from the algorithms.

In addition to the execution frequency, other properties can be analysed in these operations, such as size, commonality among the considered algorithms, and their hardware cost. From the variations showed by each operation in these property analyses, suitable sets of operations can be selected for different applications. The selection process is varied according to the specific features desired by the applications. For instance, in a code library generation application, longer operations are preferred, while in a special-purpose instruction design application, comparably shorter operations are prioritised. Thus, when a set of algorithms are given, our methodology finds similarities among the algorithms in terms of their critical operations. Consequently, suitable sets of common operations can be identified targeting different applications.

1.1 Background

In our work, a methodology to identify and analyse operations of algorithms is built with the help of algorithm/application analysis concepts. Algorithm analysis is a widely discussed topic in the field of computing science. As computer systems are designed to execute various algorithms, analysing them for different software/hardware perspectives has been considered essential since the early stages of computer science [Sedgewick and Flajolet, 2013]. The trend still continues to date, making algorithm analysis strategies key functionalities of many active research fields, such as signal processing, bio-informatics, pattern matching, complexity analysis, special-purpose processor design, networking and communication, optimisation and data mining [Bojoi et al., 2012; Gries, 2004; Jain et al., 2001; Rashid et al., 2008; Sedgewick and Flajolet, 2013; Sistanizadeh, 1990; Srinivasan et al., 2005; Thomas and Olukotun, 2005].

In algorithm analysis, "characteristic operations" are identified and studied to understand the behavior of an algorithm [Levitin, 2007]. In terms of execution frequency, critical operations are the operations that can define the run-time behaviour of an algorithm [Sedgewick and Flajolet, 2013]. These operations are known as "basic operations" or "abstract operations" in complex analysis, and are also referred to as "characteristic operations" or "special-purpose operations" in other fields. They are expensive in cost since most of the processor time is spent on them during execution. Therefore, in algorithm analysis approaches, they are identified and optimised to obtain performance improvements in their algorithms [Galuzzi and Bertels, 2011]. For example, in real-time signal processing, filter functions (a critical functionality of signal processing) of different algorithms are compared with each other to find the most suitable filter for an application [Mili et al., 1987]. (In this context, *application* refers to an

application domain where the selected operations are going to be deploy). Here the expected performance improvement of the algorithms can be subjective to a particular application. That is, for instance, the selection of the filter function in the above example is performed based on the used parallel filter banks and their specifications which may prioritise either accuracy or speed, or both, of the output. Therefore, identification of dedicated characteristic operations can be used to improve performance of the corresponding algorithm from various perspectives.

In order to identify such operations, many approaches have been proposed in the last few decades. In spite of being critical to the algorithms, identification of characteristic operations is not a straightforward process [Amold and Corporaal, 2001]. It requires a lot of human effort and time to obtain successful results [Amold and Corporaal, 2001]. Existing approaches are often built upon either static or dynamic strategies [Galuzzi and Bertels, 2011]. Hybrid approaches with both static and dynamic strategies are not common due to their lack of generalisation or the limitations of deep analysis [Galuzzi and Bertels, 2011]. Most earlier approaches were performed manually and later they were converted to automated procedures so that considerably large programs can be handled within a reasonable time frame [Arnold and Corporaal, 1999]. However, automated approaches built on static approaches are the most commonly used to achieve good results [Galuzzi and Bertels, 2011].

Nevertheless, the main drawback of these existing approaches is that they are typically built for analysing one algorithm at a time. Heavy and complex functionalities of most existing operation identification methods might limit the feasibility of applying them for analysing multiple algorithms at a time, since obtaining results even for a single algorithm is computationally costly. On the other hand, even though some approaches concentrate on finding operations targeting multiple algorithms, general methodologies are not available for use in different application domains [Arnold and Corporaal, 1999; Fanucci et al., 2006; Shirai et al., 1990].

Therefore, as a solution, a general application analysis methodology is developed in this thesis to analyse operations of multiple algorithms simultaneously. All the operations of a given set of algorithms are analysed in terms of their execution frequency to measure their criticalness towards the run-time behaviour of the algorithm. In addition, to support the desired characteristics of a given target application, the operations are further analysed in terms of different additional properties such as size, commonality, and hardware cost. Quantified criticalness

towards such different properties of the operations will guide the selection of the most suitable set of operations to represent the objective algorithms in a target application domain.

In order to produce our methodology in a generalised way, the feasibility of calibrating the selection of characteristic operations targeting different applications is essential. Thus, the final resultant operations can be used by many different *target applications*, which effectively prioritises different properties of the operations found. This is achieved by using a weighting system. For instance, if the target application is *algorithm clustering*, the *commonality among the algorithms* of an operation would be the most important property, but it would be the *execution frequency* in a *custom instruction generation for a specific processor* application. Therefore, in the first case, *commonality* would get a higher weight, while in the latter case *execution frequency* would get a higher value in comparison to other considered properties.

Overall, the goal of our work in this thesis is to develop a methodology to identify characteristic operations for multiple algorithms and calibrate them to target different application areas. To do this, new strategies for analysing operations of multiple algorithms in terms of different properties are developed. In the development process, three main technical challenges were tackled in order to build a complete and comprehensive methodology (they are discussed in detail in Section 1.3). In addition, the computational cost of the methodology was kept light in order to handle multiple algorithms simultaneously. Therefore, simple and lightweight data structures and analysis strategies are used throughout our whole methodology.

1.2 Motivation

Identifying and improving characteristic operations using algorithm analysis is a widely used technique to improve the performance of computing systems. When the critical operations are identified successfully, many techniques are used to improve their performance; implementing hardware functional units, parallelising the execution, implementing library functions, and improving the software implementation, are a few of these techniques. These distinct strategies improve the algorithms and the computer systems they are running on, in terms of different performance variations such as speed, complexity, power consumption, hardware cost, code size, accuracy and refactoring etc [Barry et al., 1994; Halambi et al., 2002; Lin and Fei, 2010; Rinard and Diniz, 1996; Sun et al., 2004].

Improving the performance of a common functionality may significantly benefit multiple

algorithms at the same time. Algorithms define the procedure for performing a variety of functionalities in a computer system. Many different algorithms are developed and used to perform various activities. However, even when performing different functionalities, many algorithms share similar operations internally. For instance, the multiply-accumulate (MAC) operation is a frequently used micro operation in two different application areas: signal processing and graphics applications [Stelling and Oklobdzija, 1997]. Therefore, it is often implemented as a separate hardware functional unit to improve the execution speed and energy performance of its dependent algorithms. Likewise, if common operations can be identified in multiple algorithms successfully, improving the performance of the objective algorithms can be made easier and more efficient because multiple algorithms will be served at the same time instead of a single one. The performance of the algorithms may be considered from various perspectives such as their execution speed, memory usage, code size, accuracy of the results, complexity of the implementation and hardware cost.

However, the similarity of an algorithm's main functionality, or being in the same family, might not guarantee that algorithms share similar operations [Clark et al., 2005]. Therefore, determination of these similarities among algorithms may not be trivial. Consequently, to find such relations in a given set of algorithms in terms of their operations, deep analyses are required. These relations can be then used to improve the performance of multiple algorithms in numerous application areas such as designing hardware functional units, parallelizing functional units, creating library functions, algorithm clustering and identifying common "hot-spots".

Most existing approaches are designed to identify characteristic operations for a *single* algorithm [Arnold and Corporaal, 1999; Cong et al., 2004; Yang et al., 2007; Zhao and Bian, 2010; Zhao et al., 2008]. With the existence of many alternative algorithms to achieve the same functionality, limited research work has been carried out on analysis of targeting multiple algorithms. These approaches are limited, and most are found in the field of ADSP (Application Domain Specific Processors) design, where the primary goal is to identify special-purpose operations, used by several considered algorithms, to implement as custom instructions. The most significant work proposed to address this is presented by Clark et al. [2005], where special-purpose operations are identified, selected and implemented as custom instructions in an application specific processor. In addition to this, a few more approaches are presented to address this problem in the same field [Arnold and Corporaal, 1999; Fanucci et al., 2006; Shirai et al., 1990]. In contrast to the work of Clark et al., the main drawback of these approaches is the

lack of a generalised methodology applicable in different application domains. For instance, the results from Arnold and Corporaal [1999], Fanucci et al. [2006] and Shirai et al. [1990] focus on identification of operations common to multiple target algorithms. However, they do not provide a formal procedure or a process suitable for other application domains.

In contrast, the work presented by Clark et al. [2005] positively addresses this problem by presenting a general methodology to identify critical operations of a given application domain. In spite of targeting multiple algorithms, they design custom operations for the selected set of special-purpose operations from a *single* algorithm. The other considered algorithms are then generalised in such a way that they can use the already designed custom operations. Based on their experiments, this research effort shows successful results for some related algorithms which were expected to share similar special-purpose operations.

However, this might not always be the case, even for algorithms that perform the same functionality. For instance, when considering exact string matching algorithms, according to the case studies we presented in Section 5.4, the Boyer-Moore and Not-So-Naive algorithms do not share even a single special-purpose operation between them even though they perform the same functionality of matching two given strings. Therefore, the above approach does not guarantee to benefit other related algorithms which are considered to be potential targets of the designed custom instructions. This is also confirmed by the evaluation results of Clark et al. [2005].

Thus, the task of obtaining a set of special-purpose operations that are truly common to multiple algorithms remains unanswered. This thesis addresses this task, by presenting a methodology to identify such operations. Instead of identifying characteristic operations for a single algorithm, the goal of this work is to target multiple algorithms when selecting final operations. Furthermore, the new method facilitates selecting specific operations for different application domains, so that the methodology can be used as a general approach.

1.3 Problem Statement

The focus of this research is on the development of strategies to identify special-purpose operations common to multiple algorithms and calibration of the operations appropriately, in a way that they can be useful for different applications. To develop such a methodology, there are mainly three obstacles to overcome which are briefly discussed below.

First, identification of executionally expensive operations is not trivial since the run-time behaviour of operations can be difficult to examine. As discussed above, the key characteristic of the special-purpose operations is their frequency of execution. Therefore, to explore these operations accurately, our analysis needs to be capable of determining the run-time execution frequencies of candidate operations. There are many sophisticated profiling tools that have been introduced to locate hot-spots (operations that consume most of the processor cycles in run time) by dynamic analysis. The most important quality of dynamic analysis is being "precise" since no approximation or abstraction is needed [Ernst, 2003]. However, dynamic analysis depends on the data sets used at a given time and hence cannot be guaranteed to be consistent for all the cases. In addition, in our studies "precise" execution frequencies of operations might be unnecessary since the frequency values are only used to make comparisons between operations to find the most critical set. Compared to dynamic analysis, static analysis produces results that are applicable in a wide range of data sets. In existing approaches, static analysis is used more often than dynamic analysis for this reason. This research thesis also uses static analysis strategies to make its methodology reliable with different applications. However, static analysis suffers from conservativeness, which reports weaker properties than may actually be true [Ernst, 2003]. To overcome this drawback, this thesis introduces some additional strategies, which are capable of foreseeing the run-time execution of operations statically.

Secondly, when there are many algorithms to be considered, as in our case, finding common characteristic operations to all or most algorithms can be challenging because of possible implementation variations. Therefore, it is useful to understand their basic properties beforehand, and distinguish or group them accordingly. For instance, if we are to identify a characteristic operation targeting a large number of algorithms, operations common to all may not exist. Grouping selected algorithms according to their common operations helps find algorithm clusters that have common behaviours. This results in much more efficient and specific operations for the particular algorithm cluster. In such a case, we need to develop strategies to measure the closeness of algorithms in terms of their primary functionalities and cluster them accurately based on their shared operations.

Thirdly, the selection of the final set of characteristic operations needs to be calibrated for specific application domains. The final characteristic operation list must satisfy the required properties of the target applications. Therefore, we need to customise the selection of operations by prioritising the specific properties demanded by the application. For this purpose, different

kinds of characteristics of operations, important to the considered applications are identified. With regard to these characteristics, all the operations are examined by the appropriate analysis to measure the criticalness of each operation. Finally, from the quantified measurements of the properties of each operation, the most beneficial set of operations is selected as the final set of characteristic operations for the objective applications.

In summary, to create a general approach to determine the similarities between a given set of algorithms, the following three key problems arise.

1. How to statically determine the run-time frequency of specific operations in algorithms?
2. How to quantify the 'closeness' of related algorithms?
3. How to create a static analysis framework for algorithms that can be adapted easily to different applications?

Solutions to these problems form the basis of our research project.

1.4 Significance of the Research Problem

This work addresses three main problems as mentioned above. These problems can be generalised easily so that the proposed solutions can serve different application domains.

Determining the run-time frequency of an operation is a widely discussed topic in many research fields. A number of approaches have been introduced using static, dynamic or both strategies. All these approaches only consider a single algorithm in the analysis. However, most often, computer systems are designed to execute multiple algorithms (except Application Specific Integrated Circuits (ASICs)) rather than a single algorithm at a time. Therefore, in order to achieve general performance improvements of a computer system, it is important to consider all the objective algorithms, so that benefits of the improvements can be utilised by all or most of them. Obviously, analysing all target algorithms together as a single exploration will benefit the efficiency and effectiveness of the process rather than repeating the same procedure separately. Thus, as a general methodology for analysing multiple algorithms together, the analysis proposed by our work could benefit many research fields.

Algorithm clustering techniques are used to predict or understand the behaviour of similar algorithms by analysing their similar characteristics [Woodward and Swan, 2010]. To the best of

our knowledge, algorithm clustering based on characteristic operations cannot been found in the literature, although some clustering techniques consider usage of specific functionalities [Kwok and Ahmad, 1998; Ong et al., 2006; Riedl and Richter, 1996]. As these operations represent the core functionalities of an algorithm, they can be used to define the true characteristics of their corresponding algorithms. Therefore, this clustering strategy could result in more accurate and beneficial clusters of algorithms, which can be obtained for different kinds of algorithm sets.

Most commonly, special-purpose operations are selected to target one application domain in existing studies. In these methodologies, all the analysis approaches are performed with a single application domain in mind. This makes these approaches specific to a particular application. In contrast, our methodology facilitates calibration of the selection of the final operations for different types of applications. As a general approach, it can be applied to many application areas.

1.5 Contributions of this Research

In our methodology, we aim to solve the three problems discussed above, i.e. determining execution frequency of operations, quantifying relations between algorithms and calibrating the selection of characteristic operations for different applications. This thesis provides answers for the above questions by developing static based analysis processes and confirming their validity in real time executions. The main contribution of this thesis is a customisable framework to identify characteristic operations for multiple algorithms arising from different target applications. It mainly consists of three phases to provide solutions for the problems discussed above. Thus, the additional contributions of this thesis are claimed as follows;

1. A new static analysis method is proposed to determine the execution frequency of operations in a given algorithm;

2. A novel algorithm clustering method is developed to identify closely related algorithms based on their common operations; and

3. A heuristic weighting system is presented for choosing operations for different target applications.

1.6 Evaluation Criteria

We have carried out several different experiments to evaluate the steps of our framework separately and as a whole system. All these experiments were carried out in a dynamic environment to verify the accuracy of our static analysis-based strategies. This thesis discusses the evaluation criteria of each experiment with its corresponding experimental set-up.

In this research work, several sets of algorithms were used for the experiments. They were taken from different application domains such as *exact string matching algorithms* [Charras and Lecroq, 2004], *cryptographic algorithms* [Bakker, 2014], *sorting algorithms* (standard implementation) and *sequence alignment algorithms* [Needleman and Wunsch, 1970; Smith and M.S., 1981; Yu et al., 2010]. These algorithms are introduced in detail with their background information in Section 2.7.

1.7 Organisation of the Thesis

This thesis is organised as follows. Chapter 2 discusses the related work in three areas: Algorithm analysis, Algorithm clustering approach, and Priority weighting systems. In Chapter 3, the big picture of our framework is presented with brief conceptual and implementation details. Chapter 4 presents the static-based analysis for determination of the execution criticalness of the operations of an algorithm, to answer the first question in Section 1.3. The algorithm clustering approach developed as a solution for Question 2 is described in Chapter 4. To answer the third question of the problem statement, Chapter 5 presents a heuristic weighting system. It also presents three comprehensive case studies carried out to demonstrate the usability and to evaluate the accuracy of our framework as a whole. Finally, Chapter 7 draws conclusions from this research and discusses future work.

1.8 Publications Arising from this Research Work

- M. M. E. Karunarathna, Y.-C. Tian, C. Fidge, and R. Hayward. Algorithm Independent Application Analysis for Customized Instruction Identification, *Microprocessors and Microsystems*, 2014, 38(7), pp.637-648.

- M. M. E. Karunarathna, Y.-C. Tian, C. Fidge, and R. Hayward. “Algorithm clustering for multi-algorithm processor design”, *in Proceedings of the IEEE 31st International*

Conference on Computer Design (ICCD), 2013, Asheville, NC, USA, Oct, pp.451-454.

Chapter 2

Literature Review

This chapter provides a review of the literature related to our research project. The methodology developed in our research work deploys static based algorithm analysis techniques to identify characteristic operations of multiple algorithms. This chapter mainly consists of two parts: the key research work developed in the area of algorithm analysis, and algorithm clustering. As the fundamental concept of our project is to find relationships between multiple algorithms, this chapter also illustrates the importance of such relations in some existing research work. It also emphasises the importance and the lack of existing general approaches for performing algorithm analysis targeting an entire set of algorithms.

2.1 Relationships between Algorithms

With the rapid evolution of computing systems, the versatility of their applications has become enormously broad over the past decades. Their capabilities range from day-to-day routing tasks to space exploration. The tasks performed by a computer system are defined by algorithms to explain the step-by-step procedure of their execution. Since the early stages of computer science, many types of algorithms have been developed to carry out these tasks in different architectures under various limitations and performance expectations.

Most of these algorithms are related to each other, due to many reasons. Examples of such reasons are performing the same functionality or being derived from one another, or having common key operations or a similar historical background. Relationships between algorithms in terms of their internal operations are important for performing performance improvements of algorithms and/or the computer systems on which they are executed [Sedgewick and Flajolet,

2013]. Therefore, algorithm analysis strategies with a focus on investigation of operation level relations between algorithms have become a major interest of many computer science related research areas [Checconi et al., 2012; Galuzzi and Bertels, 2011; Zhou et al., 2013].

Many different kinds of problem have been solved by algorithm analysis studies through investigating relationships between multiple algorithms. Some of the questions these studies try to answer are:

- What is the best algorithm to perform a particular task?
- Which algorithm should be used to get a specific outcome under given constraints?
- How can an algorithm be improved further by combining functionalities of other existing algorithms?
- How can an algorithm be changed to get certain results?
- To what extent are given sets of algorithms related to each other?
- How can the best set of algorithms to be implemented together be found?
- What features lead them to get different results?
- What performance improvements can be achieved by similar operations?
- Can any operation in an algorithm be used to improve the performance of any other algorithm?
- What are resource, time, complexity cost comparisons?
- What is the state-of-the-art of the application domain?

These questions have been discussed in many research articles [Chaikalis et al., 2000; Checconi et al., 2012; De Pierro, 1993; Mitchell and Mancoridis, 2001; Qadeer et al., 2013; Wei et al., 2005; Zhou et al., 2013].

We discuss some such studies dedicated to the above problems. Software engineering is one such application area which utilises the identified relations between multiple programs. For instance, the work presented by Mitchell and Mancoridis [2001] decomposes software programs into clusters of subsystems to reverse engineer the structure of the software. Similarities

between source code components are investigated using algorithm analysis techniques to find a unique set of source code components and identify the structure. They deploy suitable heuristic approaches to obtain results, because such pattern matching techniques are typically NP hard. Zhou et al. [2013] showed usage of algorithm analysis to identify algorithm relations in the field of image processing. They introduced a new remote sensing image processing algorithm. The algorithm is formed by composing useful strategies of other algorithms to support new applications such as weather forecasting, territory surveying, crop assessment, forest exploration, geological prospecting, environmental protection, disaster monitoring, and urban planning. Similarities and differences of the existing algorithms were deeply analysed to form the new algorithm by enhancing the performance of the previous algorithms. Further, identification of algorithm relationships was used to improve the performance of parallel executions. For instance, relationships between graph algorithms like Breadth-First-Search (BFS) were derived to reduce architectural limits of parallel machine executions [Checconi et al., 2012]. These approaches showed either how the performance of algorithms can be improved for an existing architecture or how a new architecture could be designed for existing algorithms.

Another application area which utilises the identification of relations between algorithms is networks and communication. Jin and Tierney [2003] have analysed different algorithms using network capacity and bandwidth measurement tools to determine how system capabilities affect measurements on networks. These studies help select the most suitable tool in a given situation. Such algorithm analysis techniques are also utilised in mathematical and medical purposes where they are applicable. For instance, De Pierro [1993] proposes a general framework to facilitate both an Image Space Reconstruction Algorithm (ISRA) and an Expectation Maximization (EM) algorithm. Based on the common mathematical background, this work extends the ISRA algorithm for “penalized likelihood” similar to the EM algorithm. Similar to this, Wei et al. [2005] investigated the relations between the Filtered Back Projection (FBP) algorithm and the Back Projection (BP) algorithm. As a result, they identified a link between X-ray CT and number theory that will be useful to improve the quality of CT images in medical imaging. The work by Chaikalis et al. analysed SOVA (Soft Output Viterbi Algorithm) and log-MAP decoding algorithms to examine reconfigurations between them [Chaikalis et al., 2000]. Analysis is performed by first reviewing the key features of two classes of algorithms and then comparing their component operations for common basic blocks and procedures. This study helps select the optimum algorithm in a particular circumstance.

In addition to the above research fields, algorithm analysis techniques are extensively deployed in the field of computer architecture design. This is because algorithm analysis is one of the key phases of the process of designing special purpose processor architectures and embedded systems for multiple algorithms. This includes the DSP (Digital Signal Processor), ASIP (Application Specific Instruction-set Processor), ADSP (Application Domain Specific Processor) and Specialized accelerator design fields. For example, the work by Kim et al. designed a heterogeneous processor for multimedia applications [Kim et al., 2012]. The processor ISA is dedicated for specific operations found in embedded media applications such as image processing, vision, and 3-D graphics. The processor supports parallel execution of common time consuming operations and also consists of a Unified Filtering Unit (UFU) to support various filtering operations such as texture filtering, Gaussian filtering and various 3×3 filters. Similarly, Sreedharan and Akoglu [2008] have designed an application-specific reconfigurable Computing (ASRC) hybrid architecture for cryptographic applications. This work considers compute-intensive hash functions MD5 and SHA-1 to identify recurring computation patterns in the algorithms. Their architecture achieved execution frequency performance improvements while saving a significant circuit area cost. Qadeer et al. [2013] designed a domain-specific Convolution Engine (CE) accelerator to implement convolution-like data-flow. This special architecture supports many application domains which utilise these operations, such as computational photography, image processing and video processing. Sugita et al. [2003] generated a specific unit for image filtering and stereo matching. Using previous knowledge they used filtering and depth-test-based optimisation of the graphic hardware (GPU).

Based on the previous studies discussed above, the importance of finding relationships between multiple algorithms and their applications is clearly seen in various research fields. However, almost all existing approaches have found relationships manually, based on the researchers' background knowledge about the considered algorithms. In contrast, our target is to develop a general algorithm analysis framework to identify similar operations between multiple algorithms automatically.

2.2 Algorithm Analysis

Algorithm analysis is used to investigate the characteristics of an algorithm. It is the fundamental strategy used to explore relationships between algorithms in the literature. However, it also implies time or other resource consumption of an algorithm as in complexity analysis [Levitin,

2007]. The most commonly used algorithm analysis techniques are built on static analysis. Many static approaches can be found in complexity analysis as well [Levitin, 2007]. They also appear in custom instruction design, where special hardware components are built to target characteristic operations found in objective algorithms [Galuzzi and Bertels, 2011].

In complexity analysis, the basic operations of an algorithm are identified by statically analysing the algorithm's source code. Then, the time and space complexity of the algorithm is defined mathematically. In our work, the concept of algorithm analysis is used in order to understand the common characteristics of algorithms in terms of their critical operations. These critical operations are also known as "basic operations", "abstract operations" and "special-purpose operations", on which most of an algorithm's execution time is spent. They are critical to the algorithm due to their high execution frequencies [Galuzzi and Bertels, 2011]. Identification of such operations is important in various research areas, to draw relationships between algorithms. Therefore, some key algorithm analysis approaches for the identification of characteristic operations will be reviewed below.

In many research fields, algorithm analysis is used to analyse algorithms' characteristics for different software/hardware perspectives. This is to achieve different performance improvements, such as execution speed, accuracy, memory usage, complexity, time cost and software/hardware cost. Parallel processing is also a leading research area in which such analysis strategies are used extensively. Studies in this area support parallel execution of applications in multiple systems such as heterogeneous architectures, network processors, distributed systems and multi-core architectures [Gries, 2004; Rashid et al., 2008]. Bio-informatics is another major research field, in which algorithm analysis strategies are employed to examine the performance of different algorithms. This helps choose the most suitable algorithms in a particular case to obtain optimal results [Srinivasan et al., 2005]. Furthermore, many other research fields such as data mining [Trabelsi and Hamdy, 2010], power electronics [Bojoi et al., 2012], optimisation [Enright, 2010; Greenberg and Loh, 1974; Price, 1982] and pattern matching [Kaufmann et al., 2010; Xian-feng et al., 2010] have also deployed algorithm analysis strategies to achieve various performance improvements.

In addition to the above application areas, the field of special processor design, also requires algorithm analysis methodologies in order to find the most beneficial operations to implement

specific ISAs. While the previously discussed approaches are dedicated to a particular application domain, much research in this area has been carried out to develop general methodologies to identify characteristic operations. Since the primary target of this area is to develop efficient architectures targeting certain algorithms, their generalisation is essential to support different types of algorithms. As this is similar to our focus we discuss some previous work in this field is discussed below.

Previous approaches can be generally divided into two categories based on the concepts used in the analysis. These are static analysis and dynamic analysis approaches. Most are developed using static analysis rather than dynamic approaches. In static analysis, a program is represented in high level source code or an intermediate representation like a data flow graph (DFG), and the code is examined to identify useful features. In dynamic analysis, the target algorithm is implemented in a high level or low level language and is executed using test data sets. At run time, the required parameters such as execution time, cycle count and instruction count are measured using profiling tools. From these results, special-purpose operations of the application are identified, and hence customised instructions can be designed. Dynamic analysis depends on specific data sets and the profiling tools used. Both static and dynamic analysis can be used to effectively extract the native features of an algorithm in different ways. However, a particular weakness of dynamic analysis is its dependence on the test set used during the analysis. If this test set is not characteristic of the actual data set, then the results can be skewed. For this reason, this thesis favours static over dynamic analysis. The following two sections discuss some key research work in these two categories.

2.3 Static Analysis Approaches

In static analysis, the source code of the algorithm is statically analysed. The source code can be presented in a high level form or in a form of pseudo code. In contrast to dynamic analysis, here the algorithm does not need to be implemented and run using real data. By looking at the algorithm's structure, its characteristic operations are identified and their behaviour is studied.

In the field of custom instruction generation, static analysis is commonly used, rather than dynamic analysis [Galuzzi and Bertels, 2011]. The custom instructions are generated to target special-purpose operations in the source code. They are considered as critical operations to the algorithm in terms of their anticipated execution frequency. Identifying such operations is the

most important step of the custom instruction generation process, as the performance of new hardware solely depends on the usefulness of these operations. Although the manual approaches introduced in early research works produced good results, they require a lot of human effort and time. Therefore, many automated methods have been introduced in the recent past to perform the special-purpose operation selection phase.

The static analysis based approaches are built upon the concept of "template matching" [Arnold and Corporaal, 1999; Atasu et al., 2003; Choi et al., 1999; Clark et al., 2005; Cong et al., 2004; Ernst, 2003; Liem et al., 1994; Yang et al., 2007; Zhao and Bian, 2010; Zhao et al., 2008]. Here, the objective algorithm is presented in intermediate representations, such as data flow graphs (DFGs) and control flow graphs (CFGs). Then previously-defined or automatically-generated sub-graphs, known as templates, are generated as potential special-purpose operations. Generally, custom templates are generated incrementally starting with a seed and then merging nodes iteratively [Galuzzi and Bertels, 2011]. After that these templates are then matched throughout the original graph to find the most profitable ones to implement as customised instructions.

Liem et al. [1994] introduced a general approach for the first time to identify common operations of an algorithm based on a manual pattern matching technique. The high-level implementation of the algorithm was translated into a control/data-flow representation, which contained a distinct data-flow graph (DFG) and control-flow graph (CFG) interconnected through bindings. Their methodology facilitated behavioural data-flow, control flow and mixed data/control-flow representation of the source code. The representation sufficiently accommodated arbitrary functions, functions loosely bound to Resource Classes, and functions bound to specific units in the target architecture. Candidate operations were manually generated and matched with the other patterns by considering their data and control flow relations. Custom instruction selection is mapped to a covering problem using dynamic programing techniques [Pelegri-Llopart and Graham, 1988]. In the work of Choi et al. a similar pattern matching technique was used to select the custom instructions for DSP applications [Choi et al., 1999]. The selection problem was defined as a subset-sum problem and was able to identify multi-cycle complex instructions as well as single-cycle complex instructions. In this approach, specialty of the candidate operations was achieved by considering pre-defined sets of instructions dedicated to the target application domain. Arnold and Corporaal [1999] presented an extended pattern matching approach to investigate the critical operations of an algorithm. This approach mainly

consisted of three steps in identifying frequently occurring operations, grouping the matched patterns and implementing special functional units. To avoid the drawbacks of the candidate pattern generation by Liem et al. [1994], Arnold and Corporaal [1999] proposed their own pattern generation method without the need of pre-defining the expected patterns.

Inspired by the above key approaches, many fully automated algorithm analysis methodologies were presented in the same field during recent years. Most of these approaches proposed different techniques to reduce the complexity of the template matching problem. Such problems are often mapped to covering and isomorphism problems that $O(2^N)$ and $O(NP)$ hard in the worst case [Atasu et al., 2003; Clark et al., 2003; Galuzzi and Bertels, 2011]. For instance, Atasu et al. [2003] presented a fully automated custom instruction identification methodology considering fundamental architectural constraints, such as area and power cost. The candidate operations were selected as the maximum-speedup convex subgraphs of the considered algorithm. Thus they were able to prune the design space significantly, while achieving fast execution in the experiments. Clark et al. [2002, 2003] also proposed an automated characteristic operation identification methodology using a similar subgraph matching approach. The candidate patterns of the instructions were identified using DFG analysis and the most beneficial set of operations was selected by implementing and measuring their performance as hardware functional units under hardware constraints. Cong et al. [2004] further improved the template matching methodology to include instruction overlapping to speed up the execution performance of the resulting custom instructions. Similarly, the enumerations were performed as a sub-graph enumeration problem while pruning the problem size based on the numbers of input and outputs of operations. The selection of the custom instructions was made with the performance estimations for all the generated patterns for the algorithm.

In addition to the above key approaches, many similar research works have been conducted to improve the basic pattern matching approach and reduce the size of the problem domain under different constraints and techniques. Work done by Zhao et al. [2008], Yang et al. [2007], Lin and Fei [2010], Fanucci et al. [2006] and Shirai et al. [1990] are a few of such approaches.

Thus, these algorithm analysis methodologies have shown significantly good results by identifying characteristic operations of algorithms. However, most of these general approaches are designed to target a single algorithm, while our main focus is on identification of characteristic operations common to multiple algorithms. The work presented by Fanucci et al. [2006], Shirai

et al. [1990] and Arnold and Corporaal [1999] notably emphasized multiple algorithms instead of a single one and is thus the closest to our approach. Fanucci et al. [2006] discussed designing common customised instructions for multiple nonlinear image processing algorithms. However, in their approach, a set of special-purpose operations were found based on previous knowledge of the application. They did not introduce a process to identify previously unknown common operations. Shirai et al. [1990] performed static analysis on a basic block representation of the target algorithm and identified special-purpose operations, which can become customized instructions by considering their static frequency. Although they concentrated on designing a special purpose architecture for a set of thirteen digital signal processing algorithms, no method was proposed to identify common operations. Similarly, Arnold and Corporaal [1999] carried out static analysis on DFG graphs and proposed a template matching algorithm to identify characteristic operations. Even though their approach considers a set of benchmark algorithms from digital signal processing as the target application domain, it does not focus on identifying operations common to the objective algorithms. Thus, these approaches are unable to present a standard methodology as a general approach to identify special-purpose operations for a given set of algorithms.

2.4 Dynamic Analysis Approaches

In dynamic analysis, the algorithms are analysed in terms of their actual execution performance for a given data set. Therefore, dynamic analysis is considered to be more precise than static analysis as the actual execution details are not considered in static analysis. However, the main drawback of dynamic analysis is the inability to generalise the results obtained for one data set to another. Therefore, to get a general outcome, multiple executions are needed to cover all possible test cases. This is a highly complex and time consuming process.

Nevertheless, there are many dynamic analysis approaches for identification of critical operations of algorithms. In the field of custom instruction design, the most important characteristic of a potential operation is high execution frequency. Thus, in most dynamic analysis approaches, runtime execution is profiled. By examining the profile details, the most critical operations, which are often referred to as “hot spots”, are identified. They are then implemented as custom instructions of an application specific processor to target a particular application domain. The following are a few examples of such analysis approaches. Eusse et al. [2013] introduced a Multi-Grained Profiling (MGP) technology to support identification of hot spots of

an algorithm. They can be implemented as custom instructions of a special purpose architecture. The tool set they proposed analyses real-time execution details at different granualities, ranging from function execution counts to per-variable value history traces. To increase the generality of the approach, they produced profiling information for various cases, which they believe to be enough to identify high-level hot-spots successfully. Sang et al. [2005] presented another methodology to identify characteristic operations based on dynamic analysis. Their approach profiles the compiled code of the algorithms and identifies its "hot-spots" by examining the profile results. Once the candidate operations are identified, the selection of final custom instructions is performed using their pattern matching algorithm (MAXMISO algorithm), which is capable of identifying maximal single-output subgraphs from a program tree. Similarly, the approach proposed by Zou and Liu [2005] reduces the design space by using profile information of CDFGs (Control Data Flow Graphs) of the algorithms. The algorithm analysis technique presented by Hubert and Stabernack [2009] focused on memory access, performance and power usage. The profiling data of these parameters are monitored when analysing an application. The works presented by Nery et al. [2014], Nohl et al. [2003], Hoffmann et al. [2005], Kun et al. [2006], Sun et al. [2004] and Mbaye et al. [2005] are also similar algorithm analysis approaches, which are capable of reducing the design space of the typical pattern matching techniques.

Thus, approaches based on dynamic analysis techniques also have produced good results in the application analysis stage. However, as mentioned above, the lack of generalisation of the approach to arbitrary data sets prevents us from using them as a general approach for analysing many different types of algorithms. Even though they reduce the design space of a typical static based pattern matching analysis, obtaining a general solution is highly time consuming and complex due to the fact that such a solution needs to cover all possible execution alternatives. Therefore, as mentioned in Chapter 1, we favour static analysis techniques over dynamic ones in our analysis development.

2.5 Summary of Algorithm Analysis Related Work

As discussed above, there is a wide range of applications for identification of relations between multiple algorithms. Within the many research works carried out in various fields, the area of special-purpose process design has been of enormous interest. This related work can be divided mainly into two categories: Static analysis based and dynamic analysis based approaches. In static analysis, the concept of template matching was commonly used. In this analysis process

the considered algorithms are converted into a tree structure to match the operations with each other as sub-graphs of the tree [Galuzzi and Bertels, 2011]. Although these proposed static analysis approaches can produce good results, they suffer from high computational complexity [Galuzzi and Bertels, 2011] as well as the challenge of producing the templates themselves. Dynamic analysis was performed by executing the algorithms for different datasets in the run time environment [Eusse et al., 2013; Hoffmann et al., 2005; Hubert and Stabernack, 2009; Kun et al., 2006; Mbaye et al., 2005; Nery et al., 2014; Nohl et al., 2003; Sang et al., 2005; Sun et al., 2004]. The main drawback of dynamic techniques is their high dependence on the specific applications considered and the specific test data set used. In addition, the key static and dynamic approaches have only considered a single algorithm in an attempt to find characteristics of an application area which typically consisted of many alternative algorithms. Since our main target is to perform analysis targeting multiple algorithms, these approaches cannot be employed for our purposes. Therefore, we develop new analysis strategies to find critical operations from multiple algorithms, when their criticalness is defined by the specifications of target applications.

2.6 Algorithm Clustering

In addition to algorithm analysis techniques, an algorithm clustering methodology will be introduced in Chapter 4. Therefore, a detailed literature survey was conducted in this area to understand the useful techniques available in existing methodologies.

In general, algorithm clustering has been used to help understand the properties of algorithms and predict their behaviour [Woodward and Swan, 2010]. So many algorithm clustering methods have been introduced. In image processing, for instance, Zampirolli and Lotufo [2000] classified *distance transformation* algorithms into three categories, parallel, sequential raster (and anti-raster), and propagation, based on how the pixels of images are scanned to perform distance transformation. In order to support selection of a *memetic algorithm* for particular tasks, Ong et al. [2006] proposed a mechanism based on adaption. They performed clustering by considering the adaption type and adaption level of memetic algorithms. Targeting the same objective of choosing a suitable algorithm for a given task, Riedl and Richter [1996] presented a *load distribution algorithm* clustering methodology. The algorithms are categorised with consideration of five criteria: objectives, the type and amount of information used, the source of the distribution, the parameter time, and the initiating instance. This allows the user to compare

and select a suitable load distribution procedure for their problem. Kwok and Ahmad [1998] clustered graph scheduling algorithms into different groups by considering their assumptions and functionalities in order to understand their performance, using a 'fair' set of benchmarks.

Although these approaches have produced good results in their particular application area, they are limited in flexibility. They are based on specific properties of particular types of algorithms and hence do not provide a general solution for other application domains.

Mehrotra et al. [2009] introduced a method of algorithm clustering using the worst case execution time of algorithms. This method classifies algorithms into two groups: finite and infinite execution time, without execution of large test cases. They used both high-level and low-level static analysis on operations and functions of algorithms, grouped using a threshold value. However as their focus was on dividing the selected algorithms into two categories, based on dynamic performance, this approach is not applicable for our purposes.

By contrast, our approach does not consider algorithm properties such as predicted execution times, data usage, etc, nor does it rely on subjectively-assessed properties. Instead, it clusters algorithms purely on the basis of shared functionality, expressed in terms of the 'special-purpose operations' that best characterise the algorithms. This does not aim to help understand the algorithms or predict their behavior, but to determine how the algorithms are related to one another in terms of the computations they embody.

With respect to the process of "clustering" itself, there is a rich body of literature on data clustering algorithms that can be divided mainly into partitional and hierarchical clustering. However, existing data clustering algorithms were not suitable to use in our work due to various reasons such as high complexity, inability to handle high dimensional data and the requirement to pre-specify the number of clusters [Xu and Wunsch, 2005]. Therefore, we developed a new clustering algorithm, based on K-means clustering concepts [MacQueen, 1967], but avoiding its native drawbacks, such as rapidly growing complexity with high dimensional data and the need to pre-specify the number of resultant clusters.

2.7 Algorithms Investigated in this Thesis

This section introduces a few sets of algorithms, investigated in various evaluation experiments throughout our research work, including Exact string matching algorithms, Cryptographic algorithms, Sequence alignment algorithms and sorting algorithms. These algorithms were implemented in C/C++ and converted to assembly or other intermediate representation as per the requirements of experiments. The fundamental functionalities of these algorithms are outlined below.

2.7.1 Exact String Matching Algorithms

Exact string matching algorithms have been extensively used in many research areas for text processing purposes such as information retrieval, natural language processing, data mining and bioinformatics. These algorithms have been implemented since the early stages of computing science. Thus, many alternative algorithms are developed to match two strings known as "pattern" and "text", until all occurrences of the *pattern* are found in the *text* [Fischer and Paterson, 1974]. Table 2.1 lists the algorithms investigated in our experiments. The C/C++ implementations of these algorithms were taken from the *Handbook of Exact String Matching Algorithms* [Charras and Lecroq, 2004].

In this thesis, experiments performed with string matching algorithms are used to support the development of the analysis strategies. In Phase 1 and 2 (Chapters 4 and 5 respectively) of our framework, where various static analysis and clustering techniques are employed, the successfulness of the results are primarily measured using all or groups of string matching algorithms. In addition, these algorithms are also used to evaluate the weighting system (Phase 3) and finally the framework as a whole. Relations between these algorithms in terms of historical and experimental evidence are further discussed with corresponding experimental results.

2.7.2 Cryptographic Algorithms

In addition to above string matching algorithms, eight cryptographic algorithms are investigated in the dynamic experiments in this research work. These are different types of commonly used ciphers and differ from public-key cryptography algorithms such as RSA. They are employed here to evaluate the results of individual steps and to demonstrate the utilisation of our automated system. The C/C++ implementations of these algorithms are taken from the open source

Table 2.1: String matching algorithms analysed.

Algorithm	Abbreviation
Apostolico-Crochemore	AC
Apostolico-Giancarlo	AG
Boyer-Moore	BM
Brute Force	BF
Colussi	CL
Galil-Giancarlo	GG
Horspool	HP
Knuth-Morris-Pratt	KMP
Morris-Pratt	MP
Not So Naive	NSN
Raita	RT
Shift Or	SO
Turbo Boyer-Moore	TBM
Zhu-Takaoka	ZT

PolarSSL library [Bakker, 2014].

Eight algorithms are formed by using different combinations of five fundamental cryptographic algorithms (AES, BLOWFISH, CAMELLIA, DES and DES3) and two hash functions (MD5 and SHA1). "Cryptography" is the study of mathematical techniques related to aspects of information security such as confidentiality, data integrity, authentication and non-repudiation. Here, "Confidentiality" is the property of protecting the content of information from all users other than the ones intended by the legal owner of the information. "Data integrity" is the property of protecting information from alteration by unauthorized users. "non-repudiation" is the property of ensuring that entities that have committed to an action cannot deny that commitment at a later time [Menezes et al., 1996]. Cryptography is a broad application area which plays a major role especially in the field of information security in computer based systems. There are many algorithms that have been introduced in the field during last few decades. Among them, we have selected above widely used algorithms and their different modes for our case study (listed in Table 2.2).

2.7.3 Sequence Alignment Algorithms

Three sequence alignment algorithms (listed in Table 2.3) were used to evaluate the techniques in our framework. They were basically employed to test the flexibility of our system by adding

Table 2.2: List of Cryptographic algorithms used in the case study

Algorithm	**Abbreviation**
AES in CBC mode with MD5 hash	AES CBC MD5
AES in CTR mode with SHA	AES CTR SHA
BLOWFISH in CBC mode with MD5	BLOWFISH CBC MD5
BLOWFISH in CTR mode with SHA	BLOWFISH CTR SHA
CAMELLIA in CBC mode with SHA	CAMELLIA CBC SHA
CAMELLIA in CTR mode with MD5	CAMELLIA CTR MD5
3-DES in CBC mode with SHA	3-DES CBC SHA
DES in CBC mode with MD5	DES CBC MD5

more experimental case studies. These algorithms are also used to evaluate the accuracy of our algorithm clustering approach, as presented in Chapter 5. For the experiments, the standard algorithms were implemented in C/C++ and converted to assembly level when required.

Table 2.3: Algorithms used in the case study in Section 5.4.2.

	Algorithm	**Abbreviation**
Sorting Algorithms	Bubble Sort	BS
	Quick Sort	QS
	Shell Sort	SHS
	Selection Sort	SL
Sequence Alignment Algorithms	CV Tree	CV
	Needleman-Wunsch	NW
	Smith-Waterman	SW

Sequence alignment is a way of aligning strings of characters to investigate similarities or differences between them. In bioinformatics, these algorithms are extensively used to identify similar regions of DNA, RNA or protein sequences, which can be consequences of functional, structural, or evolutionary relationships [Mount, 2004]. In addition, usage of these algorithms is significant in other fields such as natural language or financial data processing.

The experiments are performed for well-known *sequence alignment* algorithms: *NeedlemanWunsch* [Needleman and Wunsch, 1970], *SmithWaterman* [Smith and M.S., 1981], and *Composition Vector Tree* (CV Tree) [Yu et al., 2010]. The Needleman-Wunch and the Smith-Waterman algorithms perform the matching phase using a pairwise alignment method (based on dynamic programing concepts). It finds the best-matching piecewise (the SmithWaterman algorithm) and global (the Needleman-Wunch algorithm) alignments of two query sequences.

On the other hand, the CV Tree method follows an alignment free method to compare genome sequences by creating composition vectors to represent them. Then the comparisons are performed over these vectors by calculating distances between each pair of sequences.

2.7.4 Sorting Algorithms

Four sorting algorithms are used in a case study to evaluate our algorithm clustering approach. The task of a sorting algorithm is manipulating elements of a list in a certain order. Various sorting algorithms are introduced, targeting different performance aspects such as complexity, speed, accuracy, memory usage, hardware usage, etc. In our experiment, we have selected four well-known sorting algorithms as listed in Table 2.3. The standard implementation of these algorithms in C/C++ is used in the case study described in Section 5.5.

Chapter 3

A Customisable Framework for Identification of Similarities between Multiple Algorithms

---

The primary target of our research is developing a methodology to identify similar operations from a given set of multiple algorithms. In the process of developing such a methodology, three obstacles were identified. In our work, each of these obstacles was separately addressed by developing suitable strategies, and evaluated using comprehensive case studies. Thus, our methodology consists of three phases, which can be customised depending on the considered algorithms or the target application/s. Figure 3.1 illustrates how these three phases (coloured) are connected to each other as a customisable framework to achieve the above ultimate goal. The task of this chapter is to present the big picture of this framework. This chapter thus introduces our framework with its *basic concepts* and *abstract implementation* details. A detailed description of each phase is presented in Chapters 4, 5 and 6.

3.1 Overview of Common Operation Identification Framework

The first phase of the framework introduces two static analysis techniques. Similar operations among multiple algorithms are used to improve the various performance factors of all the considered algorithms at the same time. The execution frequency of operations is a key property to guide the selection of such beneficial operations. In the first phase of the framework, identification of similar, special-purpose operations is targeted. To achieve this, two static analysis strategies called "sequential code analysis" and "nesting level analysis" were employed. Using our static analysis strategies, all valid operations of algorithms are analysed to determine their execution criticalness quantitatively. These measurements are then used to identify the common

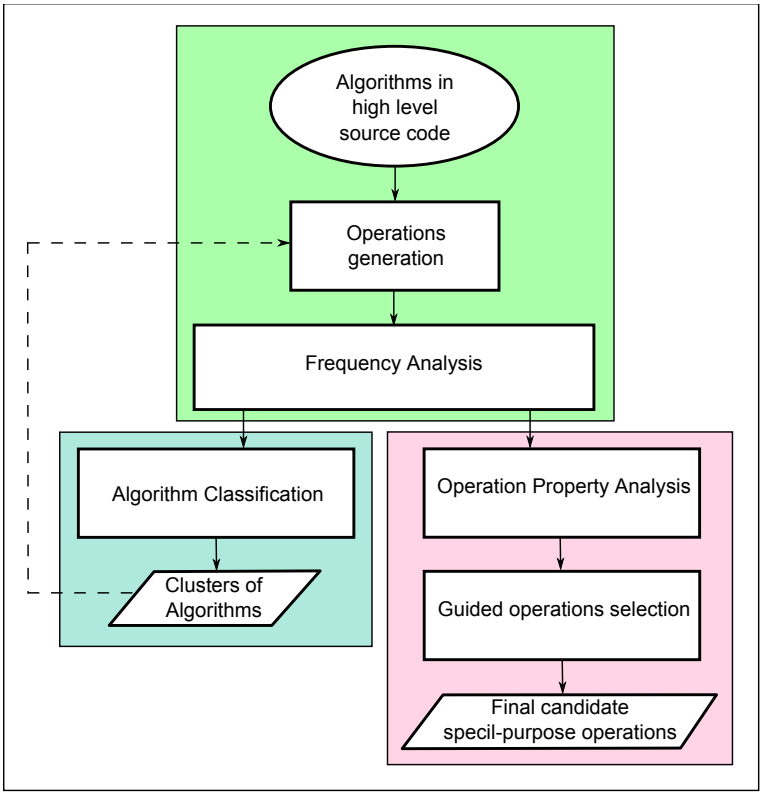


Figure 3.1: Abstract Overview of the Framework

special-purpose operations of the considered algorithms targeting a given application. The flow of this phase is shown in Figure 3.2. Since these analyses are the core of the framework, the next two phases are dependent on this phase.

In the second phase, the given multiple algorithms are pre-analysed to form groups of algorithms considering similar characteristic operations. According to the work presented by Clark et al. [2005], it can be demonstrated that operations selected for one benchmark poorly affect the performance of other benchmarks, even within the same application domain. That is, an operation selected for a particular algorithm is not guaranteed to benefit other algorithms, even when they perform the same functionality. Since the main focus of our work is identifying operations for multiple algorithms, rather than a single algorithm, the selected operations must represent all of the considered algorithms. However, as mentioned, finding similar functionalities among a set of algorithms is not trivial.

Our clustering approach considers common operations to form clusters. Therefore, the resultant clusters are guaranteed to share characteristic operations. It also allows the user to define the expected closeness between algorithms in a cluster, based on the requirements of a particular problem. These resultant algorithm groups can be then used to select the most suitable set of

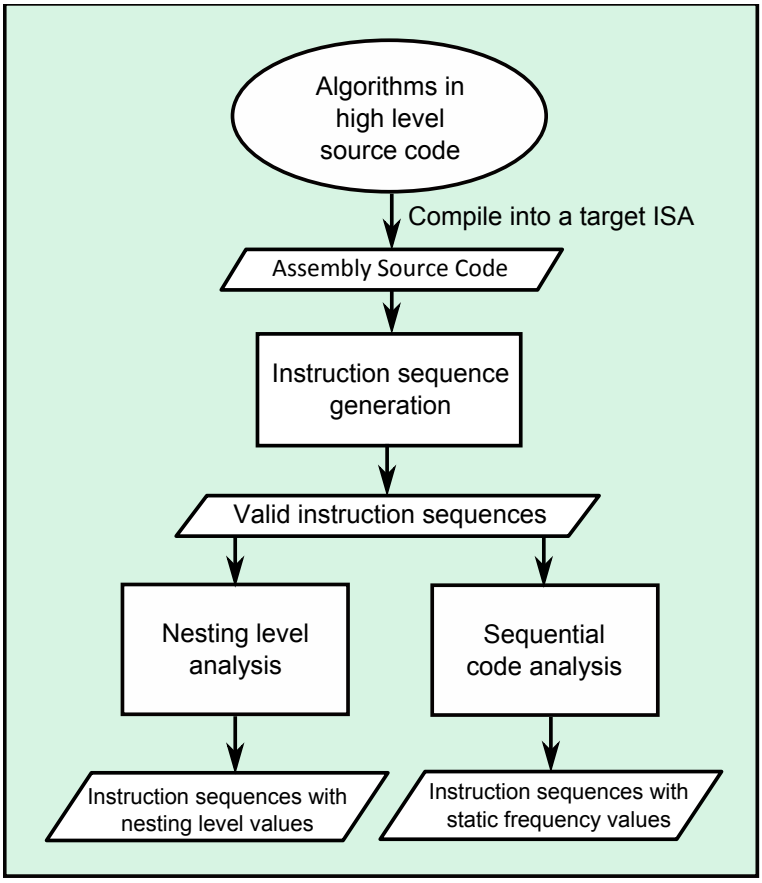


Figure 3.2: Static Frequency Analysis Approach

operations for different applications. Here, the frequency analysis steps developed in the first phase are used to select suitable operations to represent the algorithms in the clustering process. In addition, this algorithm clustering approach can be used for various other purposes as a stand-alone algorithm clustering method when the algorithms and their characteristic operations are given. Furthermore, our framework can be modified to make this clustering an optional step when not required (in cases in which only a few algorithms are considered). Figure 3.3 shows the steps involved in this phase with the core functionalities of the framework.

Phase three incorporates the insights of a weighting system to calibrate the characteristic operation selection process. It is capable of selecting the most appropriate set of characteristic operations from a set of given algorithms, targeting different application areas. As mentioned above, in the first phase, all valid operations of a considered set of algorithms are analysed to predict the execution frequency of each operation. Characteristic operations then can be easily found by ranking them, based on numerical scores for the operations. However, in addition to execution frequency, there are many other properties used to describe the characteristics of an operation, such as its implementation size, software/hardware cost, implementation complexity

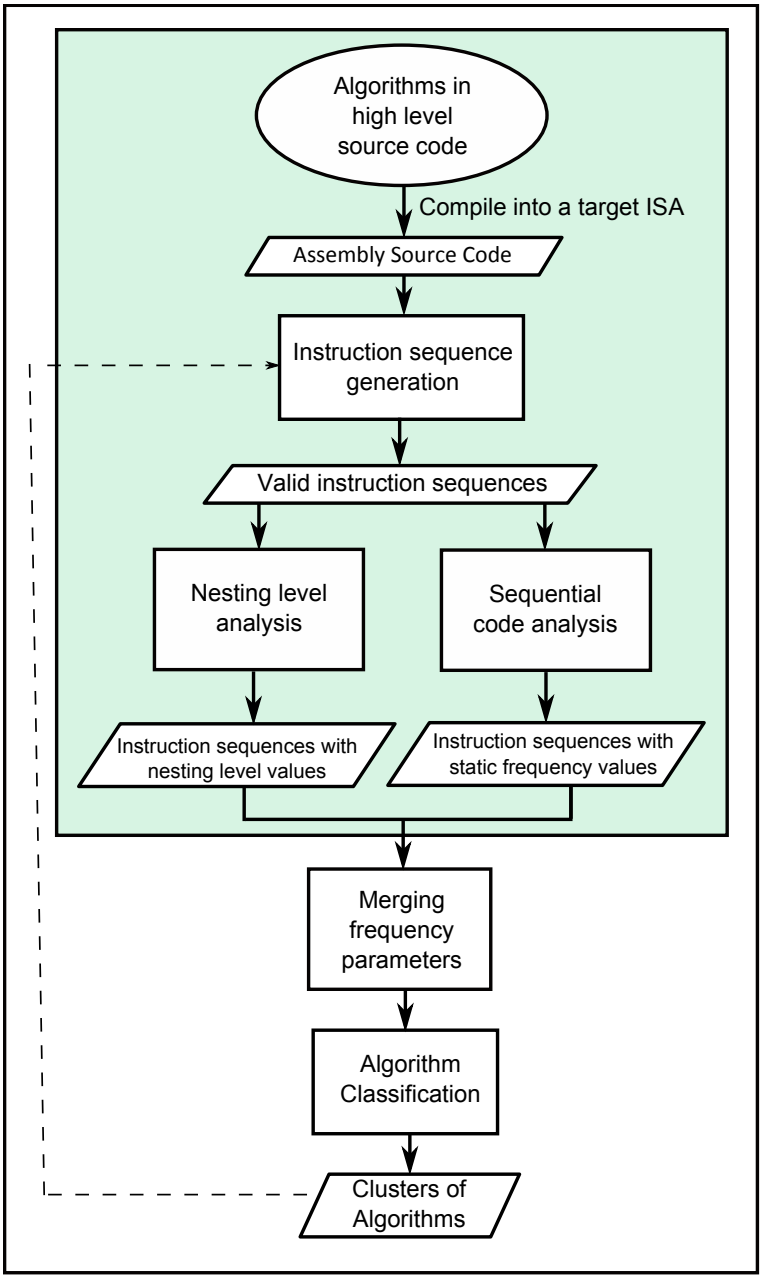


Figure 3.3: Clustering Approach Overview

and commonality. When selecting characteristic operations targeting a particular application, these properties can be used to calibrate the resultant set of characteristic operations. Therefore, the calibration needs to be done based on the properties of operations preferred by the target application. In our approach, this calibration is performed using a weighting system, which facilitates prioritising the desirable properties by assigning suitable weightings. For instance, in a library function generation application, the property of the *size of the operations* will be given a higher priority (hence weight) than the hardware cost, while in a custom instruction generation application, the priorities of these two properties will be reversed. Thus, in this phase, we

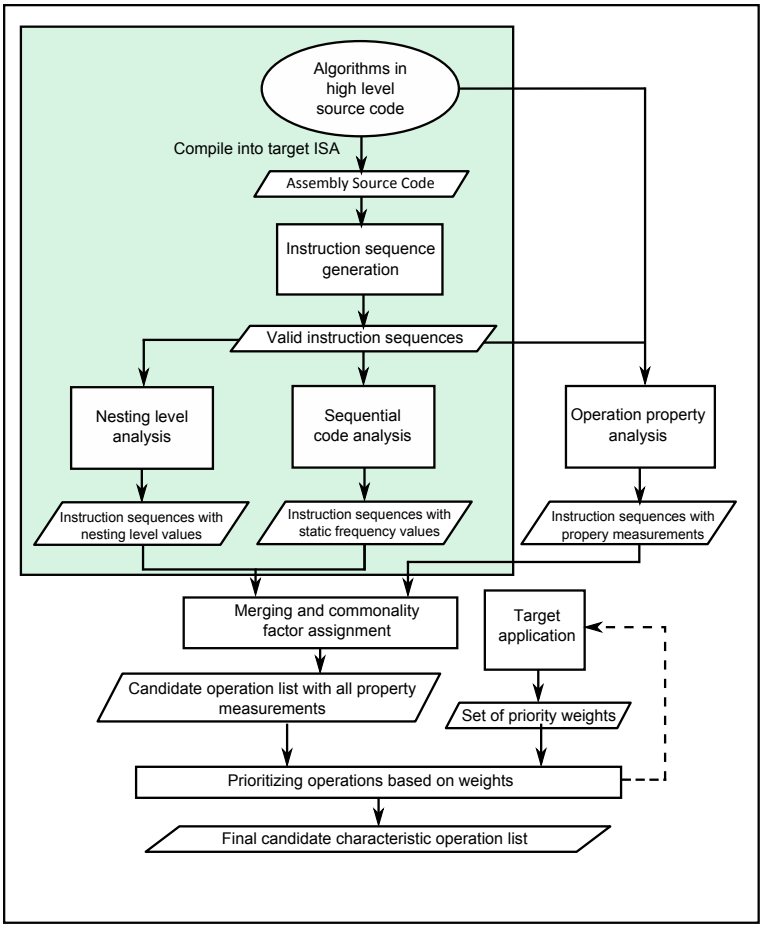


Figure 3.4: Weighting System Overview

perform extended analyses of operations in terms of different properties of operations. This phase is illustrated in Figure 3.4 with the core framework.

The framework integrates the above three phases together to form the overall framework. The intermediate steps were added to help in passing input and output data between the major steps. An overview of this integrated system is shown in Figure 3.5. A detailed description of Phase 1: static analysis strategies to investigate execution frequency criticalness of operations, Phase 2: algorithm clustering methodology to group sets of algorithms based on their common operations and Phase 3: weighting system to calibrate the selection of characteristic operations, are discussed in detail in Chapters 3, 4 and 5 respectively.

The framework is implemented in C/C++ to carry out all its functionalities automatically. It takes a set of algorithms implemented in high level source code (in this case C/C++) as input and produces all their possible operations with different property measurements. The process produces all unique candidate instruction sequence lists to represent all valid operations. It also analyses operations with regards to six different properties such as the frequency, size, hardware

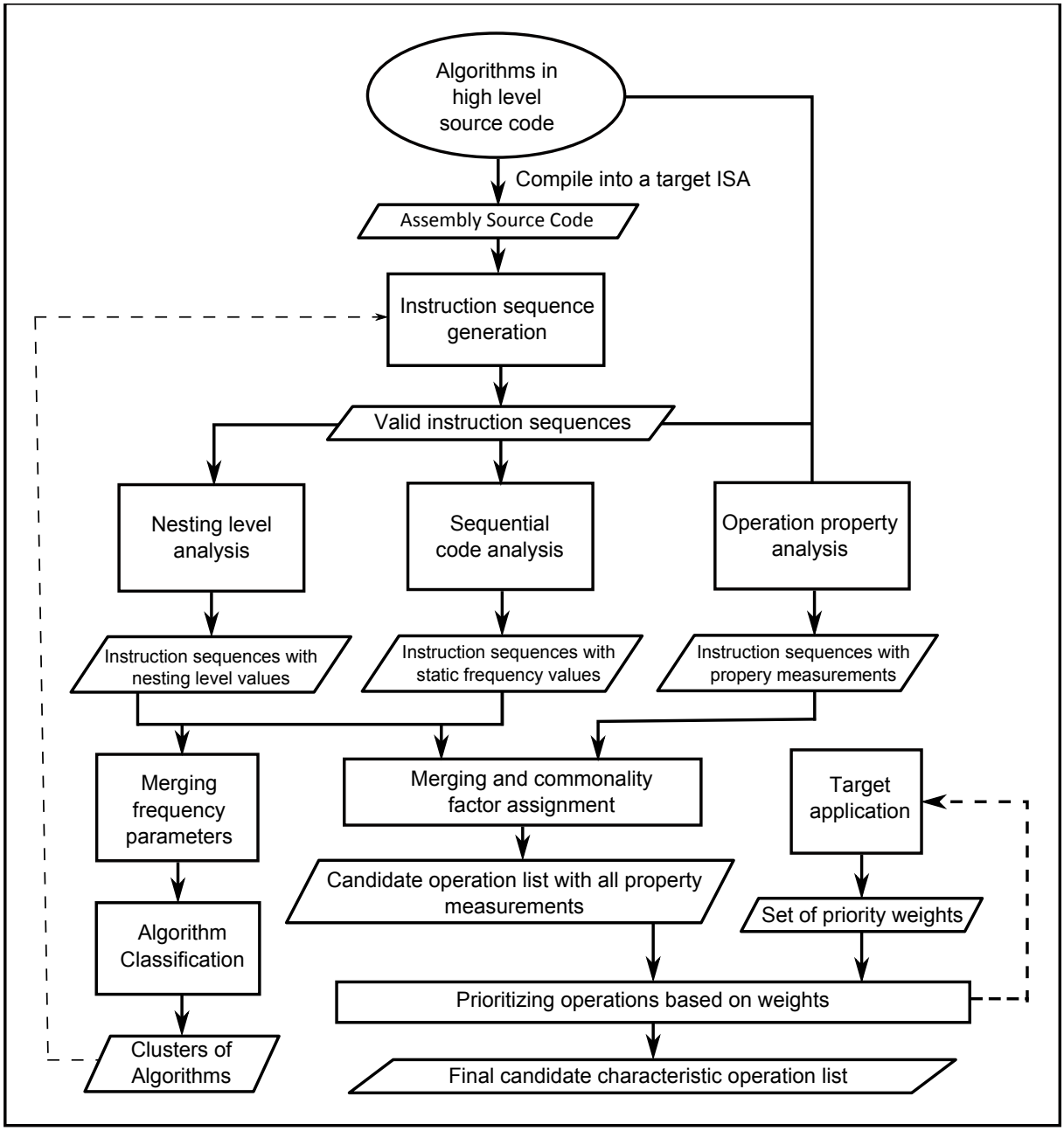


Figure 3.5: Overview of the Framework

cost, etc. The most profitable operations can then be selected by prioritised sorting on desired properties. Its general behaviour supports various types of algorithms and different application domains.

3.2 Overview of Steps Involved in the Framework

This framework is a collection of individual steps designed to perform various tasks. Different tools were utilised or built to perform each of them. Many software tools were developed in C/C++ according to our requirements. We also used several instances of existing software such

as a pretty printer and the Xtensa Xplorer simulator [Tensilica, 2014]. In the integration of the phases as a single system, steps which can be performed together were implemented together. This helped reduce implementation and execution complexity of the system. An overview of the tools we used in each step is illustrated in Figure 3.6. The following sections discuss the high level implementation details of each of these steps. The purpose of the following section is to present the abstract overview and show how they connect with each other to form a single system.

3.3 Generation of Atomic Instruction Sequences

As per Figure 3.5, the initial step of the process is compiling the high level source code into a target ISA to obtain the assembly instruction code. In our case studies, a GCC compiler for Tensilica's Xtensa ISA is used with all compiler level optimisations turned off. This is done to directly allow mapping assembly instructions to corresponding high level source code later in the process. The filtered assembly source code is then used to generate all possible operations involved in a given algorithm. In the operation generation process, the semantics of the instruction sequences are examined to avoid resulting potential false-positive valid operations.

3.4 Static Frequency Analysis Steps

According to the framework diagram shown in Figure 3.5, the next step is analysing the generated operations for different properties. The output of the previous step was the atomic instruction sequences generated for all the considered algorithms, separately. These operations then can be used to analyse their characteristics. In order to determine characteristic operations of an algorithm, their execution frequency is the key property to consider. Thus, the next step of the framework performs analysis of the criticalness of the operations. As described previously, two analysis strategies are performed. These analyses can be performed in parallel or sequentially in any order, since they are independent to each other. They are briefly introduced in the following subsections.

3.4.1 Sequential Code Analysis

Once all possible valid instruction sequences are generated, different property analysis strategies can be carried out on operations. *Sequential Code Analysis* is one of the key analysis steps of the framework, designed to measure the operations' execution frequency. In this

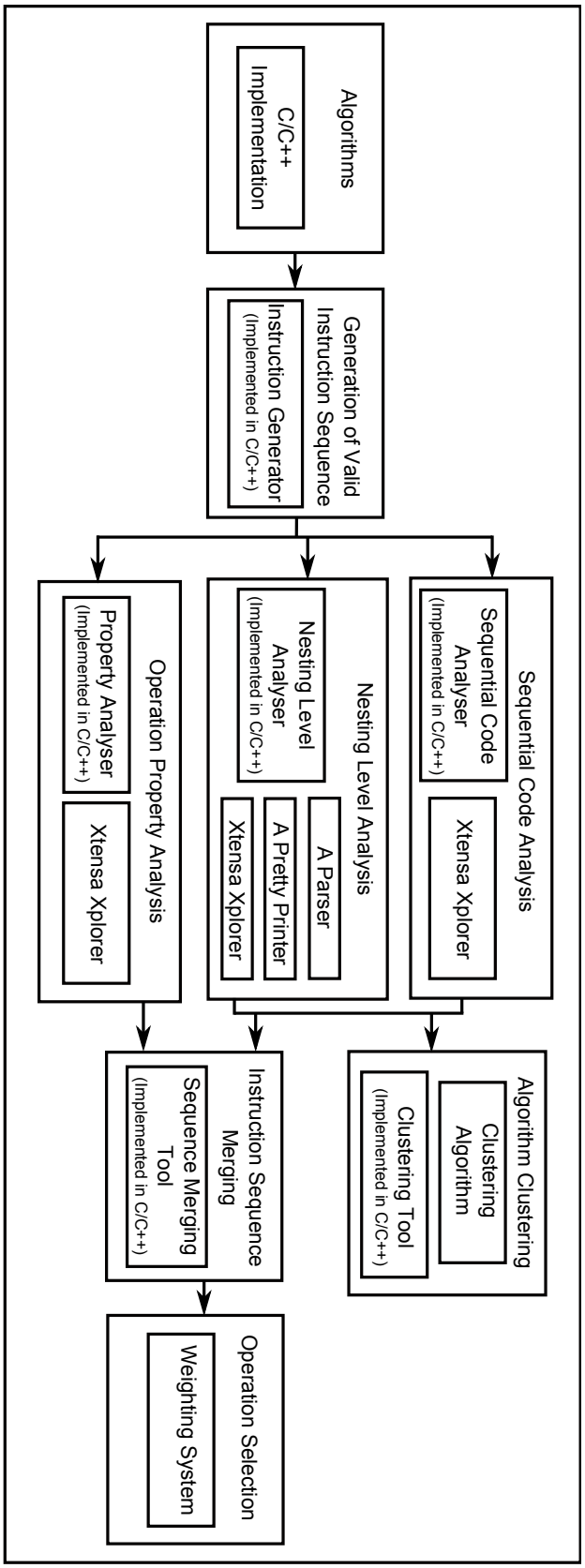


Figure 3.6: Overview of the tools used in the steps of the framework

step, the static frequency of each operation in the program code is counted automatically. All the generated valid instruction sequences from the above step are considered as candidate operations. For each operation, the repetitive occurrences within each algorithm are counted by matching them with each other. Thus, the output of this step is all valid instruction sequences with their static frequency measurements.

3.4.2 Nesting Level Analysis

Nesting Level Analysis is the other analysis performed to determine the criticalness of operations in terms of their execution frequency. If an operation is nested within loops it is most likely to execute frequently at run time. Therefore, it can become a critical operation of the algorithm as the behaviour of the whole algorithm may depend on it. This analysis takes valid instruction sequences generated for all considered algorithms as input and outputs these operations with their corresponding nesting levels.

Determining nesting levels in assembly instructions is not a straightforward process. This is because loops and iterative codes cannot be clearly identified where they are converted to a sequence of assembly instructions. Therefore, first, nesting levels of the algorithm are marked by examining their high level implementation. Then the high level source code is mapped with the corresponding assembly instructions to obtain the corresponding nesting level. Since the assembly instructions are obtained with compiler optimisations turned off, the high level to assembly level mapping can be performed unambiguously.

3.5 Operation Property Analysis

In addition to execution frequency, many other properties define the characteristics of an operation. The execution frequency is the key property for an operation to become critical to a particular algorithm. However, additional properties support specialising the operations towards a given application. In this step, the operations of the objective algorithms are analysed in terms of such properties. As mentioned in Chapter 1, our case studies produce three sets of characteristic operations targeting three applications: library function generation, custom instruction generation and execution performance improvement (presented in Chapter 6 in detail). For these three applications, in addition to execution frequency, we chose four more properties of operations to calibrate the selection process of the characteristic operations. They are operation size, commonality, number of input/output modes and completeness. These

properties are considered based on the literature and our knowledge about target applications. Since our frequency analyses are built on text-based syntactic strategies, the above properties can be easily analysed at the same time automatically. Therefore, the added time and complexity of the framework is significantly reduced by avoiding repetitive tasks common to the property analysis steps.

3.6 Intermediate Results

This step merges the results of the analysis steps with sets of unique operations of algorithms. According to the illustration in Figure 3.5, the results of the above analysis steps are valid operations with nesting level scores, static frequency counts, size, completeness and number of modes. These operations lists contain the same set of operations with different parameter measurements, as they considered the same set of algorithms. All these analysis techniques produce separate lists for different algorithms with unique sequences found in each algorithm. Here, the similar operations are merged together while recording corresponding property measurements.

According to the flow of our framework (shown in Figure 3.5), the operations lists of algorithms can be mainly used for two different purposes. One is selecting the most suitable set of operations targeting different applications directly. The other option leads to clustering of algorithms based on commonly executed operations within the given set of algorithms. This can be considered as an optional step, especially, when a large number of algorithms presents or algorithms without any background knowledge, need to be classified. Once the clusters of algorithms are formed they can be analysed against different characteristics of the operations for the target application.

3.7 Algorithm Clustering Approach

Once the intermediate results are obtained, *algorithm clustering* is the optional path to use these results effectively. From the frequency analysis steps, *Sequential Code Analysis* and *Nesting Level analysis*, each operation is measured in terms of their execution frequency. By combining the results obtained for an algorithm in the analysis, we can retrieve the most frequently executed operations. Thus for all the considered algorithms, the characteristic operations in terms of execution frequency can be identified. Using these operations, a clustering is performed to group algorithms that share similar operations.

The process of clustering is not straightforward, especially when there are many algorithms to consider. This is because the clustering involves many computations to make accurate comparisons. Therefore, we have developed a tool to automate the clustering process using C/C++. It is capable of forming clusters when the characteristic operations and the objective algorithms are given. The tool represents the relations between algorithms and their characteristic operations in a 2-dimensional table. From these data, distances between each pair of algorithms are calculated in terms of their characteristic operations. Considering these distance values, algorithms which share expected similar operations (defined by the user) are grouped together.

The resultant clusters of algorithms then can be used for different property analyses. Hence the characteristic operations dedicated to different application domains can be identified by following the procedure of our framework. In order to evaluate the accuracy of our algorithm clustering methodology, case studies and evaluative experiments were performed in a dynamic environment. This clustering approach and its experimental results are presented in detail in Chapter 5.

3.8 Instruction Sequence Merging and Commonality Factor Assignment

Once the intermediate results are obtained, the next task is to merge and determine the commonalities of operations. In order to make comparisons between operations in the selection process, all unique operations need to be identified by performing “intra” and “inter” merging. As the above property analysis is performed separately for the algorithms, the generation of unique sequences is done within algorithms (intra) and across algorithms (inter) as well. When all possible operations are generated by reading sequences of assembly instructions, it is obvious that many identical sequences are generated. Therefore, in the merging step, unique operations of algorithms and hence, among a whole set of algorithms are generated.

During the intra merging, the identical operations form a single operation with collective measurement results. That is, considering property scores, a single value is appropriately calculated to represent all instances of an operation throughout the algorithm. For instance, the *static frequency* for such an operation is calculated by adding all the frequencies of all its instances.

Likewise in inter merging, a unique set of operations from all algorithms is obtained. In this

case, unique operations of each algorithm are matched with those of other algorithms. Similar to intra merging, when identical operations are found, one instance of these operations is included in the final unique operation list with its property measurements, except for commonality. The commonality factor (describes commonality of an operation within the set of considered algorithms) is obtained during the matching phase, by keeping track of the number of algorithms by which an operation is shared.

3.9 Characteristic Operation Selection

This is the final step of the process where the relations between multiple algorithms are determined in terms of special-purpose operations. Throughout the process many analysis strategies are performed to characterise the operations for different applications. The degree of importance of each property of each operation is denoted by numerical values scored from these analysis steps. Therefore, selection of the best set of operations for a particular application can be done by prioritising specific parameters favorable to the application. This functionality is supported by our weighting system implemented in the framework.

Thus the resultant operations of this phase will be the final set of operations which are specifically critical for a particular application. From the case studies and evaluational experiments carried out throughout the thesis, it is shown that our framework can be applied to different algorithm sets and different applications. In order to demonstrate and to evaluate our framework, several case studies were carried out using dynamic analysis. They are presented in Chapter 4, 5 and 6 where the development of our framework is further described in detail.

Chapter 4

A Static Analysis Approach for Analysing Run-time Execution Frequency of Operations

---

Common functionalities between multiple algorithms are used to improve various performance perspectives of the algorithms and computer systems they are running on. These commonalities among the given algorithms are investigated in terms of their characteristic operations, which are the most frequently executed operations of an algorithm. As discussed previously, determining the run-time execution of the operations using static analysis is a challenge. This chapter addresses this challenge by introducing our static based analysis strategies. They are capable of foreseeing the run-time execution frequencies of the operations. These analysis strategies are the core of our framework as identification of executionally critical operations is a key essential.

In contrast to existing methodologies, we favour "shallow" syntactic "text-based" analysis concepts in order to reduce the computational complexity of the problem. Several evaluative experiments were carried out in a dynamic environment to analyse the accuracy of our new strategies for different application domains. This chapter presents solutions for the first question raised in Section 1.3. The fundamental concepts and implementation details of phase one of our framework is discussed in detail.

4.1 Background of Characteristic Operation Identification

Algorithm analysis techniques are extensively used in many research areas. Especially, in the field of custom instruction generation, there is a rich body of literature compared to other areas, such as complexity analysis and algorithm classification. Custom instruction design is a major

part of the field of application specific processor design. It is a leading research field that provides cost effective performance improvements for multiple applications. During the last few decades, many comprehensive algorithm analysis approaches have been introduced. As stated previously, most of these generalised approaches are built on static analysis.

These approaches use the concept of template matching in the analysis [Galuzzi and Bertels, 2011]. They statically analyse the compiled algorithm at the assembly instruction level or some other intermediate representation. All operations of an algorithm are expressed in sequences of basic instructions of a particular ISA. Most commonly, the compiled code of the algorithm is represented as a Control Flow Graph (CFG). A CFG graph consists of nodes of basic blocks and directed edges to describe control flow between them. Each basic block of a CFG can be further represented as a Data Flow Graph (DFG). Each instruction (atomic operation) is denoted by nodes, and the data dependencies of their operands (the data flow) are denoted by edges. A collection of instructions and hence the corresponding sub-graph of the algorithm is known as a *template*. A selected set of such templates is called a *library of templates* [Galuzzi and Bertels, 2011].

To perform the template matching, a library of templates is generated based on the objective algorithm. Then these templates are matched with all possible clusters of instructions represented by sub-graphs of the program. The most frequently occurring sub-graphs are selected as a candidate list of critical operations of the algorithm.

The template matching analysis can be categorised into two groups: pre-defined templates and custom templates [Galuzzi and Bertels, 2011], based on the method of generating their template libraries. In the case of pre-defined templates, a set of previously selected templates are considered as the template library. These pre-defined templates are selected by considering the relevant background knowledge of the application domain. For custom templates, a set of operation templates are generated automatically, based on their target algorithm and other application requirements [Galuzzi and Bertels, 2011].

The template matching method is capable of analysing an algorithm deeply at the instruction level and identifying frequently executed operations accurately. These properties of the method are achieved by its graph representation of the relations between operations and operands semantically. This makes it possible to identify true operations with relevant data flow, which typically span several assembler instructions. However, in the sub-graph matching phase,

the number of possible sub-graphs increases exponentially and hence it becomes highly time consuming [Galuzzi and Bertels, 2011]. Therefore, template matching approaches are often comparable to graph isomorphism and covering problems, which are well known to be computationally complex (NP hard) problems [Galuzzi and Bertels, 2011].

On the other hand, dynamic analysis approaches have been developed using various profiling tools. They identify critical operations in a dynamic environment for a particular data set. As mentioned earlier, even though these approaches are also capable of producing good results, their lack of flexibility prevents them in other application domains. Hence, they cannot be considered to be general solutions.

4.2 Basic Principles of Frequency Analysis Strategies

This section describes the basic principles of our static analysis strategies. The selection of fundamental properties of the analysis is introduced, such as the level of analysis, data structures, and concepts of the analysis.

4.2.1 Basic concepts

First, two main ways to find frequently executed operations in an algorithm's source code are identified. One way is to statically analyse the number of occurrences of the operations within the algorithm. The other is to determine whether the operations are nested within loops. The latter might be more critical since such operations are likely to execute repeatedly at run-time [Levitin, 2007]. In complexity analysis, such operations are considered as the basic operations. The complexity of the algorithm is expressed in terms of the execution behaviour of these operations [Levitin, 2007]. However, most existing static-based analysis approaches identify operations only in one way or the other [Galuzzi and Bertels, 2011]. In contrast, our work aims to identify operations in both ways, so that all critical operations of an algorithm can be successfully identified.

Next, for our purposes a suitable method analysis is selected. As mentioned previously, existing approaches were built based on two strategies: dynamic and static analysis. Dynamic analyses are considered to be "precise" since they can examine the actual run-time execution behaviour for a given data set [Ernst, 2003]. However, the disadvantage of using dynamic analysis is that many analysis scenarios are required to cover all the execution possibilities. On

the other hand, static analysis is considered "sound" because its results are accurate regardless of the current input or its run-time environment [Ernst, 2003]. Therefore, almost all existing *general* approaches are static analysis based. Likewise, in order to enhance the generalisation of our work, we also use static analyses instead of dynamic analyses. However, the major weakness of static analysis is its conservativeness, which can produce weaker results than expected [Ernst, 2003]. In order to overcome this drawback, new strategies were needed to be developed, especially to better forecast the run-time behaviour of the operations of the algorithms.

In static analysis, the source code of an algorithm is closely examined to identify characteristics and performance behaviour of the algorithm. Almost all existing static methodologies perform operation identification in assembly code or different forms of intermediate representations (IRs). These representations facilitate understanding or studying functionalities involved in the algorithm from a low level, where even the atomic operations can be clearly seen. Thus, in order to achieve deep investigations in our frequency analysis process, we use assembly level analyses with some necessary help from the high level implementation.

As discussed in Section 4.1, existing static analysis approaches suffer from their high computational complexity. This is because even for a single algorithm they need to construct and traverse complex data structures. It is important to reduce the complexity of the problem in our analysis as we consider multiple algorithms. Therefore, alternative light-weight analysing strategies are required, especially to perform the costly operation matching phase. To achieve this, a simple technique is needed to store and analyse algorithms' data including instruction operations, operands and, data flow and control flow relations.

Therefore, to reduce the complexity of our system we deploy light-weight text based data structures throughout the whole methodology. Even though this syntactic strategy is shallow compared to semantic analysis based strategies, accurate results can be achieved by a simple and fast process.

In our approach, the operations of algorithms are treated as sequences of assembly instructions. First, assembly instructions with their operands are considered as individual strings. Then, an array of strings is used to represent the whole algorithm by taking one instruction as one entry of the array. By manipulating the string array, all possible special-purpose operations are generated (generation of all valid operations is described in Section 4.4 i n detail). Each of

these operations is again considered as single strings consisting of several instructions and their operands. In order to track the data flow between instructions, we use a symbol substitution strategy, as will be described in detail in Section 4.4.2. Once the whole set of operation strings is generated, they are matched with each other to identify occurrences of similar operations. Thus, the matching phase is reduced to a problem of text matching, which is obviously a simpler approach than a problem of pattern matching of graphs.

4.2.2 Insights for the Frequency Analysis Strategies

In this subsection, the fundamentals of our analysis strategies are discussed. First, it is necessary to understand the nature of the properties of the operations to be captured. As mentioned previously, there are two main ways to find the frequently executed operations. The first form is due to an operation occurring frequently throughout the algorithm. These types of operations can be identified by statically counting the number of occurrences. The other way is operations appearing within loops. Here the criticalness of the operation cannot be determined by only counting the static frequency, as their run-time frequency is likely to be much greater than the static count. Instead, a different analysis strategy is needed to accurately locate such operations and determine their criticalness in terms of runtime execution. Thus, this analysis consists of two complementary strategies compatible with the selected data structures and basic concepts presented in the last section.

In the development of the strategies, generating the list of *all possible atomic* operations for a given algorithm is a challenge. As mentioned in the last section, the operations are generated as sequences of assembly instructions. Therefore, in order to generate all possible operations, different sequences of consecutive instructions must be considered. This may result in a significantly large operation list. However, all these resultant instruction sequences may not form semantically valid operations. The process of generating the valid operations must be sophisticated enough to locate only eligible operations.

The next challenge is reducing the complexity of the core matching process without compromising the accuracy. Matching two operations with each other is an important operation to be performed repeatedly by both frequency analysis steps. For instance, in static frequency counting, the textual number of occurrences are counted. All the unique operations are matched with each other to count the number of matches, giving the static frequency. Since the matching is performed as a text matching task, the computational complexity and time consumption are

significantly reduced. However, this string representation of the operations needs some extra effort to track data flow and control flow details in the matching phase. When two operations are matched to check whether they are similar or not, their data dependencies between instructions must be semantically mapped. For instance, consider the two sequences in Figure 4.1. In sequence 1, the "sub" operation utilises the same data "$a6$" as that loaded from "$X2$" during the previous "add" instruction. In sequence 2, data "$a6$" is a previously unknown value in the sequence. Thus, these two sequences are treated as two different sequences even though they are syntactically similar by operation names. They are semantically different by operands.

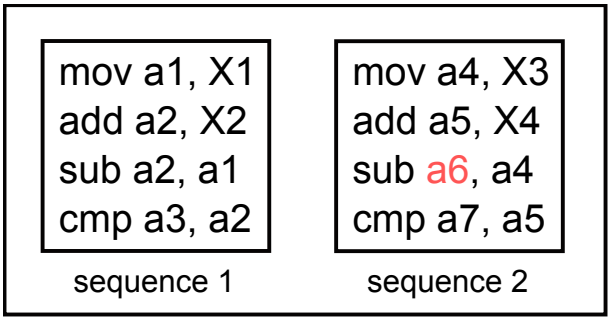


Figure 4.1: Example of two sequences in which data flow is different semantically

4.3 Fundamental Design of Analysis Steps

This section presents the initial versions of the analysis strategies built upon the above-mentioned concepts. In order to evaluate the success of these strategies, experiments were carried out along with the development, using a set of *exact string matching* algorithms introduced in Section 2.7.1.

4.3.1 Basic Static Frequency Counting

The target of this analysis is to count the number of static occurrences of all possible special-purpose operations of an algorithm. Since the analysis is performed at the assembly level, the assembly instruction code of the algorithm is obtained first. Typically, an algorithm can be implemented using more than one function in high level source code. But in this step, we are interested in non-branching instruction sequences only. Therefore, when the high level code is compiled on any Instruction Set Architecture (ISA), function handling instructions such as *call*, *ret*, *push* and *pop* or any other such operations (depending on the ISA) are eliminated from the compiled code.

In the static frequency counting the number of matches needed to be performed is minimized prior to the matching phase. The assembly instruction code of the algorithm is obviously much longer than the high level source code. Therefore, when all potential operations are generated, large sets of instructions need to be considered. This leads to the problem domain expanding rapidly with all possible subsequences of assembly instructions. Hence, the time consumption of the matching process may also increase accordingly. In the initial development of this analysis, the matching phase is broken into two parts (partially automated) to reduce its complexity. This is achieved by performing the semantic mapping between operands only for the potential operations extracted from the first part.

In the first part of the matching process, all possible instruction operator sequences are matched with other instruction sequences. At the same time, the number of occurrences of each sequence is counted automatically. Here, the string of assembly instruction operations (without considering their operands) of a sequence are compared with that of other sequences to determine their syntactic similarity. When such a matching is successful, the frequency count of the particular instruction sequence is incremented by one. Thus, frequencies are counted for all potential operations formed from sequential instructions. Then, in this way these sequences are ranked according to their frequency scores. Marking a threshold value for the frequency, the sequences with lower frequencies are eliminated from the candidate operation list. The threshold value can be user-defined because it is subjective to the considered algorithm and frequency scores of its operations. Thus, the eligible candidate operation list is cut down significantly, and only a sub-set of operations are sent through the next costly semantic matching step. The reduction of the candidate operations helps increase the speed of the analysis process.

In the second part, these operations are again matched semantically with each other to count their new frequencies. The technique used to perform this semantic matching is described, where the final static frequency analysis step is presented in 4.4.2. The operations list is again ranked according to the new frequency scores. The top operations can then be considered as the most critical operations of the algorithm in terms of static frequency.

The above approach of identifying frequently executed operations was evaluated by performing experiments. A set of exact string matching algorithms listed in Table 2.1 were used for this purpose. In this experiment, the maximum allowed length of a sequence is limited to 15 instructions. In exact string matching algorithms, the functionalities were simple and hence

most of the basic operations were only few instructions long. Therefore, instruction sequences more than 10 instructions long only got a few (about 1 to 5) frequency counts. In addition, the threshold value used was 30, since this was near the average of frequencies at which a set of less frequently occurring operations can be eliminated easily. Thus, approximately about 65% of raw instruction sequences could be eliminated from the candidate operation list by the first part of the syntactic matching phase in this case study.

However, the resultant candidate operations at this stage still cannot be considered as candidate operations. This is because they have been only analysed syntactically but not semantically with operand data flow taken into consideration. Therefore, in the second part, a semantic matching of instructions is performed on the remaining sequences of the list. During semantic matching, instructions in all sequences are appended with their corresponding operands and frequency counting is performed again. In this case, a reasonably small number of sequences are analysed since a larger number of sequences were already eliminated in the first part.

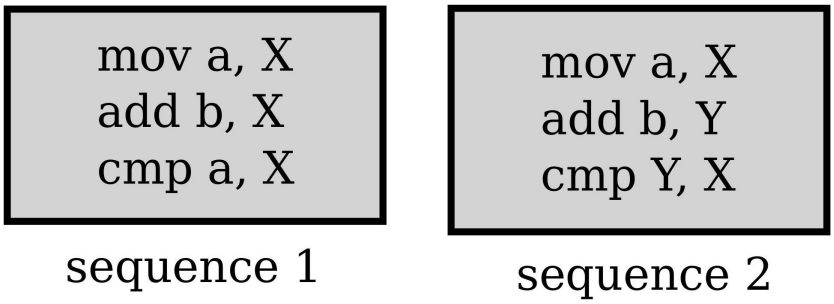


Figure 4.2: Example of syntactically similar but semantically different instruction sequences

Nevertheless, the results obtained in this initial experiment were not found to be adequate. This is because the frequencies of previously highly ranked operations dropped dramatically in the second analysis part. The main reason was that, even though frequently occurring operations were successfully found in the first part, most of them were partitioned into semantically different sequences after performing the semantic analysis on these operations. Figure 4.2 shows examples of two such sequences that perform different functionalities, even though their pattern of operation names are the same. Hence, the frequency count of corresponding operations changed significantly. Consequently, most of the candidate operations, which were considered critical due to their high frequency scores in the first analysis step, were later found to be “false-positives” when semantics were analysed. Therefore, the ranked candidate operations list needed to be re-organised to identify the most critical set of operations. Hence,

previously low-ranked candidates based purely on syntax could be re-instated as high-ranked candidates following the semantic analysis. Inspired by this analysis and its results, the final static frequency analysis strategy was improved so as not to eliminate any potential critical operations during the process. This fully automated approach is described later in Section 4.4.2 *Sequential Code Analysis*.

4.3.2 Finding the Nesting Levels of Operations

As mentioned in Section 4.1, the second way to identify characteristic operations is to see if they are nested inside loops. Due to their executional criticalness, such operations are considered as "*basic operations*" of most algorithms according to complexity analysis concepts [Levitin, 2007]. However, these operations cannot be identified from a textual frequency counting method like that described above. The reason is that such methods do not consider the control flow location of operations in the code, when the frequency is counted. In existing static approaches, no formal method is presented to identify these types of operations. On the other hand, in dynamic analysis approaches, these operations are likely to be found easily by various profiling tools [Choi et al., 1999; Mbaye et al., 2005; Sun et al., 2004]. These dynamic approaches are suitable to find deeply nested operations that behave consistently for different data sets in run-time. However, in general, many analysis scenarios using different sets of data are required to be performed to identify critical operations accurately. Therefore, in our approach, we build a static analysis procedure to determine the nesting level of operations in an algorithm. Hence the execution criticalness of operations can be numerically represented in terms of their level of nesting. Thus, we add an additional property analysis step to our methodology to help predict the runtime frequency of an operation.

In order to achieve this, the lexical nesting level of each candidate instruction sequence needs to be discovered. In our initial approach, the algorithm is represented in a scoping tree structure. Their deeply nested functionalities are identified by studying the depths of the branches and leaves of the program tree. In the experiments, the nesting levels of primitive instructions and hence candidate operations were determined. The efficiency of this initial process needs to be improved to use strings rather than tree structures (to reduce the complexity). This improved approach is described in detail in Section 4.4.3 as the *Nesting Level Analysis* step.

4.3.3 Block Level Analysis

There are two main types of operation identification methods in terms of the nature of resultant operations. They are known as *fine granularity* and *coarse granularity*. In fine-grain, operations are entirely enclosed within single-entry and singe-exit basic blocks. In coarse-grain analysis, loops or procedures are considered [Galuzzi and Bertels, 2011].

Thus, our previous two frequency analysis steps (described in Section 4.3.1 and 4.3.2) fall into the fine-grain category. This is because they find the operations only within basic blocks (this is shown in Section 4.4.2 in detail). However, many loops in algorithms are considered crucial for the performance of the algorithm [Mbaye et al., 2005]. Such loops may enclose multiple basic blocks separated by branching instructions. Hence, the above two analysis steps are not capable of identifying these operations.

We extend the previous static frequency counting strategies further to capture frequently executed loops. Then, they are examined as potential critical operations of the algorithms. The purpose of this analysis is to identify instruction sequences including branches, provided that the target addresses are within the block itself and the block consists of Single-Input, Single-Output (SISO) control flow. A block is the smallest instruction sequence, which cannot be further split without ensuring that it has only a single entry and a single exit point.

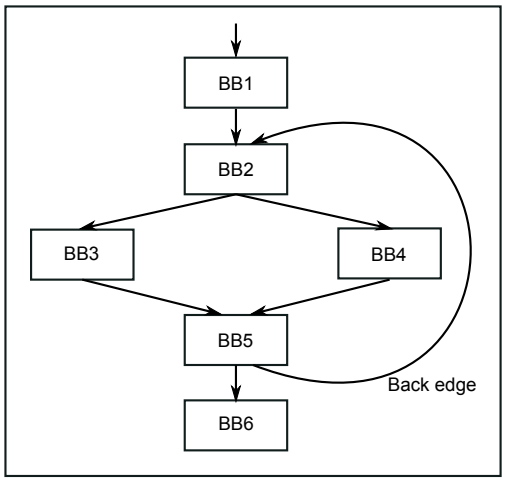


Figure 4.3: Loop segment in a control flow graph.

To perform this analysis, sequences of instructions are generated with branching instructions. The target address of the branching instructions are tracked to determine whether the code segment fulfills the requirements of a SISO block. This scenario can be explained with the example Control Flow Graph (CFG) representation shown in Figure 4.3. In the CFG, the basic

blocks are represented as nodes and control flow between them as directed edges.

According to the basic control flow analysis concepts [Tarjan, 1973], a loop can be identified as follows. First the dominators which dominate other nodes of the graph are identified. Node A dominates node B if and only if A is the unique immediate predecessor of B or A is a dominator of all immediate predecessors of B [Allen, 1970]. For instance, in Figure 4.3 node BB1 is a dominator of node BB5. Then, the back edges are identified. A "back edge" is an edge where the node at the head is a dominator of the node at the tail. Then all the nodes and edges in between the head and tail of the back edge are identified. In the selected set of basic blocks, other than the back edge, if incoming edges only come to the basic block with the back edge head and, other than the back edge, only one outgoing edge from the basic block with the tail of the back edge exist, it is considered as a loop (Figure 4.3) [Tarjan, 1973]. In this way all the iterative code segments of an algorithm can be found.

To examine the usefulness of this analysis, an experiment was carried out manually for the same set of string matching algorithms presented in Section 4.2.2. All the objective algorithms were analysed to find such SISO blocks and the number of their occurrences. According to the results shown in Table 4.1, different types of loops were found in almost all algorithms. Notably, some loops were used by multiple algorithms providing evidence of the criticalness of these operations. However, all the above loops appear in the outermost nesting level of the algorithm. So consideration of nesting levels of these blocks is ignored in this step. The basic concepts of this block level analysis is similar to the loop-free static frequency counting approach, except for tracking branching instructions and their target addresses. Therefore, it was integrated into the static frequency counting process so that they could be performed together.

4.4 Development of Final Analysis Strategies

This section discusses our frequency analysis steps with their technical implementation details. It consists of two main types of analysis strategies: *Sequential Code Analysis* and *Nesting Level Analysis*. As described in Section 4.2.2, these two types of analysis determine run-time execution frequencies of the operations by investigating their static occurrences and nesting levels, respectively. The pros and cons of the initial developments presented in the previous Section 4.2.2 were thoroughly assessed to improve the analysis and to automate the whole process.

Table 4.1: Control flow analysis results for thirteen string matching algorithms (the omitted elements are 0), showing the frequency of occurrences of loops.

No.	Special Operations	MP	KMP	AX	BM	TBM	ZT	HO	RT	CO	GG	NSN	SO	BF
1	**while** ($j > -1$ **and** $x[i] \neq x[j]$) $j = a[j]$	1	1	1							1			
2	**for** ($i = 0$; $i < A$; ++i) $x[i] = a$				2	2	1	1	1	1	1			
2	**for** ($i = 0$; $i < a - 1$; ++i) $y[x[i]] = a - i - 1$				1	1		1	1					
3	**for** ($i = 0$; $i \leq a - 2$; ++i) $b[a - 1 - x[i]] = a - 1 - i$				1	1	1							

4.4.1 Atomic Operation Generation

The first task of these frequency analysis steps is to generate all possible candidate operations when the assembly source code of an algorithm is given. The operations are made out of multiple instruction sequences [1]. In the analysis process, all possible operations are generated automatically as different subsequences of the algorithm's instructions. Generating instruction sequences blindly may result in invalid executable operations as candidates. For instance, according to the source code shown in Figure 4.4, the green and blue coloured operations have dramatically different run-time frequencies. The green coloured code segment would be executed 255 times repeatedly before executing the blue coloured instructions once. However, in an assembly level static environment, when operations are generated by binding multiple instructions together, the instructions do not convey any semantics regarding their potential execution frequencies. As a result, this distinction among operations is not visible. Therefore, an unguided candidate operation generation approach may produce many false-positive results.

In our approach, generation of eligible operations is therefore guided by distinguishing different basic blocks. The candidate operations are limited within the boundaries of basic blocks. Hence, all possible operations inside a basic block are identified. Based on the concepts of a basic block, boundaries are captured by tracking *entry* and *exit* points, which are typically performed by *return* and *branching* instructions.

[1] Here *atomic instructions* are not considered as eligible operations since, obviously, they cannot be further optimized.

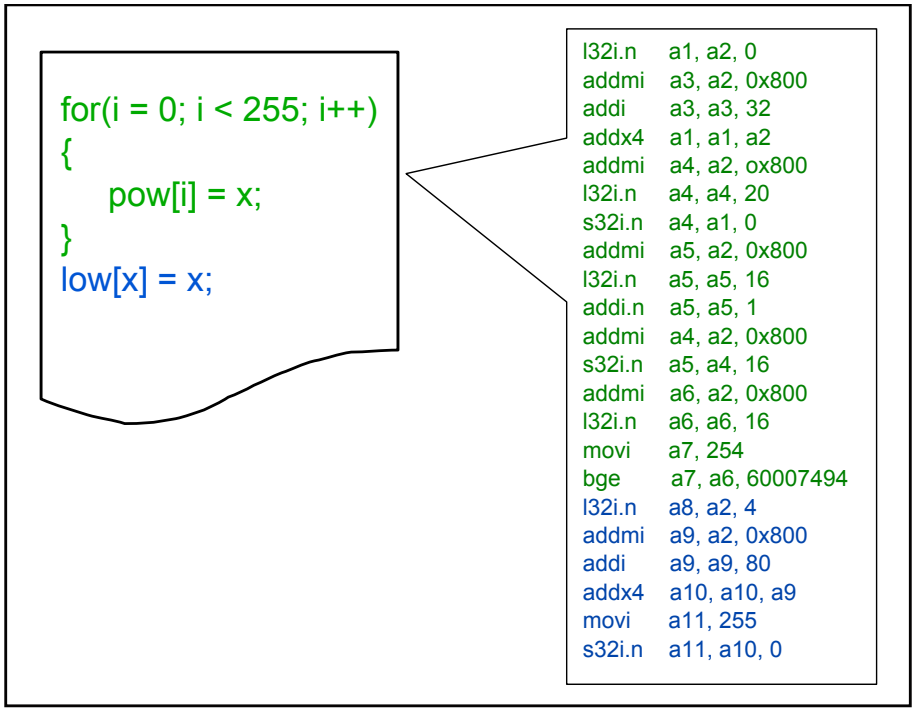


Figure 4.4: Example of a false-positive instruction sequence

In addition, this basic block tracking technique is capable of limiting the size of the resultant sequences automatically. Since the candidate operations are formed from different subsequences of instructions, the length of a sequence could reach the length of the whole algorithm. However, when the sequences are limited to the boundaries of a basic block, the maximum length of the sequence is limited to the size of the basic block. Extremely long sequences are not beneficial for constructing special-purpose operations, as their execution frequency will be very low. In addition, when the basic block itself is very long, we limit the length of candidate instruction sequences to a given maximum. This value can be determined by the user, considering the nature of the expected operations and requirements of the target application.

A tool was developed to generate a valid operation list as described above, when the source code of algorithms is given. Starting from a size of two (instruction sequences which consist of two instructions) to the maximum allowed length of expected sequences set by the user, all possible instruction sequences are generated by binding sequential instructions together as sequences. For instance, the assembly instruction source segment shown in Figure 4.5 forms a maximum of six different instruction sequences. Here the smallest operation is two instructions long and the longest one is limited to three.

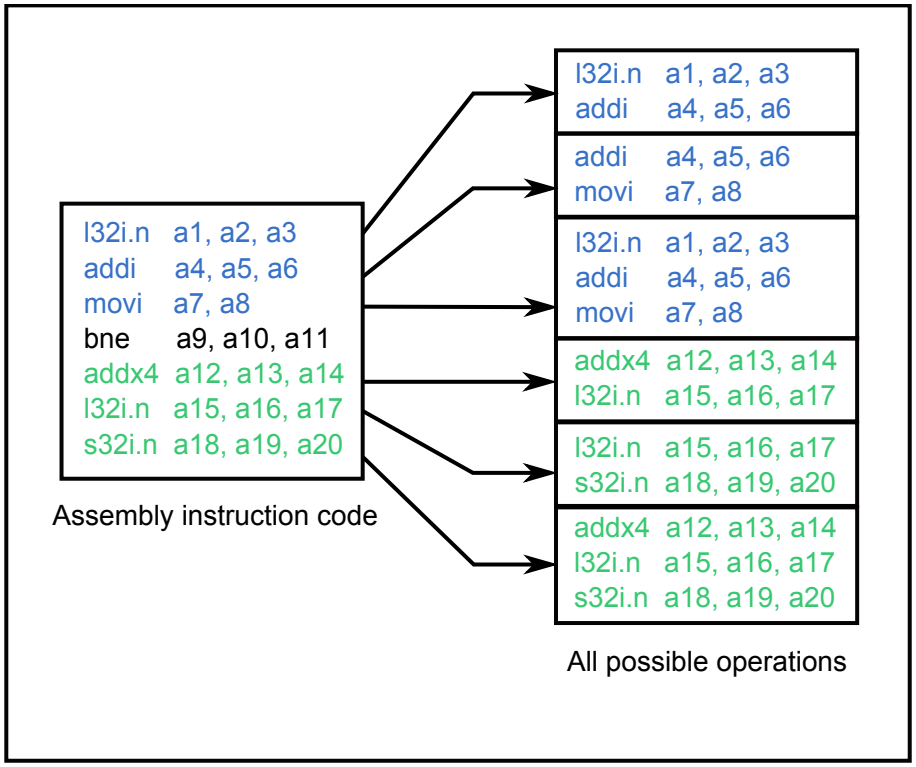


Figure 4.5: Sample assembly instruction code segment belonging to two basic blocks and all their possible instruction sequences

In the automated implementation, separation of the basic blocks and generating valid operations is performed using text-matching techniques. Typically, in the assembly instruction level, instructions like "branch", "call" and "return" instructions imply boundaries of basic blocks (*entry* and *exit* points). While reading the assembly instructions, each instruction is matched with a pre-defined list of such operations, which can be a potential *entry* or *exit* point of a basic block. Once these instructions are found, the instruction sequences are broken into different basic blocks from that point. An example of a such a scenario is presented in Figure 4.5. Here, the original assembly code is divided into two parts by the branch instruction "bne". Therefore, the two differently coloured instructions belong to two different basic blocks. From these two basic blocks, six distinct operations are generated. Operations produced by this step are called *valid instruction sequences* and hence are *valid atomic operations* in this context. They are thus truly executed in runtime by the algorithms.

With lightweight text matching techniques, our valid operation generation program was able to produce complete results in a reasonable time. According to dynamic experimental results carried on for the cryptographic algorithms introduced in Chapter 2 (Table 2.2), the program generated an average of about 63513 instruction sequences from about 4221 lines of assembly

Table 4.2: Valid Operation Generation Execution Performance

Algorithm	**Length of source (assembly)**	**No. of instruction sequences**	**Elapsed time /(ms)**
AES in CBC mode with md5 hash	9640	156640	3094
AES in CTR mode with sha	3445	50574	1016
BLOWFISH in CBC mode with md5	3824	61032	1203
BLOWFISH in CTR mode with sha	1737	6761	172
CAMELLIA in CBC mode with sha	3054	31727	641
CAMELLIA in CTR mode with md5	5021	85956	1656
3-DES in CBC mode with sha	2723	35528	719
DES in CBC mode with md5	4321	79886	1547
Average	4221	63513	1256

instructions. The average elapsed time to achieve these results was 1.3 seconds for an Xtensa processor (the maximum length of an instruction sequence was kept as thirty). Thus, even though this step completes an important phase of our framework, it can be achieved within a few seconds for all the considered algorithms. This is due to the simple syntactic matching techniques used in our implementation. Table 4.2 summarises the number of valid operations generated with the corresponding time elapsed measurements for the eight cryptographic algorithms.

4.4.2 Sequential Code Analysis

This section discusses the development of the sequential code analysis step of our framework. Inspired by the initial implementations in Section 4.3.1, this analysis was further improved and re-organised to cover-up weaknesses and enhance its strengths. Once all valid operations are obtained, static frequency counting of the instruction sequences are performed. In our approach, in order to match two operations, they need to be matched syntactically (the instruction sequences' operation patterns) and semantically (the instruction sequences' operand patterns) as illustrated in Figure 4.6. As per the previous analysis, performing these in two sequential steps led to the elimination of some potential characteristic operations. To avoid this elimination, the syntactic and semantic matching processes, along with frequency counting, are performed in the same phase. Thus, every instruction sequence gets a fair chance to be assessed for the candidate operation list.

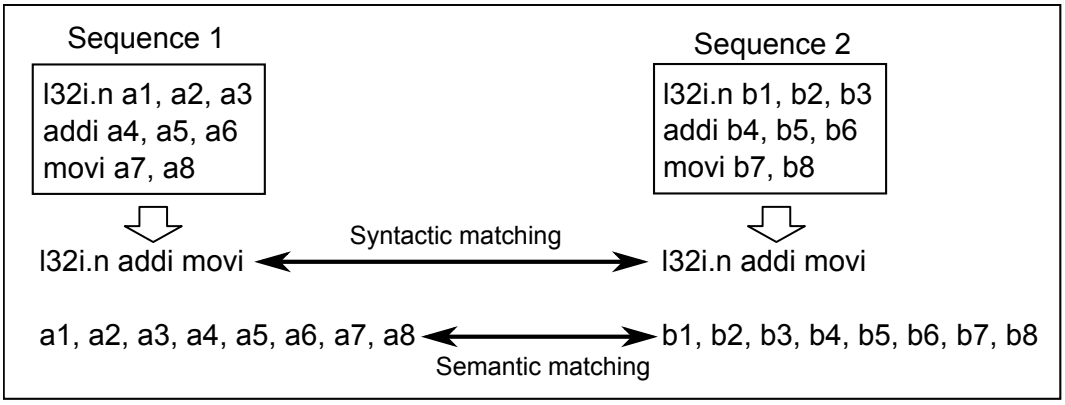


Figure 4.6: Syntactic and semantic matching of two instruction sequences

In the matching process, semantic matching is limited to only syntactically similar sequences. This reduces the complexity and the time cost of the task by avoiding pairwise semantic matchings of every sequence with others. Syntactic matching is much faster due to its simplicity, because it only matches the characters of the operation strings of the two instruction sequences. Thus, the matching phase can be accelerated without compromising any potential characteristic operations.

In addition, prior to the matching phase, the candidate sequences are separated by their lengths. This cuts down a large set of operations to be matched which are different in lengths to the current sequence (obviously, they will produce negative matches due to different length sizes). Thus, if the maximum length of resultant sequences is n, the tool will separate all possible sequences into $n - 1$ sets (from sequence length 2 to n) and matching is performed for a particular set at a time. The separation can be performed easily by tracking details from a valid instruction sequence generation step. Therefore, there is no extra complexity added for the execution of the program. Furthermore, the size of the operations can be recorded at the same time, to be used as another parameter of the final operation selection process.

In the matching phase, data dependencies of instruction sequences are tracked by their operands. A typical assembly instruction consists of an operation name followed by one or more operands. Depending on the architecture of the processor, operands may carry different types of data such as stack values, registers, immediate values and 8-bit register values and memory addresses. In our analysis, the data flow patterns between the operands of a sequence need to be traced accurately in order to identify semantically equivalent instruction sequences and hence equivalent operations. This requires unifying different operands from different instruction sequences.

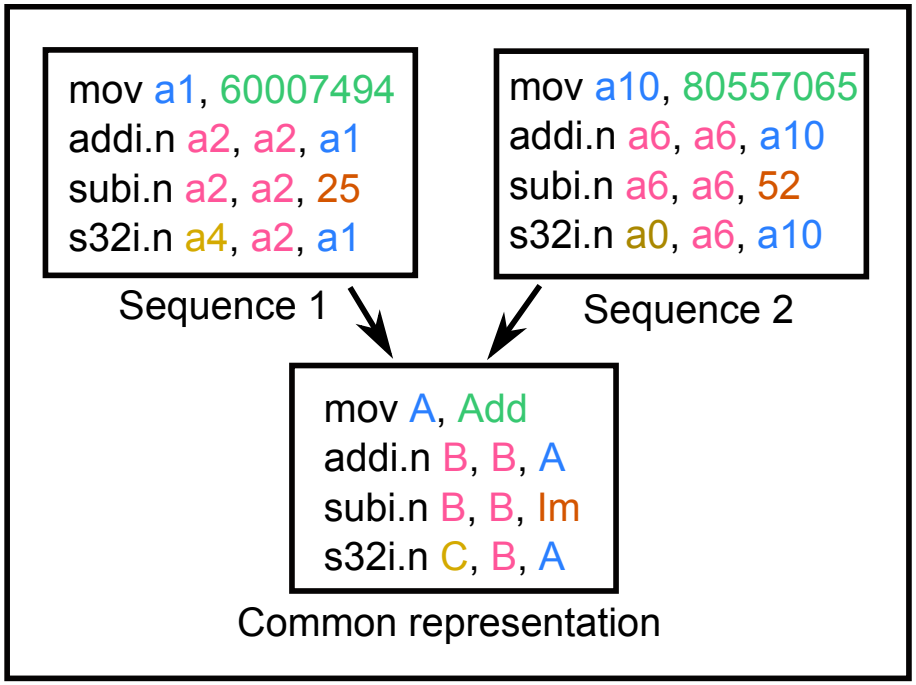


Figure 4.7: Semantic matching of two instruction sequences.

To track the patterns of operands, a list of string symbols are used. In the matching phase, when the operands of the previously generated valid sequences are read one by one, a list of string symbols are substituted for the original operand data. From the type and value of the operand, the corresponding substitutional symbol may vary. For example, if two different register operands are substituted from $X1$ and $X2$, two different immediate values would be replaced by $IM1$ and $IM2$, meaning that X and IM are two different types and 1 and 2 are two different values. Thus, if there are two semantically equal sequences, their lists of symbols will be the same. This implies that the two corresponding sequences can be considered equivalent (as they are already syntactically the same). Figure 4.7 illustrates an example of two such sequences. In the figure, the common representation is the sequence obtained by assigning unique symbols in both sequences. Consequently, semantically similar sequences will have the same pattern of operation names and symbols lists. Likewise, in the semantic matching phase, the qualified instruction sequences are converted to another "string" with different symbols appended in their operand part. These strings are matched with each other to find the semantic correspondences. Thus, this task is converted again to a *text matching* task. The static frequency is then obtained by counting successful matches of two operations via the common representation.

In order to examine the outcomes of this strategy, an experiment was carried out using the

exact string matching algorithms in Table 2.1. The algorithms were implemented in C/C++ and using the compiled code produced by a standard GCC compiler for the x86 architecture. The process successfully identified the most frequently occurring operations in three algorithms as shown in Table 4.3.

Table 4.3: Sequential code analysis results for the Colussi, Boyer-Moore and Horspool algorithms (A: Immediate value; X, Y: Registers; a, b: Stack addresses; m, n, i, j: Integer values; p: Integer array).

No.	Operation	Instruction seq.	CL	BM	HP
1	$p[i]$ (Calculate address of an array element)	**mov** a, X **mov** b, Y **mov** (Y, X, 4), Y	38	19	5
2	$m + n$ (Add two integers)	**mov** a, X **add** b, X	37	44	12
3	$p[i + j]$ (Obtain next array element)	**mov** a, X **add** A, X **mov** b, Y **mov** (Y, X, 4), 4	8	6	3

The final analysis process shows efficient outcomes for the new improvements. Representing the whole matching process as a *text-based matching* problem allows us to obtain results for all considered algorithms (i.e. all 3 to 12 algorithms in our experiments) within a reasonable time period. Even though the computational overhead was slightly increased with the semantic matching of all eligible operations in a single step, this strategy was more effective than our original multi-step process. The reason is that the accuracy of the results was not compromised and all possible operations were kept in the final list.

In addition, considering all operations as candidates will prove useful where common operations across a set of algorithms are selected. In such a situation, operations with low static frequency might also be eligible as characteristic operations if they are common across multiple algorithms. For instance, if a library function needs to be generated targeting multiple algorithms, the mere existence of such an operation in an algorithm might be sufficient rather than considering its static frequency.

Figure 4.8 illustrates the sequence of steps followed in the “Sequential Code Analysis”

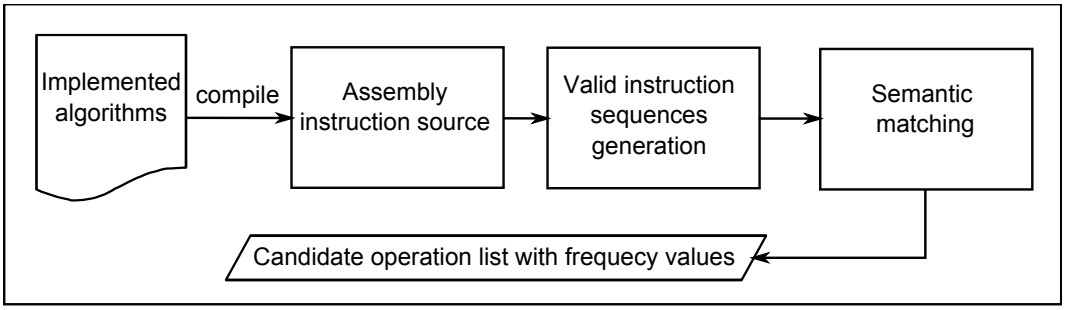


Figure 4.8: Sequential code analysis process

explained in this section. The analysis consists of two important and unique steps to generate and match instruction sequences. Hence it helps in identifying corresponding multi-instruction operations.

4.4.3 Nesting Level Analysis of Instruction Sequences

Considering nesting levels of operations is important, to determine execution frequency of an operation. The operation of comparing two characters of strings *pattern* and *text* is crucial in exact string matching algorithms. This operation is executed repeatedly until the incrementing index meets the maximum length of the *text*. However, according to the *Sequential Code Analysis* approach results as shown in Table 4.3, this operation was not ranked in the top positions of the operation list. That is because the operation appeared on only a few occasions in the source code textually, even though it was nested deeply within loops and therefore occurred frequently at run time. Here, as mentioned previously, an additional method is imperative to identify such operations.

To perform this analysis a tool is built. Inspired by the successful experimental results obtained from the approach mentioned in Section 4.3.2, a parser was developed to identify the nesting levels of all possible candidate operations automatically. However, in the assembly instruction level, loops and other special statements are not immediately visible because all assembly instructions have a similar syntactic structure. Also, because our analysis is performed in a static environment, the execution information is not usable to track iterative loops. Therefore, in this step, nesting levels are first determined in the high level source code of the algorithm where they are more obvious. Then, they are mapped to the corresponding assembly instructions and assigned the relevant nesting levels. In this case, generation of the assembly code for the given algorithms without any compiler level optimisations is important. Thus, the high level source code can be directly mapped to the assembly code by avoiding potential

ambiguities.

In the process, the high level source code of the given algorithms is sent though the above parser program. It determines the nesting level of each line of the source. First, the parser reads the source code line by line and assigns a default nesting level (i.e., the default value is 1). Then the characters of the input line are scanned to identify notable code segments such as loops and conditional statements. When the parser enters a loop like a `for`, `while` or `do-while` loop, it increments the current nesting level. When it exits, it decrements the current nesting level. This means that an operation nested within two loops will have a nesting level of three, whereas a statement in the outer most scope will have nesting level of one.

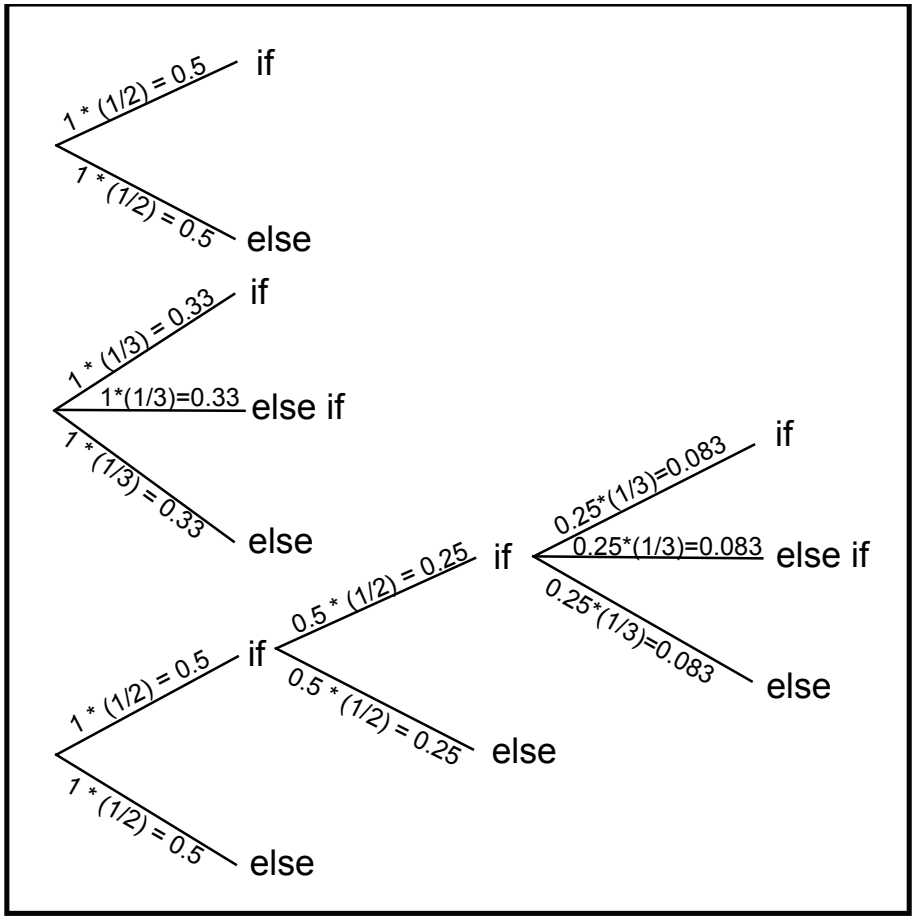


Figure 4.9: Calculation of the execution probability of conditional statements (the probability of execution of a root statement is considered as 1).

The parser considers the probability of execution to calculate nesting levels of conditional statements. Nesting levels of all conditional statements such as `if-else` and `case-switch` are calculated proportionally to the probability of their execution when all alternatives are assumed to be equally likely. When the parser enters a conditional statement the current nesting

value is multiplied by the execution probability of that statement. Figure 4.9 illustrates how the nesting levels for nested `if-else` statements are calculated. However, these calculations are only an approximate estimate for how frequently each alternative would be executed at runtime. As we use the same method in calculation of nesting levels for all objective algorithms, and our goal is to compare them with each other rather than using absolute numbers in the analysis process, this is accurate enough.

```
87
88    //reading "while" characters
89    else if(currLine[i]=='w' && currLine[i+1]=='h' && currLine[i+2]=='i' && currLine[i+3]=='l' && currLine[i+4]=='e'){
90        //get the next characters of the line
91        currLineTemp = currLine.substr(i);
92
93        //check the syntax brackets of the while loop declaration
94        if(currLineTemp.find('(') != -1){
95            //get the next characters of the line
96            posChar = currLineTemp.find('(');
97            currLineTemp = currLineTemp.substr(posChar + 1);
98
99            //check the syntax brackets of the while loop declaration
00            if(currLineTemp.find(')') != -1){
01                ...
02                //read next lines
03                getline(codeTextIn, currLine);
04                newLength = currLine.size();
05
06                //read the open bracket
07                if(currLine.find('{') != -1){
08                    //increase the current nest level by one
09                    currNest++;
10                    ...
11                    ...
12                    //push open bracket on the bracket stack
13                    bracketStack.push_back("whileBracket");
14                }
15                else{
16                    ...
17                }
18                continue;
19            }
20            else{
21                ...
22                ...
23                continue;
24            }
25        }
26        else{
27            ...
28            ...
29            continue;
30        }
31    }
32
```

Figure 4.10: Implementation of tracking the open brackets and push brackets on the bracket stack

In the implementation, to identify loops and conditional statements, the characters of each line are matched with a pre-defined rules set. For instance, as shown in the sample program code in Figure 4.10, a “while” loop is identified by checking the word character by character and the required parentheses “(”, “)” and “{” in the same line. Here checking open and closed parentheses is important. However, in general programming the format of code and use of parentheses is inconsistent from developer to developer. In addition, most commonly, when there is only one statement inside a loop or any other conditional statement no brackets are used. Since our parser depends on the parentheses used in the source code, it may fail to identify such situations. To avoid these failures, the input programming code is “pretty printed”.

The implementations of all algorithms are sent through a pretty printer before they are fed into the parser to ensure a consistent layout. The pretty printer adds an open bracket "{" in the next line and closed bracket "}" at the end of all loops and conditional statements of the algorithms. Conversion of sample code segment to consistent pretty-printed form is shown in Figure 4.11. In our case studies the open source *Artistic Style 2.04* pretty printer was used with customised options. Thus our complete parser tool set is capable of identifying `for, while, do-while, if-else and switch-case` statements accurately.

```
    for( i = 0; i < 7; i++ ){
        if( verbose != 0 )
            printf( "  MD5 test #%d: ", i + 1 );

        md5( md5_test_buf[i], md5_test_buflen[i], md5sum );

        if( memcmp( md5sum, md5_test_sum[i], 16 ) != 0 )
        {
            if( verbose != 0 )
                printf( "failed\n" );

            return( 1 );
        }

        if( verbose != 0 ){
            printf( "passed\n" );
            return( 1 );
        }
    }
```

(a) Sample source code segment

```
    for( i = 0; i < 7; i++ )
    {
        if( verbose != 0 )
        {
            printf( "  MD5 test #%d: ", i + 1 );
        }

        md5( md5_test_buf[i], md5_test_buflen[i], md5sum );

        if( memcmp( md5sum, md5_test_sum[i], 16 ) != 0 )
        {
            if( verbose != 0 )
            {
                printf( "failed\n" );
            }

            return( 1 );
        }

        if( verbose != 0 )
        {
            printf( "passed\n" );
            return( 1 );
        }
    }
```

(b) Output source code segment from the pretty printer

Figure 4.11: Demonstration of conversion of source code by the pretty printer

Getting in and out of compound statements is determined by tracking open and closed brackets. To perform this task, a string array is used to act as a stack. When the parser gets inside a loop or a conditional statement, its open bracket is identified (the type of the statement) and it is pushed in the stack. When the parser gets out of the statement (when the closed bracket is read) it identifies the closed bracket and pops the top-most bracket off the stack. This

functionality is illustrated in Figure 4.12. When considering the code segment shown in the figure, as the parser reads the first line, it identifies the `if` condition and its open parenthesis ("{") and, pushes it onto the bracket stack. Its implementation is illustrated in Figure 4.10 for a `while` loop. Likewise, the *whileBracket*, *ifBracket* and finally *forBracket* are pushed respectively when their open brackets are read by the parser. Then, for the first time in this code, a closing bracket of the *for* loop is read and the top bracket of the stack is popped. Thus, while the parser reads the next lines of the code other open brackets are also popped from the stack. Each time when an opening parenthesis is popped from the stack, based on the type of the parenthesis (`for, while, if, else,` etc), the nesting value is calculated. This implementation is briefly illustrated in Figure 4.13. Thus, nesting levels for all lines in the algorithm are determined and appended for each line (Figure 4.15). Figure 4.14 shows a sample high level source code segment with each line's nesting levels.

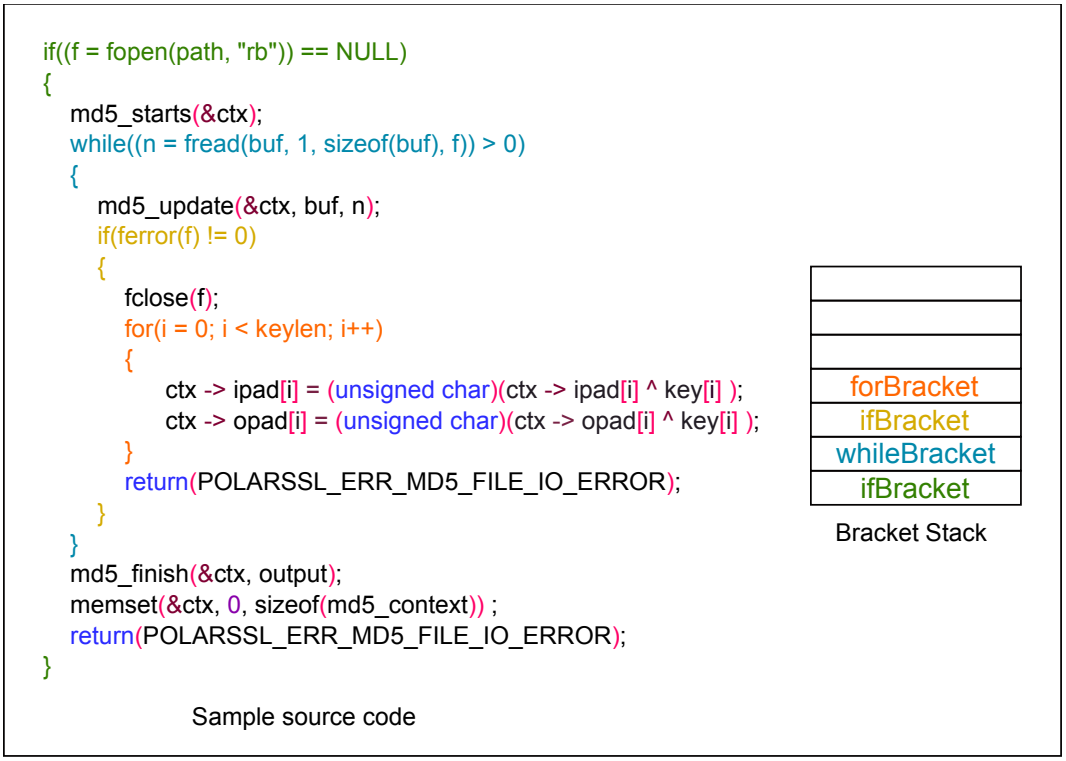


Figure 4.12: Example of stacking the brackets in the *Nesting Level Analysis*

The same nesting levels of high level source code lines are then mapped to the assembly level instruction sequences. The mapping can be easily performed with exact correspondences while avoiding compiler optimisations. Figure 4.15 illustrates a sample code segment of the MD5 hash function with the obtained nesting levels and its corresponding assembly instructions. The assigned nesting level values are also shown in the code segment.

```
    //read brackets
    else if(currLine[i] == '}'){

        //if the bracket stack is not empty
        if(bracketStack.size() > 0){

            //reduce the current size by one since one condition is closed
            currBracket = bracketStack.at(bracketStack.size() - 1);

            //check the type of last open condition and set nest levels accordingly
            if(currBracket == "forBracket" || currBracket == "whileBracket"){
                currNest--;
                bracketStack.pop_back(); //pop top most open bracket
            }

            //check the type of last open condition and set nest levels accordingly
            else if(currBracket == "ifBracket" || currBracket == "elseBracket"
                                || currBracket == "switchBracket" || currBracket == "caseBracket"){
                currNest = currNest + (currNest * probabilityFact);
                bracketStack.pop_back(); //pop top most open bracket
            }

            else{
            ...
            }
        }

    ...
    ...
    }
```

Figure 4.13: Implementation of tracking the closing brackets and calculating nesting levels accordingly

For all given algorithms all possible valid instruction sequences are obtained as described above. However, when the same operation occurs in multiple places in the code, the nesting level can be different from one occurrence to the next. In this case, an average value is calculated with consideration of all the occurrences. Also, once the nesting levels are found for all possible unique operations, the operation list then can be ranked to find the most critical operations in terms of high nesting levels. Similar to the *sequential code analysis*, all possible operations of the algorithm are kept in the final result list. They will all be available to find common operations across multiple algorithms and targeting different applications in the later stages.

The implementation of nesting level analysis can be integrated into the *sequential code analysis* step. Even though the purposes of these two analysis strategies are different, the valid operation generation step is performed by both similarly. Therefore, if the two analysis processes can be integrated into a single methodology with the two analysis steps, the repeating steps can be avoided. As a result, the time cost and complexity of the whole process can be significantly reduced. Finally, Figure 4.16 summarises the steps followed in the *Nesting level analysis* process.

In order to validate this analysis, an experiment was carried out using the same exact string matching algorithms set as in Section 4.3. Tables 4.4 and 4.5 list some of the top operations

```
356
357     for( i = 0; i < 7; i++ )
358     { //,2
359         if( verbose != 0 )  //,2
360         { //,1.0 = 2 * (1 * (1/2))
361             printf( "  MD5 test #%d: ", i + 1 );  //,1.0
362         } //,2
363
364         md5( md5_test_buf[i], md5_test_buflen[i], md5sum ); //,2
365
366         if( memcmp( md5sum, md5_test_sum[i], 16 ) != 0 )  //,2
367         { //,1.0 = 2 * (1 * (1/2))
368
369             if( verbose != 0 )  //,1.0
370             { //,0.25 = 1.0 * (1 * (1/2) * (1/2))
371                 printf( "failed\n" ); //,0.25
372              }  //,1.0
373
374             return( 1 );  //,1.0
375         } //,2
376
377         if( verbose != 0 )  //,2
378         { //,1.0 = 2 * (1 * (1/2))
379             printf( "passed\n" ); //,1.0
380         } //,2.0
381     } //,1
382
```

Figure 4.14: Nesting level assignment of a loop and a conditional statement from MD5 hash algorithm of *polarSSL* library (the default nest level is 1).

found in a few related algorithms. From Tables 4.4 and 4.5, it can be seen that the basic operation of exact string matching algorithms is ranked as a top operation, which was found to be not so critical in the previous analysis step. These results thus show that this approach is capable of identifying deeply nested characteristic operations accurately.

4.5 Experimental Results

The versatility of these analysis steps needs to be evaluated for different application domains. In order to validate the above frequency analysis steps, in terms of their accuracy in identifying dynamically frequently occurring operations, several experiments were performed. In the experiments, the two static analysis processes presented above were performed on a set of objective algorithms. The execution criticality of all possible operations in the algorithms was expressed numerically, by *static frequency* and *nesting level* property parameters. Based on these values, the operation list was ranked to obtain the instruction sequences with the highest property parameters. The operations with higher values for both *static frequency* and *nesting level* values get the top positions on the list. Then, from the ranked operation list, the top operations are considered as the most critical operations of an algorithm in terms of execution frequency.

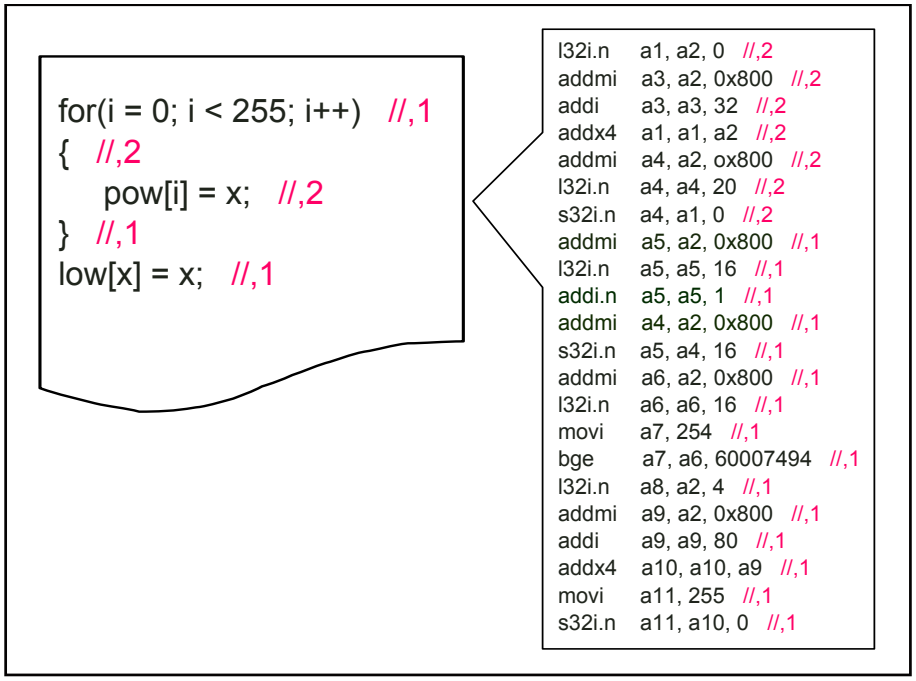


Figure 4.15: Nesting level assignment of a sequence of instructions according to corresponding high level source code,

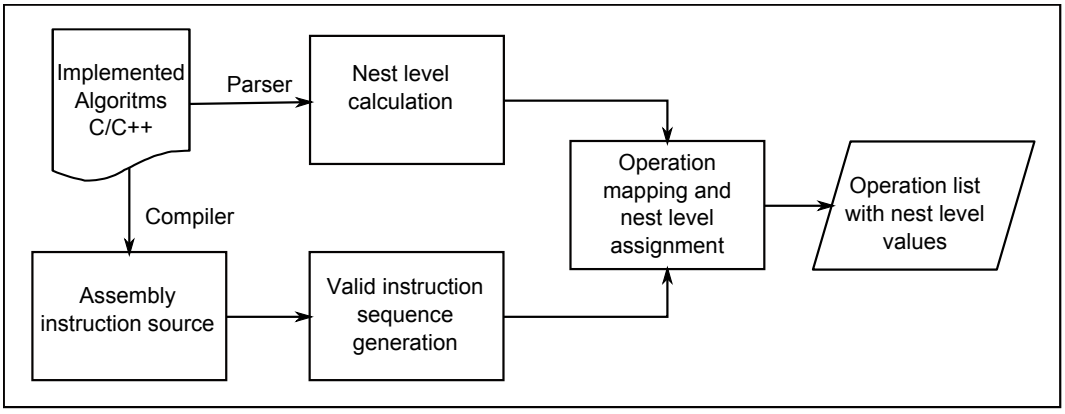


Figure 4.16: Nesting level analysis process.

In order to determine their criticalness at runtime, the execution frequencies of the top operations were counted using suitable data sets. These resultant data was plotted to examine their execution behaviour clearly. If the runtime execution results of the operations are consistent with the criticalness as forecast by our static analysis, our process has been validated successfully. This section presents such case studies for two different sets of algorithms. It shows the results obtained from both our static analysis strategies and the dynamic analyses validation.

Table 4.4: Code segments appearing up to three nesting levels: loop nesting analysis results for the Boyer-Moore, Zhu-Takaoka and Turbo Boyer-Moore algorithms (A, B: Immediate values; X, Y: Registers; a, b, c, d, e: Stack addresses; al, dl: 8-bit Register values; m, n, i: Integer values; p: Integer arrays).

No.	Operation	Instruction seq.
1	$p[i] == p[m+i]$ (Comparing next character of pattern with text)	**mov** a, X **mov** b, X **add** c, X **movz** (X), Y **mov** e, X **add** b, X **add** d, X **movz** (X), X **cmp** al, dl
2	$p[i] = m - 1 - i$ (Calculating next shift of the window)	**mov** a, X **shl** A, X **mov** X, Y **add** d, X **mov** c, X **sub** B, X **sub** d, X **mov** X, (Y)
3	$p[i] == m$ (Compare value of an array element)	**mov** a, X **shl** A, X **add** c, X **mov** (X), X **cmp** al, dl

4.5.1 Results obtained for cryptographic algorithms

The first case study was carried out for the set of cryptographic algorithms introduced in Section 2.7.2. In the experiment, the selected algorithms were implemented in C/C++. They were further converted to assembly level source code targeting the Tensilica Xtensa customisable processor [Tensilica, 2014]. These types of processor provide sophisticated GNU based tools to profile high level and assembly level implementations of algorithms. They were extremely beneficial to the improvement of the accuracy of our static analyses in the development/evaluation phases. Then, by performing our static analysis on these algorithms, all possible candidate operations were obtained with the two property parameters of *static frequency* and *nesting level*.

Table 4.5: Code segments appearing up to two nesting levels: loop nesting analysis results for the Boyer-Moore, Zhu-Takaoka and Turbo Boyer-Moore algorithms (A, B: Immediate values; X, Y, Z: Registers; a, b, c, d: Stack addresses; al: 8-bit Register values; m, i: Integer values; p, q: Integer arrays).

No.	Operation	Instruction seq.
4	$p[q[i]] = m - 1 - i$ (Initializing "bad character shift" array elements)	**mov** a, X **add** b, X **movz** (X), X **mov** al, X **shl** A, X **mov** X, Y **add** c, Y **mov** a, Z **mov** d, X **sub** Z, X **sub** B, X **mov** X, (X)
5	$p[m - 1 - q[i]] = m - 1 - i$ (Filling "good suffix" array elements)	**mov** a, X **sub** A, X **mov** b, Y **mov** c, Z **mov** $(Z, Y, 4)$, Z **mov** X, Y **sub** Z, Y **mov** Y, Z **shl** B, Z **mov** Z, Y **add** d, Y **mov** a, Z **sub** A, Z **sub** b, Z **mov** Z, (Y)

Tables 4.6 and 4.7 show some of the operations obtained for the cryptographic algorithms from *sequential code analysis* and *nesting level analysis* respectively.

Table 4.8 lists the top operations obtained by ranking the operation list based on the property parameter values. We have selected the top operations *common* to all/or most of the algorithms so that their runtime execution frequency can be evaluated by all the algorithms. The property values displayed against the operations in Table 4.8 are the total score they achieved for all the algorithms.

Table 4.6: Static frequencies found for some instruction sequences of several cryptographic algorithms.

Algorithm	Instruction Sequence	Operand Symbol String	Static Frequency
aes cbc md5	s32i.n s32i	0,1,2,3,1,2	167
	s32i.n l32i.n	0,1,2,3,1,2	383
	l32i.n movi.n	0,1,2,3,2	89
	movi s32i.n	0,1,0,2,1	3
	movi.n l32i.n	0,1,2,3,1	122
	mov.n l32i.n	0,1,2,1,3	1
	l32i.n l32i.n	0,1,2,3,1,2	347
aes ctr sha	l32i.n l32i.n l32i.n extui l32i.n	0,1,2,0,0,2,3,1,2,3,3,2,4,1,2	24
	xor l32r addx4 l32i.n xor	0,0,1,1,2,3,3,1,3,3,2,3,3,0	72
	add.n l8ui slli l32r l32i.n	0,0,1,0,0,2,0,0,2,3,2,1,4,2	21
	s8i l32i.n l32i.n s8i l32i.n	0,1,2,3,4,2,5,4,2,5,3,2,6,4,2	3
blowfish cbc md5	l32i.n l8ui slli l32i.n l8ui slli l32i.n l8ui slli or or	0,1,2,0,0,2,0,0,2,3,1,2,3,3,2, 3,3,2,4,1,2,4,4,2,4,4,2,4,4,3,4,4,0	2
	l8ui or or or s32i.n addi l32i l8ui slli l32i l8ui	0,0,1,0,0,2,0,0,3,0,0,4,0,5,1,6, 7,1,8,7,1,8,8,1,8,8,1,0,7,1,0,0,1	12
	l32i xor addi l32i.n add.n l32i add.n add.n s32i l32i slli	0,1,2,0,0,3,4,1,2,4,4,2,4,4,0,5,1, 2,5,5,4,5,5,6,5,1,2,7,1,2,7,7,2	48
	or l32i add.n s32i l32r l32i l32i xor l32i xor addi.n	0,0,1,2,3,4,2,2,0,2,3,4,5,4,6, 3,4,7,3,4,7,7,6,8,3,4,8,8,7,9,3,4	3
camellia cbc sha	l8ui or s32i.n addi l32i.n l8ui slli l32i.n l8ui slli l32i.n l8ui slli or or l32i.n l8ui or	0,0,1,0,0,2,0,3,1,4,5,1,6,5,1,6,6,1, 6,6,1,0,5,1,0,0,1,0,0,1,7,5,1,7,7,1, 7,7,1,7,7,0,7,7,6,8,5,1,8,8,1,8,8,7	3
	slli addi addi add.n addi addi l32i.n l32i l32i slli sub extui ssl sll addi addi l32i.n l32i	0,0,1,2,3,1,2,2,1,2,2,0,0,3,1,0,0, 1,0,0,1,4,3,1,5,3,1,5,5,1,5,5,4,5, 5,1,5,6,0,0,7,3,1,7,7,1,7,7,1,8,3,1	16
des3 cbc sha	slli l32i.n l8ui slli or or l32i.n l8ui or s32i.n l32i.n l8ui slli l32i.n l8ui slli l32i.n l8ui slli or or l32i.n l8ui or s32i.n l32r	0,0,1,2,3,1,2,2,1,2,2,1,2,2,0,2, 2,4,5,3,1,5,5,1,5,5,2,5,3,1,6,3,1, 6,6,1,6,6,1,5,3,1,5,5,1,5,5,1,7,3, 1,7,7,1,7,7,1,7,7,5,7,7,6,8,3,1,8, 8,1,8,8,7,8,3,1,9,1	2
des cbc md5	extui l32i.n srli extui l32r addx4 l32i.n l32r addx4 l32i.n xor l32r addx4 l32i.n xor l32r addx4 l32i.n xor l32i.n xor s32i.n l32i.n l32i.n l32i.n slli l32i.n srli or xor	0,0,1,2,3,1,2,2,1,2,2,1,4,1,2,2, 4,2,2,1,4,1,0,0,4,0,0,1,0,0,2,2, 1,5,5,2,5,5,1,5,5,0,0,1,6,6,0,6,6,1, 6,6,5,7,3,1,7,7,6,7,3,1,8,3,1,8,8,1, 6,3,1,6,6,1,7,3,1,7,7,1,7,7,6,8,8,7	2

Table 4.7: Nesting Level Measurements obtained for some instruction sequences of several cryptographic algorithms.

Algorithm	Instruction Sequence	Operand Symbol String	Nest Level Value
des cbc md5	s32i.n l32i.n l32i.n	0,1,2,3,1,2,4,1,2	1.0
	l32i.n l32i.n l32i.n	0,1,2,3,1,2,3,3,2	0.5
	l32i.n xor l32i.n	0,0,1,0,0,2,3,4,1	2.0
	and or s32i.n	0,0,1,0,0,2,0,3,4	2.0
3des ctr md5	s32i.n s32i.n s32i.n s32i.n s32i.n	0,1,2,3,1,2,4,1,2,5,1,2,6,1,2,	1.0
	l32i.n movi.n sub l32i.n l32i.n	0,1,2,3,2,3,3,0,0,1,2,4,1,2,	2.0
	extui l32i.n srli extui l32i.n	0,0,1,2,3,1,2,2,1,2,2,1,4,3,1,	1.0
camellia ctr md5	l32i l32i.n l32i.n add.n l8ui l32i.n xor sext s8i l32i addi.n s32i l32i.n	0,1,2,3,1,2,4,1,2,4,4,3,4,4,2, 5,1,2,5,5,4,5,5,2,5,0,2,6,1,2, 6,6,2,6,1,2,7,1,2	2.0
	l32i.n addi l32i.n addi l32i.n addi.n l32i.n l32i.n s32i.n l32i.n l32i.n l32i.n l32i.n	0,1,2,0,0,2,3,1,2,3,3,2,4,1,2, 4,4,2,5,1,2,6,1,2,6,1,2,7,1,2, 7,7,2,7,7,2,7,7,21,2,6,1,2,6, 1,2,7,1,2,7,7,2,7,7,2,7,7,2	0.5
	add.n addmi l8ui addi.n addmi s8i movi.n l32i.n l32i.n add.n addmi l8ui addmi l8ui	0,1,2,0,0,3,0,0,2,0,0,2,0,0,2, 4,4,2,0,4,2,0,2,4,1,2,5,1,2,5, 5,4,5,5,2,5,5,2,2,0,2,4,1,2,5, 1,2,5,5,4,5,5,2,5,5,2	2.5
blowfish ctr sha	slli l32i.n add.n l32i.n addx4 addi l32i.n s32i.n l32i.n slli l32i.n add.n l32i.n addx4 addi l32i.n s32i.n l32i.n addi.n s32i.n l32i.n movi	0,0,1,2,3,1,2,2,0,0,3,1,2,2,0, 2,2,1,4,3,1,4,2,1,5,3,1,5,5,1,6, 3,1,6,6,5,5,3,1,6,6,5,6,6,1,7,3,1,7, 6,1,8,3,1,8,8,1,8,3,1,9,3,1,10,1	3.0
	s16i l16ui l16ui movi l32i.n addx4 add.n l32i.n l32i.n addx4 addi l32i.n add.n s32i.n l16ui l32i.n l32r l32i.n addx4 add.n l32i.n xor	0,1,2,3,1,2,4,1,2,5,2,6,1,2,4,4, 6,4,4,5,4,4,2,5,1,2,3,3,5,3,3,2,3, 3,2,3,3,4,3,1,2,7,1,2,8,1,2,9,2, 10,1,2,7,7,10,7,7,9,7,7,2,7,7,8,	1.0
cbc md5 blowfish	l32i.n extui s8i l32i.n l32i.n addi.n l32i.n extui s8i l32i.n l32i.n addi.n 32i.n s8i l32i.n l32i.n addi.n l32i.n srli s8i l32i.n l32i.n addi.n l32i.n extui s8i	0,0,1,0,0,1,0,2,1,3,4,1,5,4,1,5,5, 1,5,5,1,5,5,1,5,3,1,6,4,1,7,4,1,7, 7,1,7,7,1,7,6,1,8,4,1,9,4,1,9,9, 1,9,9,1,9,9,1,9,8,1,10,4,1,11,4,1, 11,11,1,11,11,1,11,11,1,11,10,1	1.0
aes cbc md5	l32i.n l32i.n l32i.n extui l32i.n extui extui l32i.n srli extui l32i.n extui l32r addx4 l32i.n xor l32r addx4 l32i.n xor l32r addx4 l32i.n xor l32r addx4 l32i.n xor s32i.n l32i.n	0,1,2,0,0,2,3,1,2,3,3,2,4,1,2,4, 4,2,4,4,2,5,1,2,5,5,2,5,5,2,6,1,2, 6,6,2,7,2,6,6,7,6,6,2,6,6,0,0,2,5, 5,0,5,5,2,5,5,6,6,2,4,4,6,4,4,2,4,4, 5,5,2,3,3,5,3,3,2,3,3,4,3,1,2,8,1,2,	1.5

Table 4.8: Top characteristic operations resulting from the analysis.

Instruction Sequence	Operand Symbol String	Common Algorithms	Nest Level	Static Fre.
l32i.n l32i.n	0,1,2,3,1,2	8	11	1005
s32i.n l32i.n	0,1,2,3,1,2	8	8	1043
l32i.n l32i.n	0,1,2,0,0,2	8	8	969
add.n l8ui	0,0,1,0,0,2	8	18.5	187
addx4 l32i.n	0,0,1,0,0,2	8	13	467
l32i.n add.n	0,1,2,0,0,3	8	19	126
movi.n l32i.n	0,1,2,3,1	8	11	522
l32i.n xor	0,0,1,0,0,2	8	13	382
l32i.n add.n l8ui	0,1,2,0,0,3,0,0,2	8	18.5	41
l32i.n l32i.n add.n l8ui	0,1,2,3,1,2,3,3,0,3,3,2	8	18.5	37
s8i l32i.n	0,1,2,3,4,2	8	15.5	183
l8ui l32i.n	0,0,1,2,3,1	8	18.5	17
l32r addx4	0,1,2,2,0	6	13	420
l32i.n l32i.n add.n	0,1,2,3,1,2,3,3,0	8	17	83
l32r addx4 l32i.n	0,1,2,2,0,2,2,1	6	13	402
movi.n s32i.n	0,1,0,2,1	8	10	412
l32i.n extui	0,1,2,0,0,2	8	9	439
l32i.n slli	0,1,2,0,0,2	8	14	170
l32r l32i.n	0,1,2,3,1	8	7	523
xor s32i.n	0,0,1,0,2,3	8	10	366
s32i.n l32i.n addi.n	0,1,2,3,4,2,3,3,2	8	16	56
s32i.n l32i.n addi.n s32i.n	0,1,2,3,4,2,3,3,2,3,4,2	8	16	44
s32i.n s32i.n	0,1,2,3,1,2	8	8	448
s32i.n l32i.n addi.n s32i.n l32i.n	0,1,2,3,4,2,3,3,2,3,4,2,5,4,2	8	16	38
l32i.n xor	0,1,2,0,0,3	8	12	219

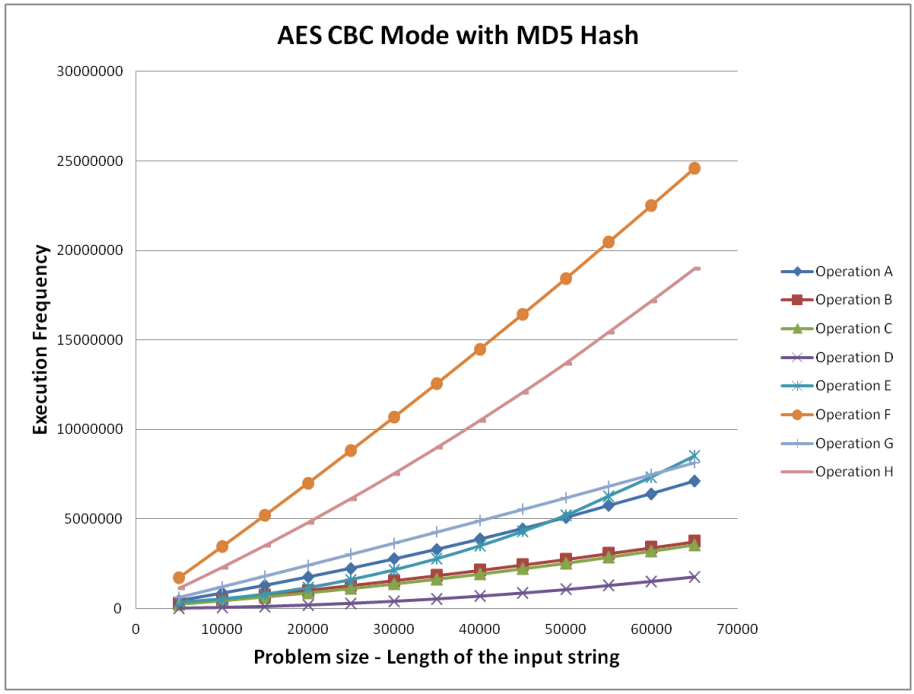


Figure 4.17: Execution Frequency of AES algorithm with CBC mode and MD5 hash function

Evaluation criteria for experiment cryptographic algorithms

As explained above, the resultant operations needed to be evaluated in terms of their execution criticality to validate our analysis strategies. Therefore, if the top operations in the resultant operation list are consistently executed frequently in the run time the accuracy of the analyses can be confirmed. To achieve this, the operations are expected to show increasing variations of their execution frequency with respect to the increasing problem size. The following section presents the dynamic experiment performed for this purpose and discusses its outcome.

Evaluation experiment for results of cryptographic algorithms

For the top eight operations presented in Table 4.8, the runtime execution frequencies were counted. In the experiment, a data set of the size ranging from $10,000$ to $70,000$ is used. The size of the *plain text* input is gradually increased. For all considered algorithms, the variation of runtime execution of these operations was obtained and is illustrated in Figures 4.17 to 4.24. It is clearly seen from these figures that the execution of all the selected operations increases smoothly along with the problem size. This confirms that these operations are critical in terms of execution frequency even in a dynamic environment, as our static analysis accurately forecasted.

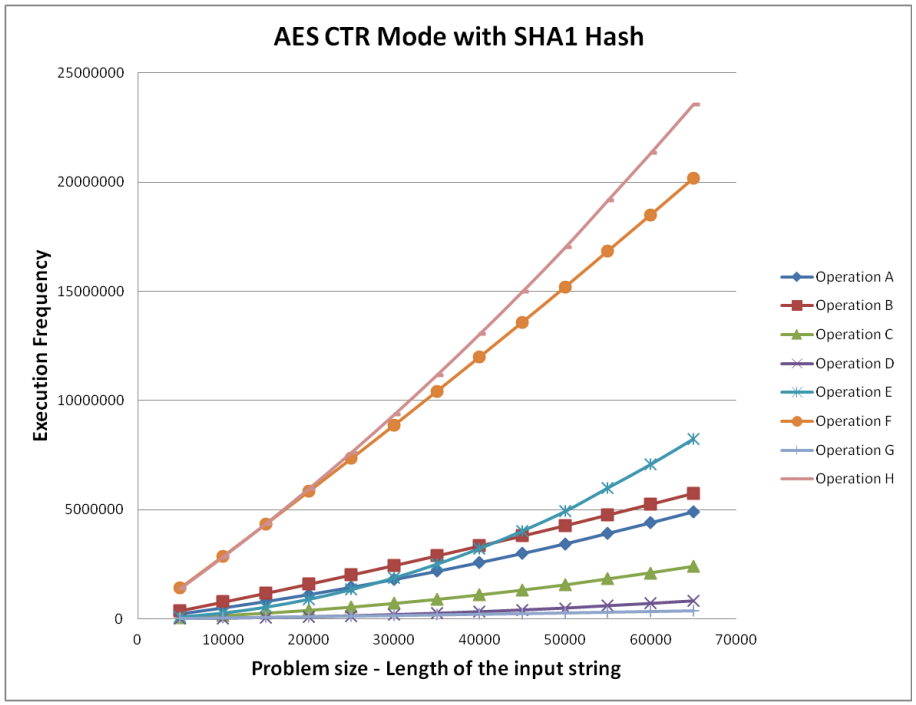


Figure 4.18: Execution frequency of AES algorithm with CTR mode and SHA1 hash function.

4.5.2 Results obtained for exact string matching algorithms

As described above, another experiment was also carried out for the *exact string matching* algorithms listed in Table 2.1. From the operations identified in our analysis as presented in Tables 4.3, 4.4 and 4.5, a few top operations were selected for runtime behaviour analysis in a dynamic environment. The selected operations are shown in Table 4.9.

Table 4.9: Candidate set of special-purpose operations.

Operation	Functionality	Reference
$p[i]$	Calculating an index of an array element	Table 4.3, Operation No. 1
$p[i] == m$	Compare the value of an array element	Table 4.4, Operation No. 3
$p[m] == q[m+i]$	Comparing next character of pattern with text	Table 4.4, Operation No. 1

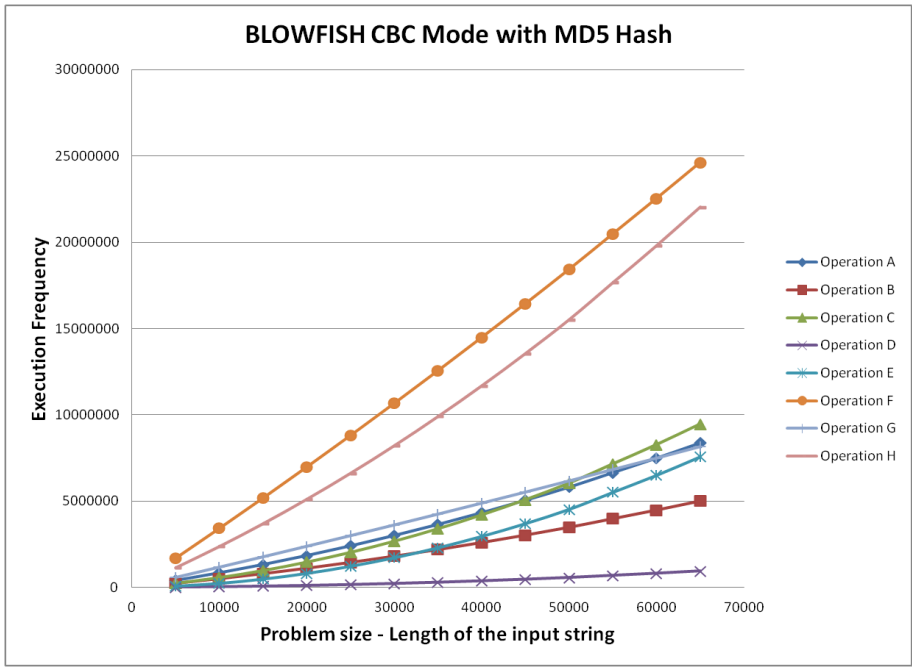


Figure 4.19: Execution frequency of BLOWFISH algorithm with CBC mode and MD5 hash function.

Evaluation criteria for results of exact string matching algorithms

In this experiment, similar evaluation criteria is used as in the previous experiment. Thus, incremental variations of execution frequencies with regard to increasing problem size are expected by the above selected operations. In addition to execution frequency variations, the relations between the exact string matching algorithms in terms of their historical background also can be used to evaluate the accuracy of the analysis strategies. Various developments to optimise the functionality of string matching were introduced in many forms of algorithms during the last few decades. Therefore, the progression of these algorithms and their performance improvements can be clearly observed from the literature. From these observations, relations between each other can be identified and hence foresee the similarities of their fundamental characteristics. If the resultant execution variations are capable of showing the similar patterns of relations between the algorithms consistent with the historical background evidence, the accuracy of our analysis approach can be confirmed. Thus, this evaluation experiment compares the execution variations of the selected top operations of the exact string matching, not only operation wise but also between algorithms.

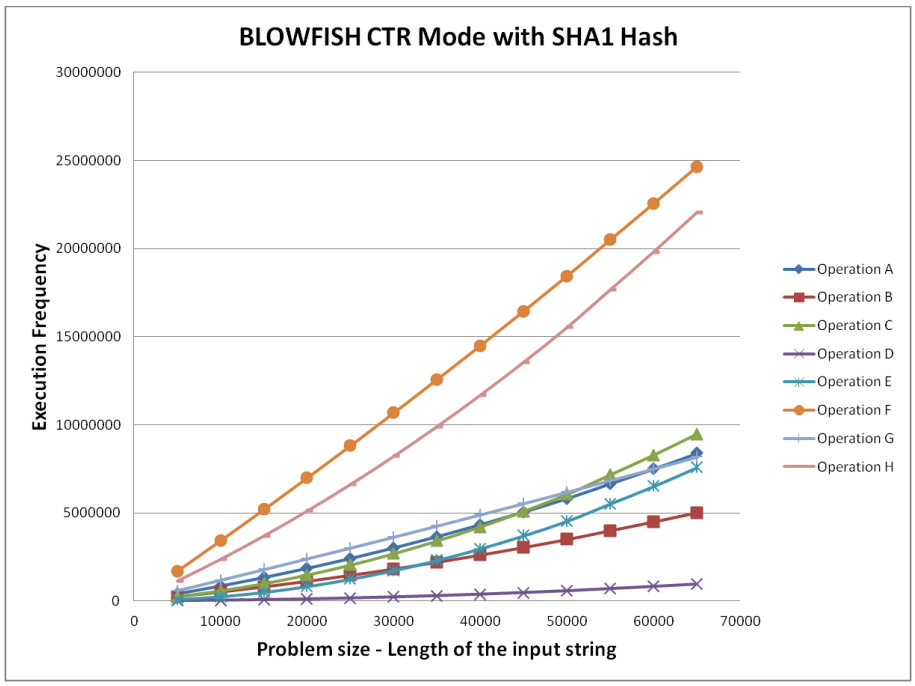


Figure 4.20: Execution frequency of BLOWFISH algorithm with CTR mode and SHA1 hash function.

Evaluation experiment for results of exact string matching algorithms

The frequency of each characteristic operation was measured for different problem sizes to analyse their corresponding frequency variations. To obtain a comprehensive set of results, the following experimental matrices were used. The length of the input *text* data ranged from $10,000$ to $20,0000$ characters, while the length of the input *pattern* ranged from 5 to 100 characters. A standard library of genome sequences was used to generate *text* and *pattern* strings. In the practical set-up, the *text* string was generated by using the same set of genomes, while *patterns* were generated randomly for each trial. The problem size was defined as $text\ length\ -\ pattern\ length\ +\ 1$, because this is at how many distinct locations the pattern may appear in the text.

The results obtained in the experiment are illustrated in Figures 4.25 to 4.28. Here the variations of the selected operations' runtime execution is plot against all the considered algorithms. Thus, the nature of the behaviour of different algorithms can be clearly seen. As mentioned earlier, the execution behaviour of an algorithm can be denoted by the execution behaviour of its characteristic operations [Levitin, 2007]. Therefore, in this experiment, if our static analysis was able to rank truly characteristic operations of the algorithms highly, these operations convey the native properties of the algorithms. Thus, we can analyse the accuracy of

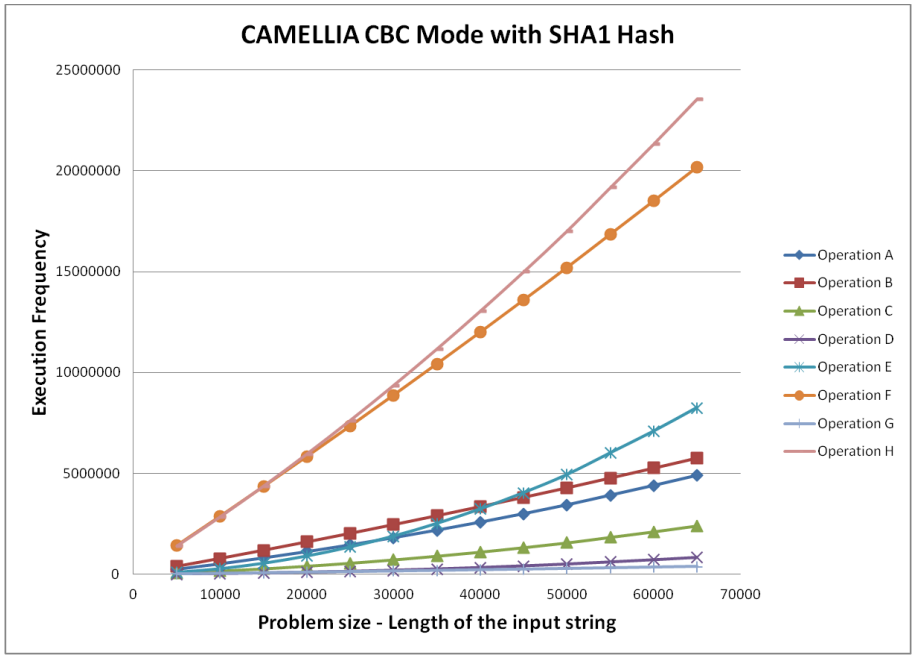


Figure 4.21: Execution frequency of CAMELLIA algorithm with CBC mode and SHA1 hash function.

the resultant operations using known facts about *exact string matching* algorithms.

It is seen that the lines in the figures increase linearly with the problem size. This is consistent with the fundamental nature of searching problems. Hence, it demonstrates the accuracy of our dynamic analysis results, and confirms the importance of the operations chosen by our static analysis process. Figure 4.25 shows the number of trivial array indexing operations performed by each algorithm. Array indexing, *p[i]*, is the first special-purpose operation in Table 4.9. Notably, algorithms related to one another generally appear together:

- The BM, TBM, ZT and AG algorithms, which all lie at the bottom of the graph, are related to each other since they are different deviations of the original BM algorithm that uses a *bad character shift* [Charras and Lecroq, 2004].
- The RT and HP algorithms also use similar functionalities in their matching phase and hence they appear together in the bottom of the graph [Charras and Lecroq, 2004].
- The CL and GG algorithms are related to each other since GG is a refinement of CL [Charras and Lecroq, 2004]. This is evident in the graph since these algorithms appear close to each other.
- The KMP and AC show close behavior to each other according to the graph. These

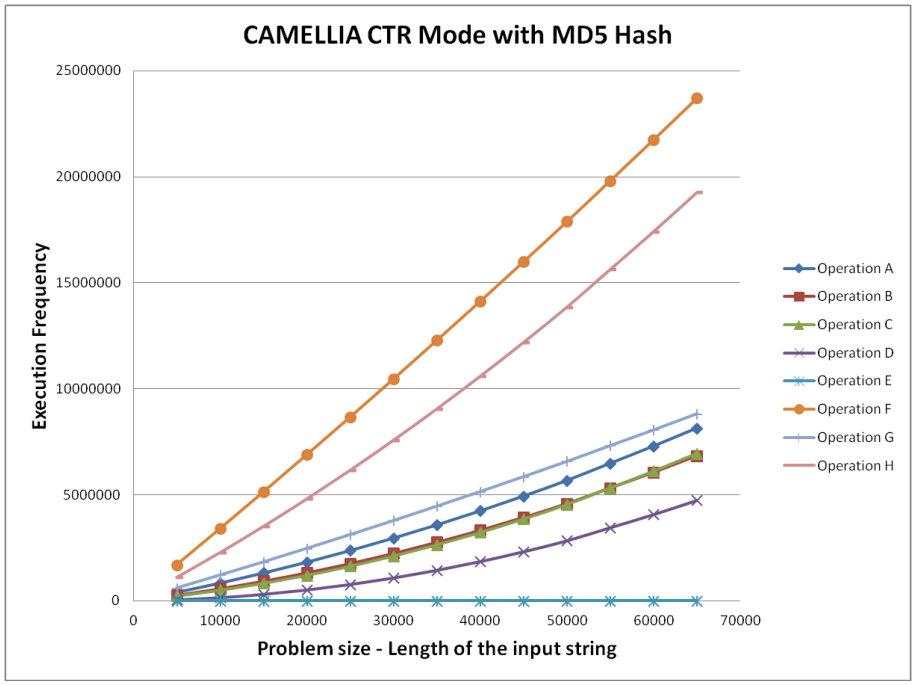


Figure 4.22: Execution frequency of CAMELLIA algorithm with CTR mode and MD5 hash function.

two algorithms are also functionally related, since both use the *shift table* technique for window shifting [Charras and Lecroq, 2004]. However, although the algorithm MP also uses the same technique, it appears at the top of the graph. This might be because the KMP and AC are extended versions of the original MP algorithm and hence they are optimised further [Charras and Lecroq, 2004].

- The algorithm NSN remains isolated since it does not have any similarity to any other considered algorithm.

Figure 4.26 shows similar results for the more complex array assignment operation, $p[i] == m$. In this case, only a subset of algorithms benefited significantly from this choice of operation.

- The HP and RT algorithms are closely related and both rely heavily on this operation.
- The TBM algorithm introduces this operation to effect a shift, so appears near the top of the graph, whereas the original BM algorithm appears near the bottom.
- The AC algorithm uses this operation in order to calculate shift indexes under particular conditions in its core matching phase. Therefore, it appears at the top of the graph while the original MP algorithm lies at the bottom of the graph [Charras and Lecroq, 2004].

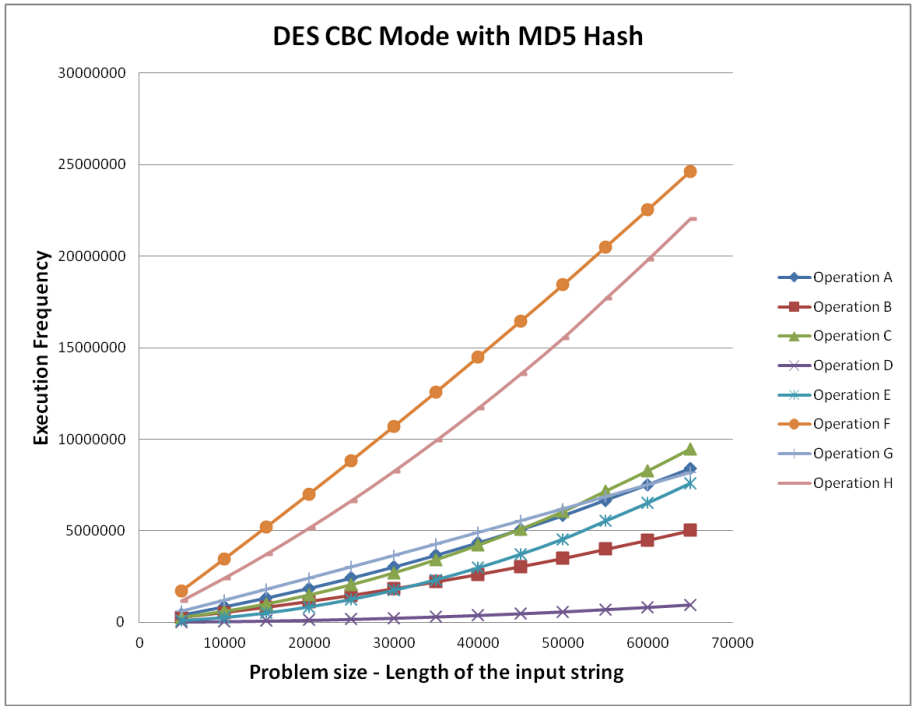


Figure 4.23: Execution frequency of DES algorithm with CBC mode and MD5 hash function.

Finally, Figure 4.27 shows all characteristic operations from Table 4.9 as used by all algorithms. Figure 4.28 expands the view of the collection of algorithms at the bottom. Exceptionally, the NSN algorithm has reached the top of the graph, indicating it being benefited by the most of the selected operations.

These results from independent dynamic analysis confirm the expected behaviour of the considered algorithms and the relations between them. They demonstrate that operations identified by our static analysis are truly critical for the algorithms. These operations are similarly critical in a dynamic environment.

4.6 Summary of Static Analysis Strategies

This chapter explains the concepts and the development of the static analysis strategies used to identify characteristic operations of our framework. This phase lays the foundation of the framework by supporting deep analysis strategies to identify executionally critical operations in the low level implementation. However, although this approach is capable of identifying such operations, the scope of the resultant operations is limited. For instance, the experiment discussed in Section 4.3.1 considers operations only up to 30 instructions long. This limitation, therefore, may avoid finding characteristic operations which are longer than that. Thus, to

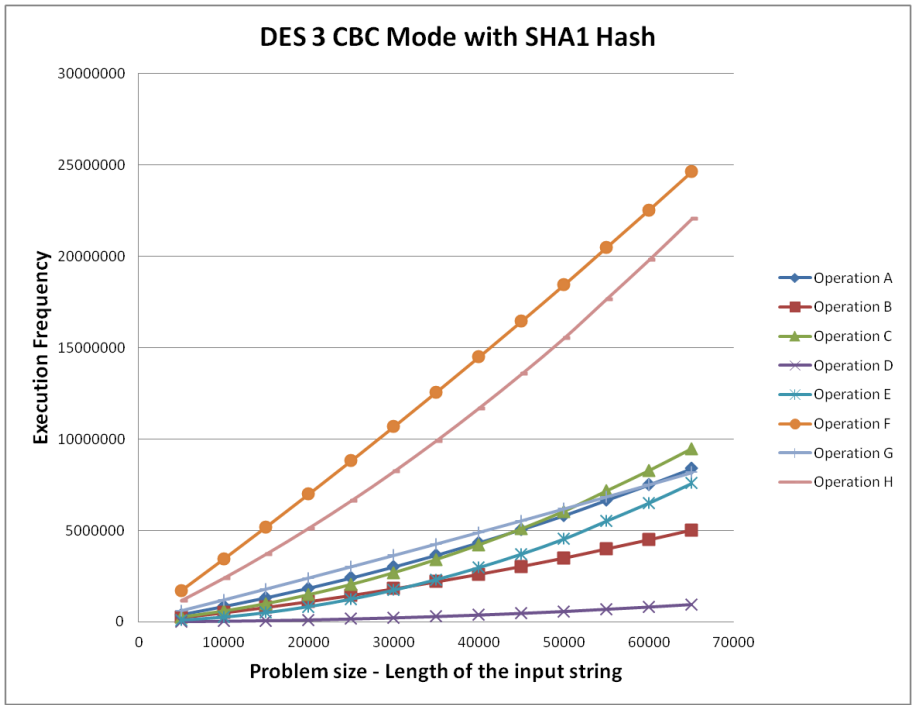


Figure 4.24: Execution frequency of DES3 algorithm with CBC mode and SHA1 hash function.

improve the effectiveness of the resultant operations, a well guided parameter selection will be useful.

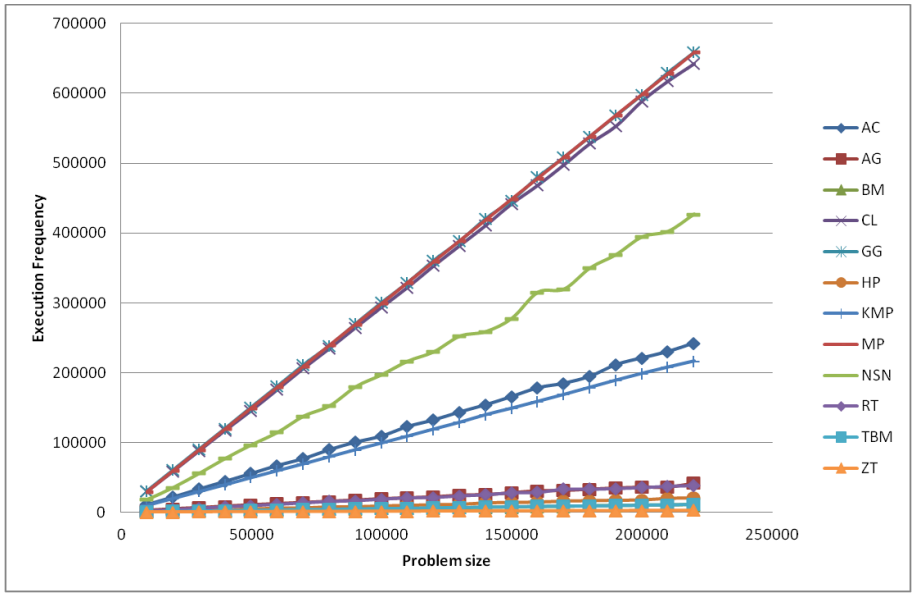


Figure 4.25: Frequency of operation $p[i]$ for pattern size = 100.

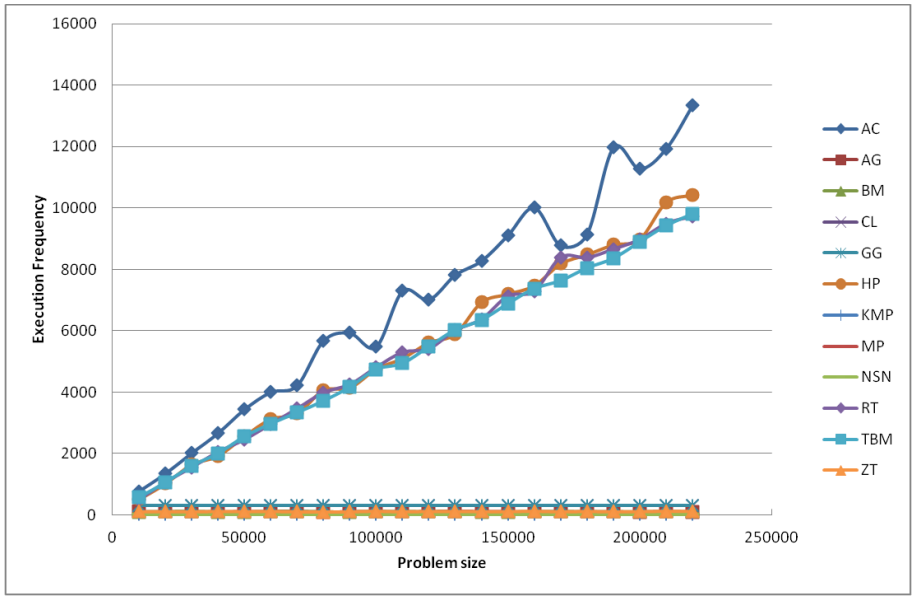


Figure 4.26: Frequency of operation $p[i] == m$ for pattern size = 100.

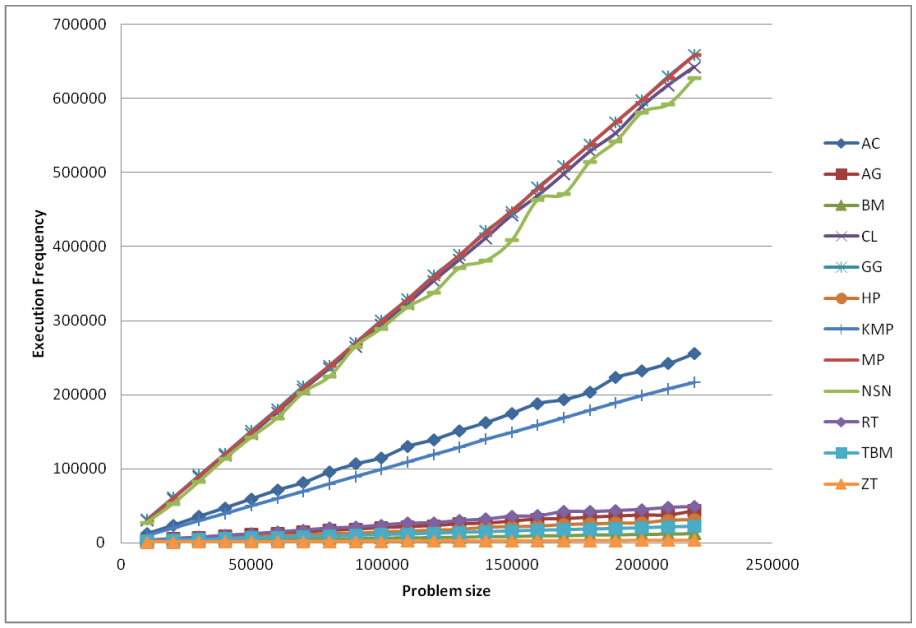


Figure 4.27: Dynamic frequencies of operation for pattern size is 100.

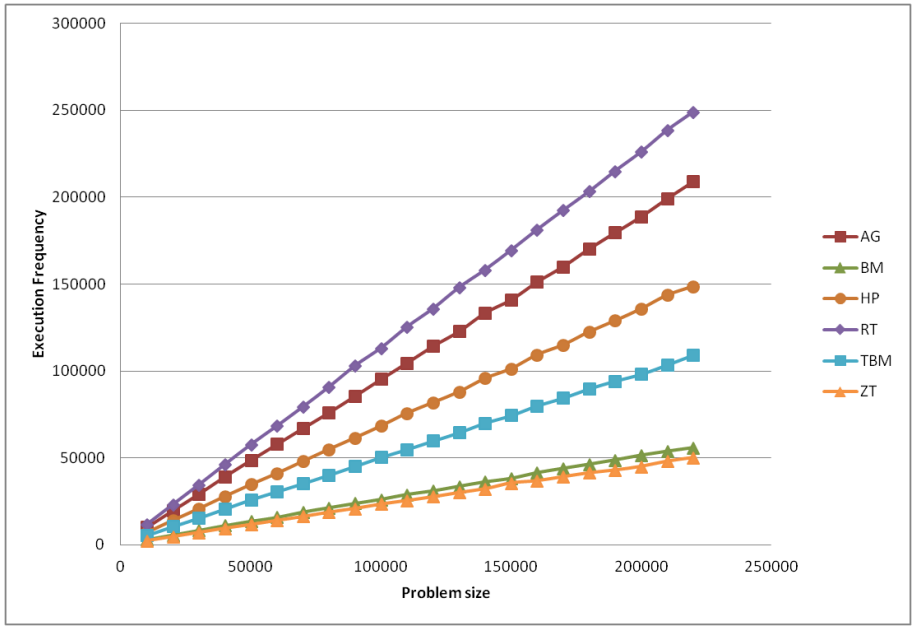


Figure 4.28: Zoom-in of the lower group of algorithms from Figure 4.27 (pattern size is 5).

Chapter 5

Algorithm Clustering Methodology

The primary goal of this research is identifying relationships between multiple algorithms. Common functionalities identified between multiple algorithms by characteristic operations can be directly used to improve their performance at once. However, when the number of algorithms to be considered is very large, operations critical to *all* of these algorithms may not exist. In such a situation, selecting the most appropriate set of operations for every algorithm would not be feasible. This is because at least a few algorithms may be disadvantaged when the operation choices are made. Nevertheless, these kinds of situations can be avoided if we have the knowledge of the characteristic operations of each algorithm beforehand. Thus, very different algorithms, which are unlikely to share similar operations, can be separated and considered as different sets of algorithms. Such differentiation can be achieved using an algorithm clustering approach that groups algorithms according to their common characteristic operations. If the clustering is successfully performed, the resulting common operations will be much more effective and efficient towards a particular application. The reason for that is, accurately grouped algorithms sets would indeed share similar characteristic operations [Arnold and Corporaal, 1999]. Thus in this chapter, we present such an algorithm clustering methodology to bind similar algorithms together considering the common operations they use.

In general, algorithm clustering has been a widely discussed topic in many application areas. Algorithm clustering is used to classify a given set of algorithms into different groups. This is done based on various properties such as the software implementation, execution performance, hardware cost, basic principles and specific functionalities. These clustering methods are performed targeting various purposes including: understanding the state-of-the-art, finding

the most suitable algorithm for a task, studying usage of basic functionalities or principles, identifying gaps, and analysing performance variations. Since our methodology performs the clustering based on common operations, many other application areas depended on common functionalities will also be served. For instance, in Application-Domain Specific Processor (ADSP) design, customised instructions are designed targeting common operations found from a set of given algorithms. In this case, our clustering approach can be applied to identify the best set of algorithms, which share the most common operations between them. Thus, this approach can be used as a stand-alone general tool, to classify a given set of algorithms based on their characteristic operations.

5.1 Basic Concepts of the Algorithm Clustering Approach

As mentioned above, our approach classifies a given set of algorithms into closely related groups considering their common operations. The operations of an algorithm become the key parameter to represent its behaviour against other algorithms in the clustering process. Therefore, selecting a critical set of operations of each algorithm in terms of its execution behaviour is essential. According to the concepts of complexity analysis, the behaviour of an algorithm is expressed by its "basic operations" [Levitin, 2007] where most of their processor cycles are spent due to their frequent execution. Therefore, in our clustering, the most frequently executed operations of each algorithm are used for the clustering process.

Clearly, this could include usage of the static analysis strategies presented in Chapter 4 to identify the frequently executed operations of algorithms. However, this algorithm clustering approach is presented as an independent and self-contained one. Therefore, the freedom of selecting the most critical operations is not limited to our static approach here, but is widened for any other available algorithm analysis strategy such as dynamic approaches, static template matching strategies and complex analysis strategies.

Our general clustering approach measures the relationships between algorithms by the number of characteristic operations they share. It also allows the user to calibrate the closeness of algorithms by defining thresholds for the expected similarities in terms of shared operations. This helps the clustering approach to be more flexible since it can be applied to different algorithm sets by setting up input parameters appropriately.

However, the main challenge of the clustering process is representing the relationships

between an algorithm and its operations. This representation is needed to support comparison of multiple algorithms accurately. To overcome this challenge, the problem is converted to a mathematical model where algorithms are represented as vectors. Then, relationships between these vectors are studied statistically to form clusters. However, to confirm that such a static analysis process accurately produces cohesive clusters, we also need to prove that the dynamic properties of algorithms in the same cluster are similar. Therefore, finally we show how the resulting clusters of algorithms can be analysed to confirm their cohesiveness with respect to the primitive operations they actually execute at run time.

5.2 Development of Algorithm Clustering Method

Our algorithm clustering method consists of six main steps. It starts by taking algorithms and their frequently executed operations. It produces groups of algorithm clusters based on the operations they have in common. Figure 5.1 shows the overview of these steps. The outline of these steps is presented in the following procedure. These steps are described in detail in the next subsections.

Algorithm clustering procedure	
Step 1:	**Selection of characteristic operations** *The most critical set of operations in terms of execution behaviour of the objective algorithms are determined.*
Step 2:	**Generating the Boolean table** *Existence and non-existence of all the selected characteristic operations within the algorithms are indicated.*
Step 3:	**Vector representation** *Relations between algorithms and the operations are defined in a mathematical domain.*
Step 4:	**Distance calculation** *Similarities and dissimilarities between each algorithm pair are determined numerically.*
Step 5:	**Centre and member points selection** *Data points are arranged considering their distances from each other in the mathematical model to investigate potential clusters.*
Step 6:	**Generating clusters** *Algorithm clusters are formed based on the learnt mutual relationships between the algorithms.*

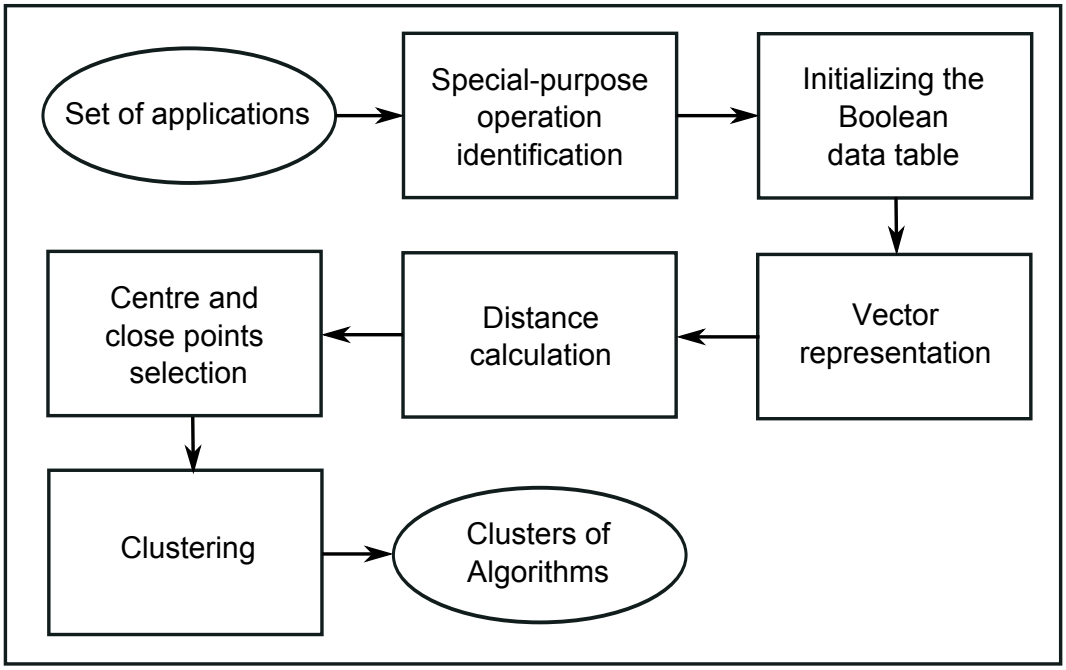


Figure 5.1: Overview of the application clustering method

5.2.1 Characteristic Operation Identification

As outlined above, the first step is to identify the characteristic operations of the considered algorithms. They are the best to characterise each algorithm's purpose and where the most of its execution time is devoted. As per Section 5.1, the characteristic operations are retrieved by analysing all the considered algorithms.

5.2.2 Initialising the Boolean Data Table

In this step, the characteristic operations found in each algorithm are listed against all the algorithms. The existence and non-existences of each operation within the algorithms are clearly indicated in a data table. This data representation helps to organise the details of similar and dissimilar operations between the algorithms, to process in the next steps. From our analysis steps, characteristic operations are determined with high frequency scores (static frequency or nesting level values). Since we have already identified these operations as significant to runtime performance, we merely need to note their frequency scores in each algorithm. So a Boolean table is sufficient to represent these data. Table 5.2 shows a sample of such a table.

5.2.3 Vector Representation

The data collected in the above step needs to be analysed to find the degree of similarities between the algorithms. Thus, comparisons between each can be performed in terms of their operations. However, since the number of algorithms and the number of operations are large, similarities or differences are difficult to find manually at this stage. In order to support this, we use a mathematical representation to present the above details precisely. Each algorithm is converted to a Boolean vector defined by its operations. The coordinates of this vector are explained by the existence and non-existence of operations to act as independent variables in this domain. Thus, each algorithm can be represented as a multi-variable equation P such that,

$$P = Aa + Bb + Cc + \cdots = (A, B, C, \dots)$$

where a, b and c are independent variables to represent the presence or absence of characteristic operations and A, B, and C are coefficients, which are either 1 or 0.

For all considered algorithms, corresponding vectors are thus obtained. This vector representation turns the algorithm clustering problem into a mathematical vector model. It defines the properties of algorithms formally in terms of their characteristic operations.

5.2.4 Distance Calculation

The next step is determining similarities and dissimilarities between algorithms in the mathematical domain. Once the objective algorithms are represented by vectors, these vectors can be considered as points in a multidimensional space. Here, the number of dimensions indicates the number of operations considered. The distance between two such points is proportional to the difference between corresponding algorithms in terms of their characteristic operations. In a multidimensional mathematical space, the distance between two points can be determined by calculating their Euclidean distance. Thus, the difference between two algorithms can be represented by a numerical value calculated using the following equation.

Definition 1 (Euclidean distance) *If points A and B can be represented as an equation of n independent variables then the Euclidean distance of A and B, denoted by D_{AB}, is:*

$$D_{AB} = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2 + \cdots + (a_n - b_n)^2}\,.$$

5.2.5 Centre and Member Points Selection

In this step, these values are utilised to form clusters from the given data points and hence the corresponding algorithms. Whereas some clustering techniques begin from a random point [MacQueen, 1967], we instead identify the most promising *centre points* of potential clusters first. This reduces the complexity of the clustering by avoiding repetitive iterations to find the centres and their members, and comparisons between formed groups.

Maximizing the number of members of a cluster is essential for our purposes. The primary target of the clustering is finding close relations between algorithms so that improvements of critical common functionalities can benefit multiple algorithms at once. Therefore, by maximizing the number of members in a single cluster with the given expected similarities, more algorithms are benefited. For this reason, the clustering starts by forming the best possible cluster first, to include the maximum number of algorithms, and next the second best cluster and so on.

To form clusters, their centre and member points must be determined first. Here, the maximum distance between a centre point and its member points is defined by a constant value called the *distance constant* (DC). As described above, the difference between each algorithm pair is measured by the distance of their sets of characteristic operations. The DC value is calculated based on the expected number of different operations between the pairs (or conversely the number of shared operations). Thus, it is a threshold value to represent the maximum expected difference between a cluster's centre algorithm and its member algorithms. The maximum possible distance between two points, and hence two algorithms, in the same cluster will be $2 \times DC$.

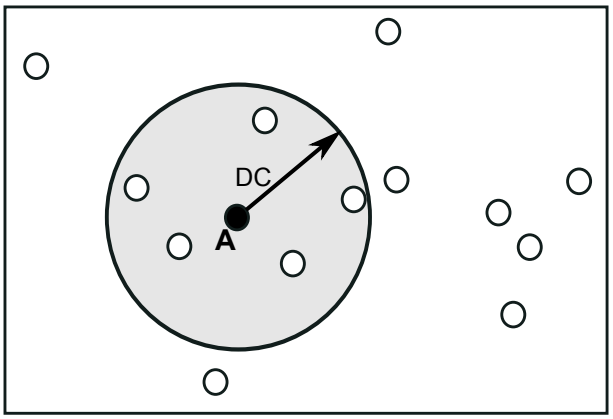


Figure 5.2: Close points to A as per distance constant DC

In our clustering strategy, initially all the points in the multidimensional space are considered as potential centre points. Then, for each centre point, the potential members that are located less than DC units away from the centre point are determined. These points are added to each centre point's temporary member point list in order to form clusters in the next step. The user is allowed to choose a suitable DC value, based on the desired coherence of the cluster. For instance, if a user expects to allow $p\%$ minimum similarity of characteristic operations between a cluster's centre and its member points, the DC threshold can be calculated as follows.

Definition 2 (Number of shared operations) *When the total number of operations considered is denoted by O_n, then the required number of operations to be shared, O_s, is*

$$O_s = p \times O_n/100.$$

Definition 3 (Distance constant) *The distance constant is*

$$DC = \sqrt{O_n - O_s}.$$

The DC threshold can vary depending on the number of algorithms, the number of characteristic operations and their expected behaviour. Therefore, users can decide on a suitable similarity percentage and hence calculate DC for the particular algorithm set of interest.

5.2.6 Clustering

Once all centre points and their member points are defined, the clustering is carried out as described in the following key steps.

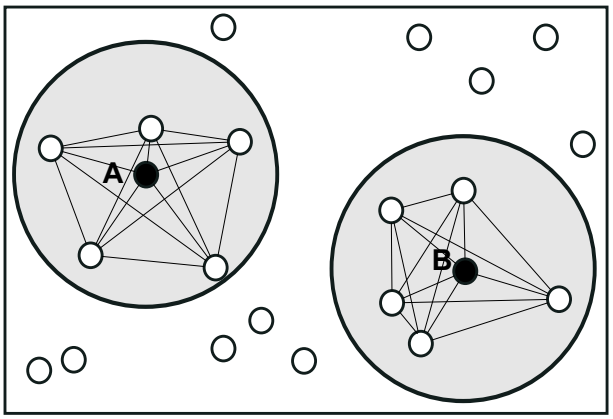


Figure 5.3: Two groups with the same number of close points

1. Selecting the first centre point

   Clustering begins from the centre point which has the longest close point list. This is guaranteed to form the largest possible set of algorithm clusters eventually to satisfy our main performance improvement target. However, sometimes more than one centre point can have the same highest number of member points in its group as shown in Figure 5.3. In that case, a *priority list* is generated in order to determine the first centre point, using the following criteria. First, the total of all Euclidean distances between all pairs of points in the groups are calculated. This can be used to measure the closeness between centre and member points of current clusters. If the total distance of a group is small, the differences between the algorithm pairs in the group are likely to be small. Therefore, this group is more cohesive and has a higher priority for becoming a useful cluster. The groups that have low values for their total distance will have higher cohesiveness and higher priority. Then, the centre point which owns the highest priority group is used as the first starting centre point. The first centre point along with its member points forms a cluster.

2. Updating the centre point list along with their member points

   When the first cluster is formed, all members of this newly created cluster are removed from the centre point list. They cannot form the centre point of another cluster since they are already in a cluster. In addition, all the members are removed from other points' member point lists in order to avoid overlapping clusters. That is, a point can be a member of only one cluster.

3. Searching for the next most eligible centre point

   Then, the centre point that has the longest member point list will be the next centre point of the newly modified centre point list after these eliminations, and so on. If there is more than one candidate centre point, the same criteria as above are followed to determine the priorities of each point and hence find the next centre point.

Clustering thus continues until all the points in the centre point list are covered or removed. The clusters obtained in this step represent the final set of algorithm groupings.

5.3 Experimental Results of the Algorithm Clustering Approach

This section demonstrates a case study carried out to evaluate our algorithm clustering methodology. The experiment was performed on the twelve *exact string matching* algorithms listed in Table 5.1. According to the methodology described in the previous section, the algorithms are classified based on their frequently executed operations. The following sections describe the evaluation experiment with its results and the result validations.

Table 5.1: String matching algorithms analysed.

Algorithm	Abbreviation
Apostolico-Crochemore	AC
Apostolico-Giancarlo	AG
Boyer-Moore	BM
Colussi	CL
Galil-Giancarlo	GG
Horspool	HP
Knuth-Morris-Pratt	KMP
Morris-Pratt	MP
Not So Naive	NSN
Raita	RT
Turbo Boyer-Moore	TBM
Zhu-Takaoka	ZT

All above string matching algorithms perform the same matching functionality in different ways. In fact, most of them are fundamentally related to each other since they are different deviations of or improved versions of early root algorithms. It is beneficial to examine these relations between algorithms historically in advance to analyse the results of our clustering at the end of the experiment. Nevertheless, basic historical relation introduction is already presented in Section 4.5.2 when the results of our static analyses are evaluated. However, we present a similar kind of study here in more detail.

From the algorithm list selected for this case study, two well-known algorithm families can be identified. One is the Boyer-Moore family. Inheriting from the root Boyer-Moore algorithm, many algorithms were developed to improve the original implementation. The Turbo Boyer-Moore, Zhu-Takaoka and Apostolico-Giancarlo are some of them [Charras and Lecroq, 2004]. Also, as the name suggests, the Turbo Boyer-Moore is an extension of the Boyer-Moore algorithm [Charras and Lecroq, 2004]. The Zhu-Takaoka is a variant of the Boyer-Moore

algorithm and performs shifting by considering the "bad-character shift" approach introduced by the Boyer-Moore algorithm for two consecutive text characters [Charras and Lecroq, 2004]. The Apostolico-Giancarlo algorithm is also a variant of the Boyer-Moore; it addresses the problem of forgetting characters it has already matched after each attempt [Charras and Lecroq, 2004]. Thus, all four algorithms share similar characteristics in terms of the way they perform string matching.

The other algorithm family found is the Morris-Pratt family. Similar to the Boyer-Moore algorithm in the above group, here Morris-Pratt is the root algorithm. From the selected algorithms, the Knuth-Morris-Pratt and the Apostolico-Crochemore algorithms belong to this family. The Knuth-Morris-Pratt is an extension of the original Morris-Pratt algorithm with slight advancements. Also, the Apostolico-Crochemore algorithm uses the *Morris-Pratt shift table* for window shifting in the *text*. Thus, these algorithms are related to each other based on how they perform the matching phase.

Another noticeable relation is found among the Colussi and the Galil-Giancarlo algorithms. The Galil-Giancarlo algorithm is a refinement of the Colussi algorithm [Charras and Lecroq, 2004]. The only difference is that it intervenes in the searching phase [Charras and Lecroq, 2004]. Both of them use similar pre-processing concepts, therefore these two algorithms also can be considered as related algorithms belonging to the same family.

According to the literature, the Horspool, Raita and Not So Naive algorithms do not share similar relations with other considered algorithms. Therefore, they should not appear in the other algorithm groups if the clustering approach is successful.

5.3.1 Characteristic Operation Identification

This is the first step of the clustering process. In the experiment, the most frequently executed operations of the considered algorithms were identified using our static analysis strategies. Here, considering all such operations for the clustering is important, since each operation contributes to the execution behaviour of the algorithm significantly. However, when all such operations are found, those operations existing only within a single algorithm are ignored, as they are not useful for developing relations between multiple algorithms. In this case study, twenty-six operations were selected to represent the considered string matching algorithms.

Once the characteristic operations were selected, a binary table was created. It indicates

Table 5.2: Characteristic operations found in the string matching algorithms

characteristic Operations	MP	KMP	AC	BM	TBM	ZT	HP	RT	CL	GG	NSN	AG
a) $x[i] \neq x[j]$	1	1	1	0	0	0	0	0	0	0	1	0
b) $m = x[i]$	1	1	1	0	1	0	0	0	1	1	0	1
c) $x[i] == x[j]$	0	1	1	0	0	0	0	0	1	1	1	0
d) $x[i] = y[j]$	0	1	1	0	0	0	0	0	0	0	0	0
e) $x[i] == y[i+j]$	0	0	1	1	1	1	0	0	0	1	0	1
f) $x[i] = m$	1	1	1	1	1	1	1	1	1	1	0	1
g) $x[y[i]] == m-i-1$	0	0	0	1	1	0	1	1	0	0	0	1
h) $x[i+m-1-n] < i-p$	0	0	0	1	1	1	0	0	0	0	0	1
i) $x[i] = x[i+m-1-n]$	0	0	0	1	1	1	0	0	0	0	0	1
j) $x[i] == x[i+m-1-n]$	0	0	0	1	1	1	0	0	0	0	0	1
k) $x[i] == m$	0	0	0	1	1	1	1	1	1	1	0	1
l) $i < m-1-j$	0	0	0	1	1	1	0	0	0	0	0	1
m) $i = x[m] + 1$	0	0	0	1	1	1	1	1	1	1	0	1
n) $x[i] = m-1-i$	0	0	0	1	1	1	0	0	0	0	0	1
o) $x[m-1-y[i]] = m-1-i$	0	0	0	1	1	1	0	0	0	0	0	1
p) $m = x[i+m-1]$	0	0	0	0	0	0	1	1	0	0	0	0
q) **memcmp** : $compare\ memory\ location$	0	0	0	0	0	0	1	1	0	0	1	0
r) $x[i-j] + m < i$	0	0	0	0	0	0	0	0	1	1	0	0
s) $x[i] = x[i-j] + m$	0	0	0	0	0	0	0	0	1	1	0	0
t) $m < i + x[j]$	0	0	0	0	0	0	0	0	1	1	0	0
u) $x[h[i]] == y[j+h[i]]$	0	0	0	0	0	0	0	0	1	1	0	0
v) **max** = $((m) > (n))\ ?\ (p) : (q)$	0	0	1	1	1	1	0	0	0	0	0	1
w) **while** $(i > -1\ \&\ x[j] \neq x[i]),\ i = a[i]$	1	1	1	0	0	0	0	0	0	0	0	0
x) **for** $(i = 0;\ i < m;\ ++i),\ x[i] = n;$	0	0	0	1	1	1	1	1	1	1	0	1
y) **for** $(i = 0;\ i < m-1;\ ++i),\ x[y[i]] = m-i-1;$	0	0	0	1	1	0	1	1	0	0	0	1
z) **for** $(i = 0;\ i \leq m-2;\ ++i),$ $b[m-1-x[i]] = m-1-i;$	0	0	0	1	1	1	0	0	0	0	0	1

the existence of each operation against each algorithm, as explained in Section 5.2. Here the frequency score of each operation obtained in the algorithms is ignored and replaced by binary "1" since all the operations are critical in terms of execution frequency (as they appeared on the top of the list). Likewise, if an operation does not exist in a particular algorithm, it is marked as "0". Thus, the data table obtained for this case study is shown in Table 5.2.

5.3.2 Vector Representation

In this step all the algorithms were represented as vectors. The characteristic operations in Table 5.2 were considered as independent variables to define each algorithm as a multi-variable linear equation. They are then represented as vectors in a multi-dimensional space. Using this concept, all the algorithms were expressed in terms of their characteristic operations by vectors as follows, where the lower case letters refer to the operations in Table 5.2. Thus, they can be marked in a multi-dimensional space as points as per their coordinates.

$$
\begin{aligned}
MP &= a + b + f + w \\
&= (1, 1, 0, 0, 0, 1, 0, \ldots, 0, 1, 0, 0) \\
BM &= e + f + g + h + k + l + v + x + y + z \\
&= (0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, \ldots, 0, 1, 0, 1, 1, 1) \\
RT &= f + g + k + m + p + q + x + y \\
&= (0, \ldots, 1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 1, 1, 0, \ldots, 0, 1, 1, 0) \\
&\ldots
\end{aligned}
$$

5.3.3 Distance Calculation

As described in Section 5.2.4, the Euclidean distance between each pair of points was calculated. Hence, the differences between the corresponding algorithms were determined. Table 5.3 shows the Euclidean distance matrix obtained for our objective algorithms set.

5.3.4 Centre and Member Points Selection

The next task is selecting centre points in order to form clusters. As described in Section 5.2.5, first a DC value was calculated as the maximum allowed distance between centre points to

Table 5.3: Euclidean distance matrix of the case study

Algorithm	MP	KMP	AC	BM	TBM	ZT	HP	RT	CL	GG	NSN	AG
MP	0.00	1.41	2.00	4.12	4.00	3.87	3.16	3.16	3.16	3.32	2.24	4.00
KMP	1.41	0.00	1.41	4.36	4.24	4.12	3.46	3.46	3.16	3.32	2.24	4.24
AC	2.00	1.41	0.00	4.12	4.00	3.87	3.74	3.74	3.46	3.32	2.65	4.00
BM	4.12	4.36	4.12	0.00	1.00	1.41	3.32	3.32	4.12	4.00	4.24	1.00
TBM	4.00	4.24	4.00	1.00	0.00	1.73	3.46	3.46	4.00	3.87	4.36	0.00
ZT	3.87	4.12	3.87	1.41	1.73	0.00	3.61	3.61	3.87	3.74	4.00	1.73
HP	3.16	3.46	3.74	3.32	3.46	3.61	0.00	0.00	3.16	3.32	3.00	3.46
RT	3.16	3.46	3.74	3.32	3.46	3.61	0.00	0.00	3.16	3.32	3.00	3.46
CL	3.16	3.16	3.46	4.12	4.00	3.87	3.16	3.16	0.00	1.00	3.32	4.00
GG	3.32	3.32	3.32	4.00	3.87	3.74	3.32	3.32	1.00	0.00	3.46	3.87
NSN	2.24	2.24	2.65	4.24	4.36	4.00	3.00	3.00	3.32	3.46	0.00	4.36
AG	4.00	4.24	4.00	1.00	0.00	1.73	3.46	3.46	4.00	3.87	4.36	0.00

their member points. In this case study, we have specified the expected similarity between algorithms to be 90% and the total number of characteristic operations is 26. Thus the DC value was calculated as follows as per Definitions 2 and 3. The number of operations O_s to be similar is $O_s = \frac{90 \times 26}{100} \simeq 23$. The distance constant DC is thus $DC = \sqrt{26 - 23} \simeq 1.73$. Then, using this DC value, member point selection was performed over the data set of Table 5.3 as described in Section 5.2.5. For the first centre point in the list, which is the Morris-Pratt algorithm, the other points which have a Euclidean distance less than 1.73 from the Morris-Pratt's vector were considered. In this case, only the Knuth-Morris-Pratt had a lower value, which was 1.41. Thus the Knuth-Morris-Pratt algorithm become the sole member point to the Morris-Pratt algorithm. Then the next centre point of the list, which is the Knuth-Morris-Pratt, was considered and its member points were found. Likewise for all the points in the centre point list, member points were identified as shown in Table 5.4.

5.3.5 Clustering

Clustering was then performed, considering the above centre points and their member points lists. As the Boyer-Moore and the Apostolico-Giancarlo algorithms' points have the highest number of member points, three in this case, clustering was started from these two groups.

Table 5.4: Initial algorithm groups

Centre	member Points	Centre	member Points
MP	KMP	HP	RT
KMP	MP, AC	RT	HP
AC	KMP	CL	GG
BM	TBM, ZT, AG	GG	CL
TBM	BM, AG	NSN	-
ZT	BM, AG	AG	BM, TBM, ZT

Since this is tied, a priority list was calculated as shown below. The total Euclidean distances of groups centred on the Boyer-Moore and Apostolico-Giancarlo is denoted by BM_{total} and AG_{total}, respectively.

$$BM_{total} = 1.41 + 1.41 + 1.00 = 3.82$$
$$AG_{total} = 1.00 + 1.00 + 1.73 = 3.73$$

Thus the Apostolico-Giancarlo algorithm was selected as the starting point since it has a lower total distance value and hence it shows better cohesiveness and higher priority than the Boyer-Moore algorithm. The first cluster of this case study was thus the Apostolico-Giancarlo algorithm and its member points. Clustering was carried out until all twelve algorithms were counted. Figure 5.4 shows the four major algorithm clusters ultimately obtained.

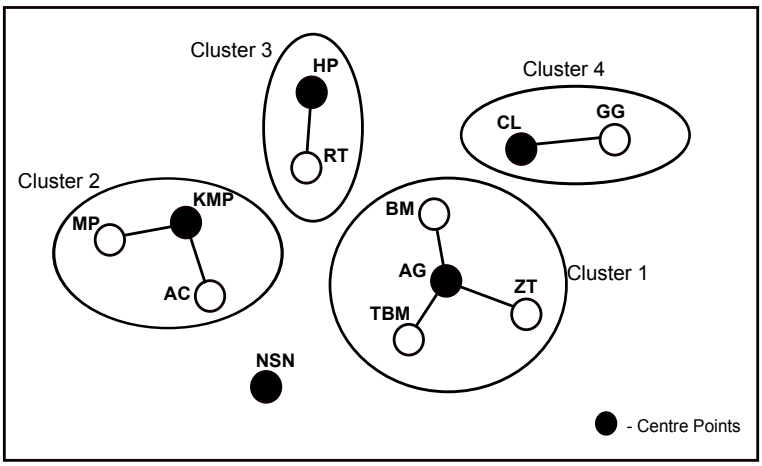


Figure 5.4: Resultant algorithm clusters

The clusters obtained in the results show a similar pattern of groupings, as was expected from historical evaluation of the algorithms. The closely related Boyer-Moore, Apostolico-Giancarlo, Turbo-Boyer-Moore and Zhu-Takaoka algorithms have all fallen into one cluster. In addition, as pointed out by the historical evidence, the Morris-Pratt family algorithms are also successfully bound together. Furthermore, the similarity identified between the Colussi and Galil-Giancarlo algorithms also proved discoverable by forming a different cluster for them. However, in the experimental results, the Horspool and Raita algorithms form another cluster, indicating they share similar operations. In fact, they both have obtained the same vectors (marking no distance between them) and hence coincided on top of each other in the multi-dimensional space. That is because, according to the table of characteristic operations (Table 5.2), they share the existence and non-existence of operations in a similar manner. Therefore, clearly they are similar in functionality. The sole outlier is the Not So Naive algorithm. It uses a unique approach for string matching, such that preprocessing can be done in constant time and space [Charras and Lecroq, 2004]. This algorithm remained isolated since it does not share a similar pattern of characteristic operations to any other algorithm, as expected by the historical background. Thus, these results confirm the accuracy of our clustering approach according to the historical background.

5.4 Dynamic Analysis Based Evaluation of Clusters

This section presents experiments carried out in a dynamic environment to evaluate the resultant clusters. Since our clustering approach was performed based on static analyses, the accuracy of the results needed to be evaluated with respect to the actual runtime performance. In this section two dynamic case studies are presented.

5.4.1 Empirical analysis results

Performing dynamic analysis on algorithms helps understanding of their runtime behaviour for a given data set. The execution performance of an algorithm basically depends on its characteristic operations, where most of its execution time is spent. In our experiment we counted the execution frequency of each characteristic operation in an algorithm. This was done to all the candidate algorithms in order to record their characteristic operation usage frequencies. These variations were then plotted in a graph to identify similar patterns between the algorithms. The algorithms that follow similar patterns of characteristic operation usage can be compared

with the static clusters in Figure 5.4.

Evaluation criteria for empirical analysis results

In order to confirm the accuracy of our resultant clusters, historical relations between the algorithms are considered. If our experimental results show similar execution variations between the algorithms in the same cluster, the successfulness of our clustering approach can be confirmed.

Evaluation experiment for empirical analysis results

In the experimental set-up, the execution frequency of each primitive operation of each algorithm was counted for a large range of problem sizes. In order to maintain a higher accuracy in the resultant dynamic measurements, 30 trials were executed for randomly generated *patterns* and the average value for each result was recoded. The length of the *pattern* sequence ranged from 5 to 100, while the length of *text* ranged from $10,000$ to $20,0000$ characters. The problem size was defined as *length of text* $-$ *length of text* $+$ 1 which is equal to the maximum number of positions where the *pattern* can appear in the text.

Figure 5.5 illustrates the linear behaviour of the execution frequency variations for 26 operations of all algorithms. This is fundamental to the nature of all searching algorithms. Nevertheless, according to the graph, groups of algorithms can be clearly distinguished from each other by their rate of growth.

Figure 5.6 shows the cluster of Apostolico-Giancarlo, Boyer-Moore, Turbo Boyer-Moore and Zhu-Takaoka algorithms, which is found at the bottom of Figure 5.5. This cluster confirms the accuracy of our static clustering method since Turbo Boyer-Moore, Apostolico-Giancarlo and Zhu-Takaoka algorithms are different variations of the historical Boyer-Moore algorithm [Charras and Lecroq, 2004]. Variations of algorithm Apostolico-Crochemore and Knuth-Morris-Pratt lie near to each other while the Morris-Pratt algorithm that belongs to the same cluster dominates slightly higher execution frequencies (Figure 5.7). Since the Morris-Pratt algorithm is the native algorithm of this cluster, the Knuth-Morris-Pratt and Apostolico-Crochemore algorithms are much more optimised in terms of execution overhead [Charras and Lecroq, 2004]. The Colussi and Galil-Giancarlo algorithms of static cluster 3 in Figure 5.4, which are shown in Figure 5.8, appear at the top of the graph in Figure 5.5 near to each other. In theory, they are closely related in terms of their functionalities since Galil-Giancarlo is a refinement of the Colussi algorithm [Charras and Lecroq, 2004]. Figure 5.9 illustrates the variations of

the Horspool and Raita algorithm cluster and the Not So Naive algorithm in the same graph. According to the graph, algorithms Horspool and Raita are close to each other, since they both use a memory comparison operation at their searching phase [Charras and Lecroq, 2004]. As illustrated in Figure 5.5 the Not So Naive algorithm shows a distinct execution frequency pattern. This distinguishes it from all the other algorithms due to its unique preprocessing phase [Charras and Lecroq, 2004]. Thus, according to these figures, algorithms in the same statically-identified algorithm clusters show similar dynamic behaviour for the given data set and hence confirm the accuracy of our static clustering method.

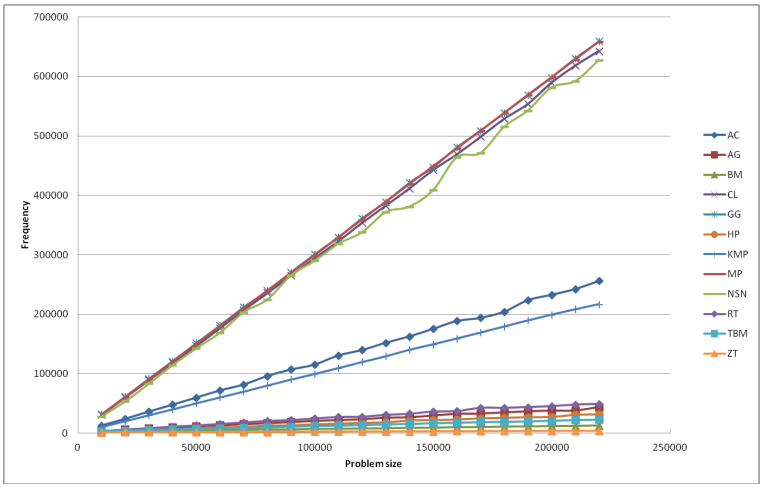


Figure 5.5: Execution frequencies of the primitive operations of algorithms (*pattern size = 100)*

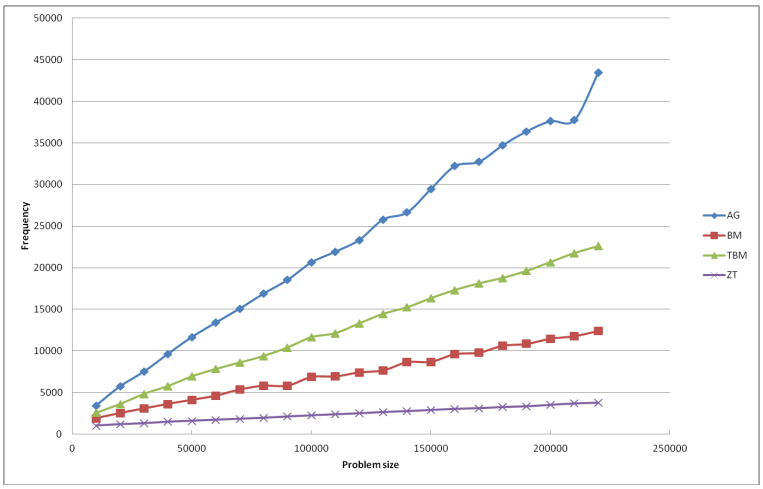


Figure 5.6: Cluster of AG, BM, TMB and ZT algorithms

5.4.2 Special purpose instruction results

This section presents a further validation of the static clusters in terms of a real world application. The application is namely design of a special-purpose Instruction Set Architecture (ISA)

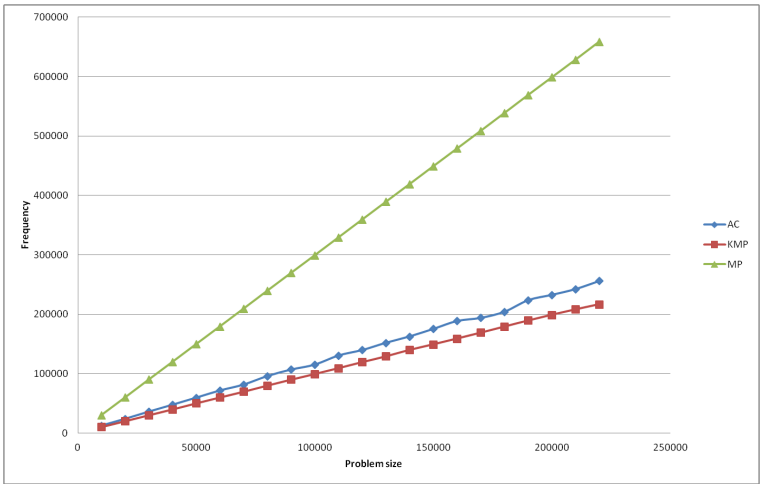


Figure 5.7: Cluster of MP, KMP and AC algorithms

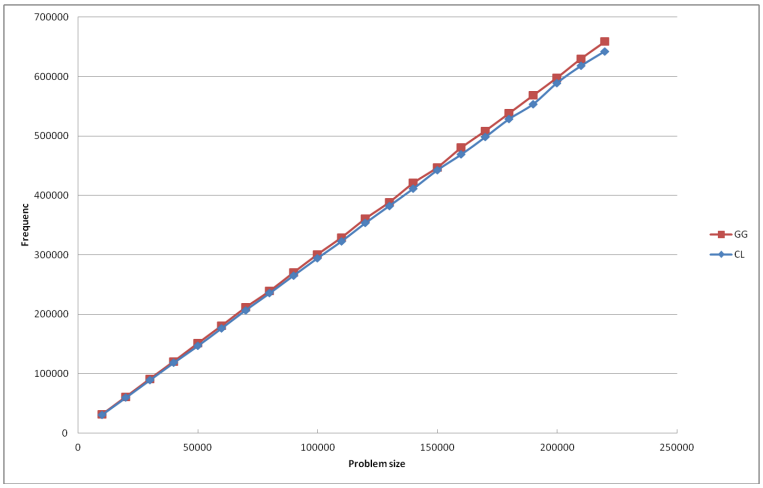


Figure 5.8: Cluster of CL and GG algorithms

for the considered algorithms. In this field, special-purpose operations common to a given set of algorithms are selected. Then they are implemented as special-functional units (known as custom instructions) in a specific ISA where the objective algorithms are executed. Thus the execution efficiency of the algorithms is increased significantly. This kind of special-purpose embedded system is widely used in real world devices. Similarly, in this evaluation experiment, we design custom instructions for a cluster formed by our case study. Then, the performance differences are measured between the other considered string matching algorithms.

Evaluation criteria for special purpose instruction results

The dedication of the custom instructions of the new ISA is considered for this evaluation criteria. If a specific platform is designed targeting one algorithm in a particular cluster, the resulting instruction set should be able to show performance improvements for other algorithms

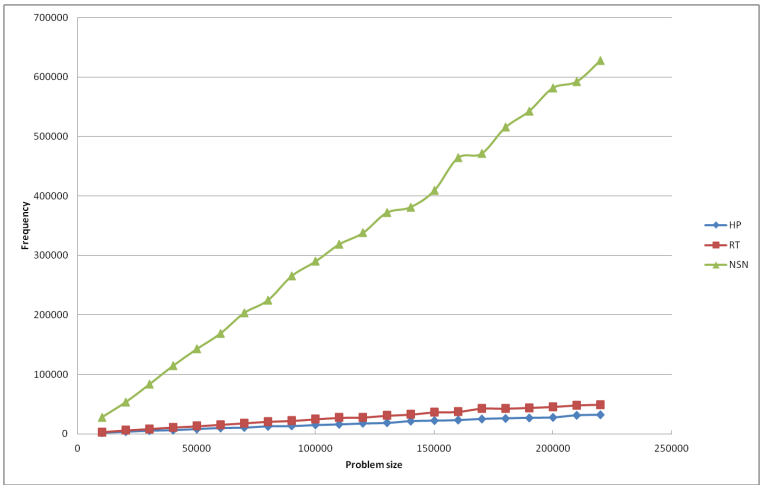


Figure 5.9: Cluster of HP and RT algorithms with the NSN algorithm

in the same cluster. On the other hand, algorithms from other clusters should not be significantly advantaged since they do not share the same operations. Thus, the static clusters in Figure 5.4 are further evaluated by designing custom instructions based on the above claim.

Evaluation experiment for special purpose instruction results

For the evaluation, five algorithms were selected to represent each cluster. Here, the *centre point algorithm* of each cluster is used as their *representative algorithm*. According to the clustering method, this algorithm is close to all the other algorithms in the cluster in terms of common operations. As shown in Table 5.5, the Apostolico-Giancarlo, Knuth-Morris-Pratt, Horspool, Colussi and Not So Naive algorithms were considered for the evaluation process along with their primitive special-purpose operations.

The next step is to design custom instructions of a selected centre point algorithm. In our evaluation, the Apostolico-Giancarlo algorithm was selected as it is in the largest cluster generated by the case study and hence covers many algorithms. To perform the evaluation, Tensilica's Xtensa customizable processor and its GNU-based software tools (compiler, Linker and profiler) [Tensilica, 2014] were used. As shown in Table 5.5, the Apostolico-Giancarlo algorithm has many candidate operations to implement as custom instructions. Implementing all of them reduces the efficiency of a potential special-purpose instruction set, as it obviously increases the hardware cost. For the evaluation's purpose, we selected four promising operations to be implemented as custom instructions considering the hardware constraints of the Xtensa architecture. In addition, these operations were selected to represent different functionalities such as a conditional statement, a loop, a comparison and a simple calculation as shown in

Table 5.5: Special purpose operations of each cluster

AG	**KMP**	**HP**	**CL**	**NSN**
$m = x[i]$	$x[i] \neq x[j]$	$x[i] = m$	$m = x[i]$	$x[i] \neq x[j]$
$x[i] == y[i+j]$	$m = x[i]$	$x[y[i]] == m - i - 1$	$x[i] == x[j]$	$x[i] == x[j]$
$x[i] = m$	$x[i] == x[j]$	$x[i] == m$	$x[i] = m$	**memcmp** : *compare memory location*
$x[y[i]] == m - i - 1$	$x[i] = y[j]$	$i = x[m] + 1$	$x[i] == m$	
$x[i + m - 1 - n] < i - p$	$x[i] = m$	$m = x[i + m - 1]$	$i = x[m] + 1$	
$x[i] = x[i + m - 1 - n]$	**while** $(i > -1$ & $x[j] \neq x[i])$ $i = a[i]$;	**memcmp** : *compare memory location*	$x[i - j] + m < i$	
$x[i] == x[i + m - 1 - n]$		**for** $(i = 0; i < m; ++i)$ $x[i] = n$;	$x[i] = x[i - j] + m$	
$x[i] == m$		**for** $(i = 0; i < m - 1; ++i)$ $x[y[i]] = m - i - 1$;	$m < i + x[j]$	
$i < m - 1 - j$			$x[h[i]] == y[j + h[i]]$	
$i = x[m] + 1$			**for** $(i = 0; i < m; ++i)$ $x[i] = n$;	
$x[i] = m - 1 - i$				
$x[m - 1 - y[i]] = m - 1 - i$				
max $= ((m) > (n))\ ?\ (p) : (q)$				
for $(i = 0; i < m; ++i)$, $x[i] = n$;				
for $(i = 0; i < m - 1; ++i)$ $x[y[i]] = m - i - 1$;				
for $(i = 0; i \leq m - 2; ++i)$ $b[m - 1 - x[i]] = m - 1 - i$;				

Table 5.6.

These custom instructions were inserted to the base ISA as TIE instructions using Tensilica's TIE language. The new instruction specifications complied to an RTL (register-transfer level) design and were attached to the processor core by the Xtensa software tool chain [Tensilica, 2014]. Then all the algorithms were executed and profiled on the new custom instructions. The performances were compared against the original processor performance. The only difference between the two processor versions were newly added custom instructions. Figure 5.10 shows the Xtensa Xplorer software we used to perform this experiment. Table 5.7 summarises the results of this study. It compares the number of cycles executed, the number of instructions

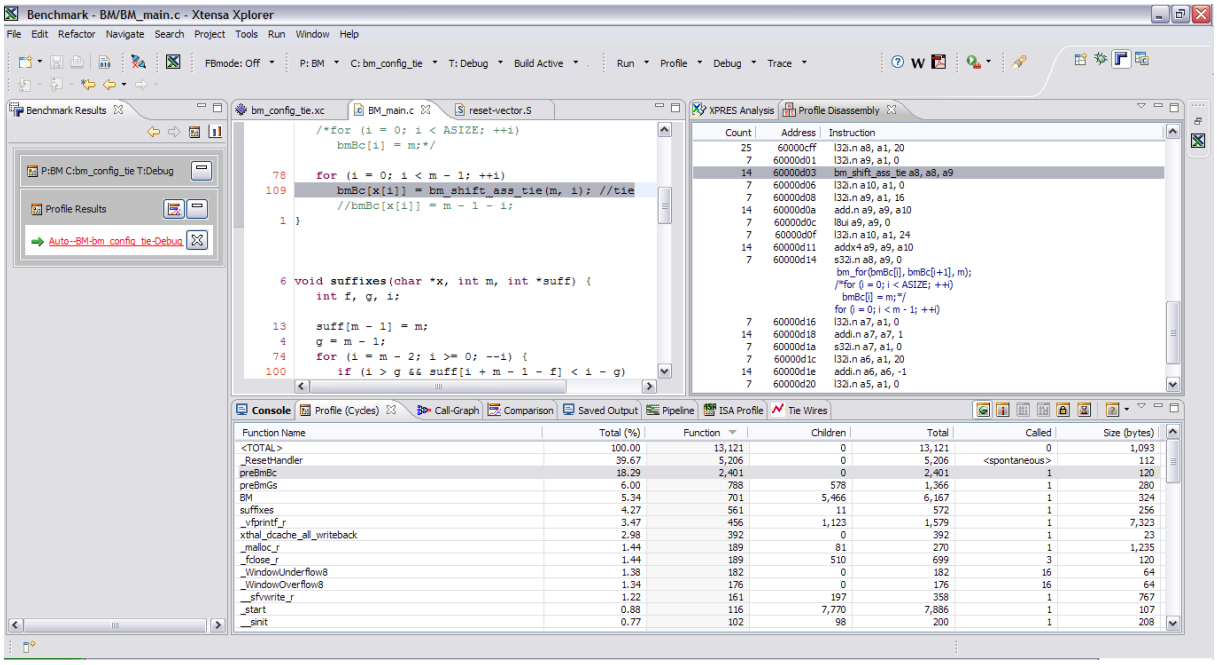


Figure 5.10: Xtensa Xplorer

executed and the total power energy consumed in both the original and the new versions of Xtensa ISA (the area cost allowance for custom instructions was kept to 5% of the whole hardware). Performance variations of both ISAs in Table 5.7 are plotted in Figures 5.11 to 5.14.

According to the results, the algorithms that belong to the Apostolico-Giancarlo cluster (Boyer-Moore, Zhu-Takaoka, Turbo Boyer-Moore) show a significant cycle count and instruction count reduction including a maximum of 30% from the Zhu-Takaoka algorithm while maintaining a considerable energy saving. More than 70% of cycle and instruction counts for the Zhu-Takaoka algorithms are spent on two level nested "bad character shift" array initialisation. Therefore, the new ISA achieves the maximum performance gain from this "for loop" instruction. On the other hand, the Horspool algorithm also has achieved significant improvements of performance since it shares the same "for loop" operation in its matching

Table 5.6: Selected custom instructions for the evaluation

Operation	**Description**
$m - i - 1$	a commonly used calculation of an array index
max = $((m) > (n))\ ?\ (p) : (q)$	compare and return the maximum
for $(i = 0;\ i < m;\ {+}{+}i),\ x[i] = n;$	for loop to initialise an array

phase. However all the other evaluated algorithms are not capable of showing significant performance improvement and almost of their performances remain the same. Thus, based on the above results, it is again proved that our static clustering method accurately classifies algorithms into similar clusters, based on the special purpose processor instructions that they may use.

Table 5.7: Evaluation results comparison for the string matching algorithms set

Algorithm	**Original**			**New**			**Performance Gain (%)**	
	Cycles	Instructions	Energy (μJ)	Cycles	Instructions	Energy (μJ)	Cycles	Instructions
AG	16091	8981	1.29	14411	8116	1.15	10.4	9.6
BM	14813	8124	1.18	1321	7259	1.04	11.4	10.6
ZT	979233	651750	1.21	619000	459321	1.08	34.0	29.5
TBM	15188	8381	86.7	13510	7527	54.6	11.0	10.2
KMP	10678	5376	0.82	*same*	*same*	*same*	0.0	0.0
HP	13211	7011	0.96	11602	6203	0.90	12.1	11.5
CL	12631	6845	1.00	*same*	*same*	*same*	0.0	0.0
AC	10255	5146	0.8	*same*	*same*	*same*	0.0	0.0
GG	12290	6604	9.74	12270	6592	9.74	0.2	0.2
NSN	9263	4457	0.71	*same*	*same*	*same*	0.0	0.0

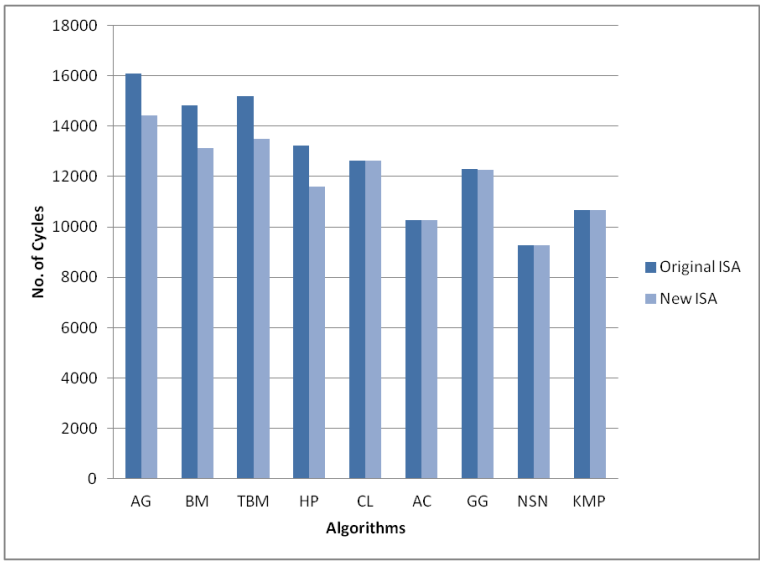


Figure 5.11: Cycle count comparison of the original vs new ISA

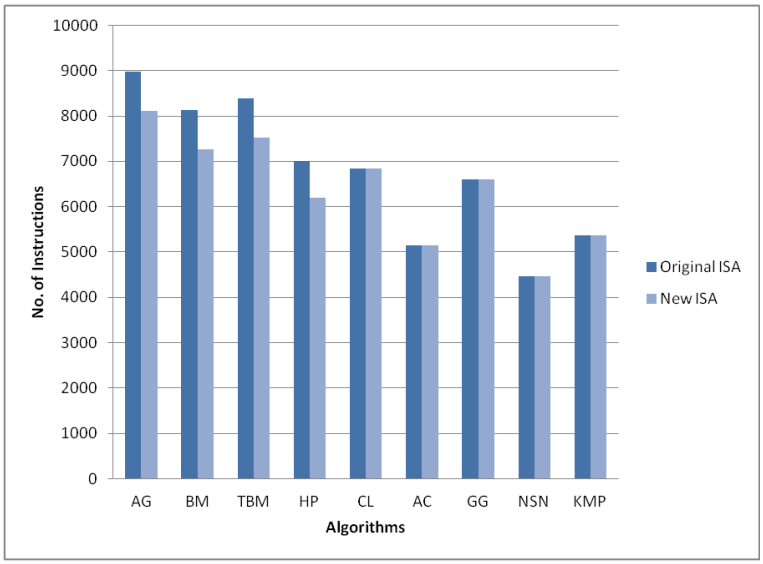


Figure 5.12: Instruction count comparison of the original vs new ISA

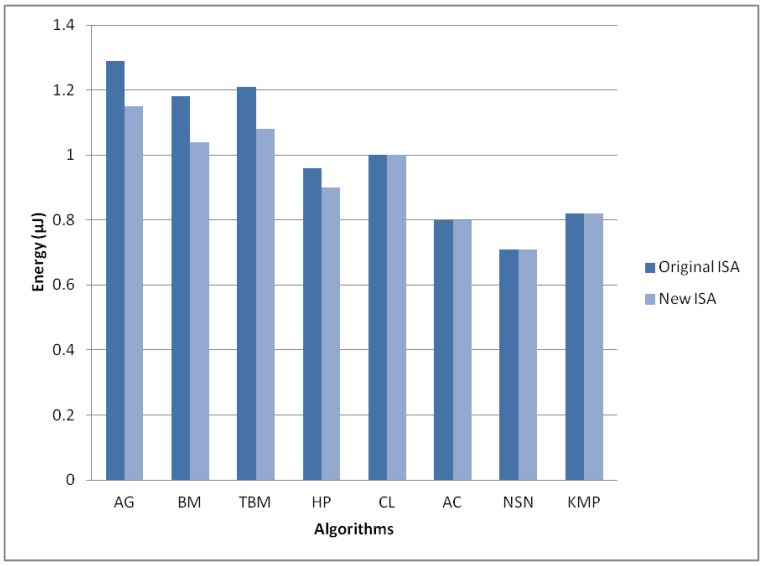


Figure 5.13: Energy consumption comparison of the original vs new ISA

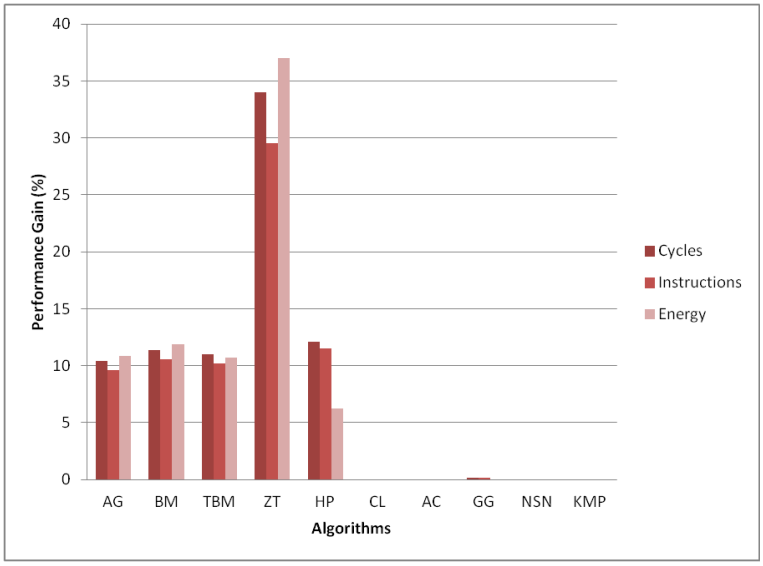


Figure 5.14: Performance gain comparison of the original vs new ISA

5.5 Evaluation of the Clustering Approach in Multiple Sets of Algorithms

The capability of applying our approach in different applications has not been yet examined in practice. Therefore, here we present another case study we carried out to evaluate the versatility of the clustering.

Evaluation criteria for multiple sets of algorithms

In this experiment, we considered three types of simple algorithms. They represent three different application domains: sorting, sequence alignment and string matching. According to

Table 5.8: Algorithms used in the case study in Section 5.5.

	Algorithm	**Abbreviation**
Sorting Algorithms	Bubble Sort	BS
	Quick Sort	QS
	Shell Sort	SHS
	Selection Sort	SS
Sequence Alignment Algorithms	CV Tree	CV
	Needleman-Wunsch	NW
	Smith-Waterman	SW
Exact String Matching Algorithms	Apostolico-Giancarlo	AG
	Boyer-Moore	BM
	Turbo Boyer-Moore	TBM
	Zhu-Takaoka	ZT

the nature of the functionalities performed in these algorithms, they utilise different operations. Therefore, if they are classified for different groups based on the operations they perform, three separate clusters are expected based on our background knowledge about them. Therefore, in this experiment, we have used this claim to determine the accuracy of our clustering method over multiple application domains.

Evaluation experiment for multiple sets of algorithms

The clustering was performed for the selected eleven algorithms shown in Table 5.8. First, each algorithm was analysed to identify its characteristic operations. These operations are listed in Table 5.9 against the algorithms with their existence details. Based on this data table, the algorithms are represented as vectors in a multidimensional space by specifying their coordinates from the selected 28 operations. Then their Euclidean distance matrix is obtained as shown in Table 5.10. As described in Section 5.2, clustering is performed to form clusters based on the common operations. In this case study, the DC value is calculated as 2.37 for 80% expected similarity between a centre point and its member points. The resultant clusters obtained in this case are shown in Figure 5.15.

Table 5.9: Characteristic operations found in the groups of selected algorithms

characteristic Operations	AG	BM	BS	CV	NW	QS	SHS	SS	SW	TBM	ZT
$x[i] = m$	1	1	1	1	0	1	1	1	1	1	1
$x[i][j] = m$	0	0	0	1	1	0	0	0	1	0	0
$m = x[i][j]$	0	0	0	0	1	0	0	0	1	0	0
$x[i] == x[i + m - 1 - n]$	1	1	0	0	0	0	0	0	0	1	1
$m = x[i - 1][j - 1] + c$	0	0	0	0	1	0	0	0	1	0	0
$x[i] = x[i + m - 1 - n]$	1	1	0	0	0	0	0	0	0	1	1
m[i] = $y[j]$	0	0	0	1	0	0	0	0	0	0	0
$m = (a \times c) + m$	0	0	0	1	0	0	0	0	0	0	0
$x[i] = m - 1 - i$	1	1	0	0	0	0	0	0	0	1	1
$m = x[i][j - 1] - c$	0	0	0	0	1	0	0	0	1	0	0
$x[m - 1 - y[i]] = m - 1 - i$	1	1	0	0	0	0	0	0	0	1	1
$x[i] \neq m$	0	0	0	1	0	0	0	0	0	0	0
$x[i] == m$	1	1	0	0	0	0	0	0	0	1	1
$x[i] = x[j]$	0	0	1	0	0	1	1	1	0	0	0
$x[i + m - 1 - n] < i - p$	1	1	0	0	0	0	0	0	0	1	1
$x[i] < m$	0	0	0	0	0	1	1	0	1	0	0
$x[i] > y[j]$	0	0	0	0	0	0	0	1	0	0	0
$x[y[i]] == m - i - 1$	1	1	0	0	0	0	0	0	0	1	1
$m = x[i]$	0	0	1	1	1	1	1	1	1	0	0
$i = x[m] + 1$	1	1	0	0	0	0	0	0	0	1	1
$x[i] = y[j]/m$	0	0	0	1	0	0	0	0	0	0	0
$m = x[i - 1] + m$	0	0	0	0	1	0	0	0	0	0	0
max = $((m) > (n))\ ?\ (p) : (q)$	1	1	0	0	0	0	0	0	0	1	1
$m = (a \times b) + c$	0	0	0	1	0	0	0	0	0	0	0
$i < m - 1 - j$	1	1	0	0	0	0	0	0	0	1	1
$x[i][j] > m$	0	0	0	0	0	0	0	0	1	0	0
$x[i] == y[i + j]$	1	1	0	0	0	0	0	0	0	1	1
$m = x[i - 1][j] - c$	0	0	0	0	1	0	0	0	1	0	0

Table 5.10: Euclidean distance matrix of the case study in Section 5.5

Algorithm	AG	BM	BS	CV	NW	QS	SHS	SS	SW	TBM	ZT
AG	0.00	1.00	3.61	4.12	4.24	3.74	3.74	3.74	4.24	0.00	1.00
BM	1.00	0.00	3.74	4.24	4.12	3.87	3.87	3.87	4.36	1.00	0.00
BS	3.61	3.74	0.00	2.83	3.00	1.00	1.00	1.00	3.00	3.61	3.74
CV	4.12	4.24	2.83	0.00	3.32	3.00	3.00	3.00	3.32	4.12	4.24
NW	4.24	4.12	3.00	3.32	0.00	3.16	3.16	3.16	2.00	4.24	4.12
QS	3.74	3.87	1.00	3.00	3.16	0.00	0.00	1.41	2.83	3.74	3.87
SHS	3.74	3.87	1.00	3.00	3.16	0.00	0.00	1.41	2.83	3.74	3.87
SS	3.74	3.87	1.00	3.00	3.16	1.41	1.41	0.00	3.16	3.74	3.87
SW	4.24	4.36	3.00	3.32	2.00	2.83	2.83	3.16	0.00	4.24	4.36
TBM	0.00	1.00	3.61	4.12	4.24	3.74	3.74	3.74	4.24	0.00	1.00
ZT	1.00	0.00	3.74	4.24	4.12	3.87	3.87	3.87	4.36	1.00	0.00

According to the results, sorting and string matching algorithms form two separate clusters as expected. However, from the sequence analysis algorithm domain, the CV Tree algorithm produced outliers from the other algorithms. The reason for this behaviour is that the functionalities and computations performed in this algorithm are fundamentally different to the Needleman-Wunsch and Smith-Waterman algorithms. Thus, this case study demonstrates that our clustering approach can be accurately applied to different application domains and successfully distinct algorithm clusters in terms of their operations.

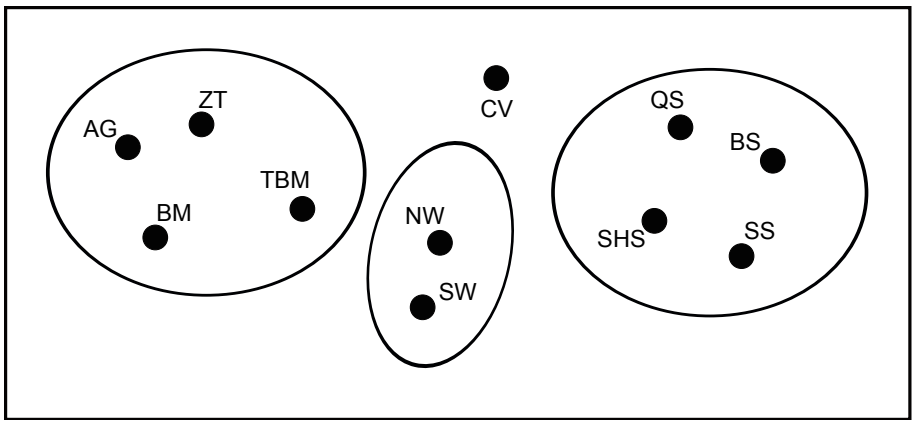


Figure 5.15: Resultant algorithm clusters produced from different application domains

5.6 Discussion

A formal comparison of our static clustering method versus an existing static data clustering method is discussed here. For the comparison, we have used the well-known K-means data classification algorithm [Lloyd, 1982] and the clustering is performed on the same set of string matching algorithms that were used in our case study (Section 5.3).

In the experiment, we implemented the K-means data classification algorithm in C/C++. The candidate algorithms were expressed as multi-variable equations in terms of their primitive operations according to Table 5.2 and hence represented as multi-dimensional vectors. These vectors were the input of the K-means classification algorithm and clustering results were obtained from 30 to an unlimited number of K-means iterations while the number of restarts was set to 12 (equal to the number of candidate algorithms).

The K-means classification produced the same static clusters obtained by our clustering method (Figure 5.4). Here, the number of expected clusters, known as 'K', was set to 5. The main drawback of the K-means classification method is a requirement for pre-specifying the number of clusters, which may result in unrelated algorithms being in the same cluster due to forced clustering. In addition, in cases where we need to identify how a given set of algorithms are related to each other in terms of their shared primitive operations (which is our focus), it is difficult to predict the number of expected clusters beforehand. Nevertheless, from these comparison, it is proven that our static clustering method is as capable of clustering related algorithms together accurately as when the conventional K-means classification is calibrated appropriately.

Chapter 6

Heuristic Weighting System for Adaptation of Operations Targeting Different Applications

---

The main target of our research work is identifying relations between multiple algorithms in terms of their characteristic operations. Improving the performance of common functionalities identified among multiple algorithms can be thus achieved on all the algorithms at the same time. This technique is useful to increase the performance of many different applications using suitable strategies such as implementing hardware functional units, parallelizing the execution, implementing library functions, improving the software implementation and reducing size and complexity. For instance, in the Application Specific Processor Design (ADSP) area, common and frequently used operations are implemented in hardware to execute them quickly. Thus, the execution speed of multiple algorithms, which use those operations, are automatically increased.

In order to obtain the expected performance improvements, selecting a set of suitable operations is essential. For instance, in the above example, high "frequency" and high "commonality" (among algorithms) are important properties of the operations to consider. These properties guide the resultant specific set of functional units especially to benefit the target application domain. In this case, other properties like the hardware characteristics of the operations also needed to be examined to see the compatibility towards the target architectures. However, a different application may prioritise different properties of operations. For instance, in the field of compiler design, potential parallelism of operations is identified so that they can be executed concurrently at runtime. In this scenario, "parallelizability" is the key property to the selection of suitable operations. As well, for instance, if we are to generate a library function

which contains common functions used by multiple algorithms, "size and complexity of the code" and "commonality" will be the key properties to decide the operations to implement. In all these cases, different properties of operations are desired by different target applications. Thus, we can learn that the selection of suitable operations is highly subjective to the *target applications* (in the above examples, design of hardware functional units, compiler design and design of library functions). Therefore, a system to calibrate the priorities of different properties of operations according to a given target application is necessary.

Existing algorithm analysis approaches do not facilitate calibrating operations for different target applications. In fact, these analyses are designed to benefit only a specific application area. For instance, the application areas discussed above each have their own algorithm analysis methodologies dedicated for their purposes [Bojoi et al., 2012; Jain et al., 2001; Rashid et al., 2008; Sedgewick and Flajolet, 2013; Sistanizadeh, 1990; Srinivasan et al., 2005; Thomas and Olukotun, 2005]. In contrast, our characteristic operation identification method is a generalised approach. Hence it is not dedicated to a specific type of algorithm or specific target application. The primary purpose of our approach is that it can be used by different types of algorithms and applications successfully. Therefore, our approach must be capable of diverting the selection of the final set of characteristic operations based on the currently considered target application.

As a solution, we present a heuristic weighting system to rank different properties of operations targeting different applications. It provides insights for a customisable operations selection process under different conditions. It considers specifications of given target applications and determines the required key properties of operations to adjust the selection towards those applications.

In this chapter, the accuracy of our framework, along with the weighting system, is demonstrated by using three evaluation experiments. These experiments were performed targeting three applications: generating library functions (Section 6.2), generating custom instructions (Section 6.3) and finding "hot spots" (Section 6.4) for a given set of algorithms. The selection of operations for these three applications is conducted by considering six parameters to represent the operations' execution frequency, commonality among the algorithms, size, implementation and hardware properties. For all three case studies, the validity of the resultant operations was demonstrated using dynamic evaluations. Thus, this chapter presents phase three of our framework to solve the third obstacle we identified in Section 1.3.

6.1 Concepts of the Weighting System

Weighting systems are used widely in many application areas. Signal processing, text processing, bioinformatics, custom processor design, network communication and ranking systems are few of them. Weighting systems are typically used to prioritise different properties under given conditions. Here, *weighting factors* are assigned to each property based on their contribution towards the expected outcome. Therefore, depending on the expected results, the weightings can be calibrated appropriately by assigning different numerical values to different properties.

However, as mentioned above, these existing weighting systems are built having only one application domain in mind. For our purposes, a weighting system is developed to accurately prioritise the most suitable set of operations for different target applications, when their desirable priorities are defined. Our system examines a variety of properties of operations, which are useful in providing the required specification for a particular application. To facilitate this, different analyses are carried out to analyse the operations in terms of the desired properties. As for the specific characteristics of target applications, weighting factors of properties differ from one application to other. Therefore, the weighting system is a heuristic approach which needed to be calibrated accurately to obtain the best results. Hence a poor understanding about the specifications might result weaker outcomes. Our weighting system consists of the following steps.

1. Determining properties of operations

   Here, based on the target applications, the required properties are determined (e.g., execution frequency, commonality and hardware cost in custom instruction generation).

2. Analysing algorithms to measure occurrences of operations

   When the required properties are determined, the objective algorithms are analysed to find the criticalness of their operations in terms of these properties.

3. Assigning appropriate weights for the occurrences and calculating priorities

   Once the significance of the potential characteristic operations are identified, they are ranked according to the requirements of the target application. Then, suitable weights are assigned for each property considering the ranks. These weighting values will determine the contribution of the property to the final operations list. Based on these weightings

and the individual scores of each operation, *priority factors* (described in detail in Section 6.1.3) of all operations are calculated. These priority values are then considered, to rank the candidate operations list. The top operations with highest score ranks are finally selected as the most critical operations for the target application from the objective algorithms.

6.1.1 Determining requirements and specifications of target applications

The weighting system basically depends on the different properties of operations we considered in the operation selection process. According to the target applications, suitable properties of operations are selected. Here, we discuss determining the properties of operations based on the requirements and specifications of our three target applications.

One of the three applications discussed here is *custom instruction design targeting special-purpose processors*. In this field, frequently executed operations are implemented in hardware so that execution speed can be increased. As mentioned earlier, ADSPs are designed targeting multiple algorithms. Therefore, when repeatedly executed common operations are found, they are implemented as custom instructions in a special-purpose ISA. Hence, the execution performance of all the target algorithms can be improved. Dot-product, MAC (Multiply-ACcumulator) operation and Filters are a few such well known operations. In this application domain the key properties of operations are execution frequency and hardware cost [Galuzzi and Bertels, 2011]. In our analysis strategies the execution frequency of an operation is denoted by its static frequency and nesting level. According to a survey conducted by Galuzzi et al, typically in this field hardware cost is determined mainly by three parameters: the number of input and outputs, surface area, and power and energy consumption [Galuzzi and Bertels, 2011]. In our case, since the operations are represented as sequences of assembly instructions, the number of inputs and outputs can be determined by counting the unique operands used within the sequence. However, in a static method based on source code analysis, measuring other hardware costs such as power and area is difficult, since accurate dynamic/simulation results are needed for such comparisons. Therefore, in this demonstration the hardware cost measurements are limited to the number of modes. In addition, if the custom instructions are generated targeting an ADSP [1] rather than an ASIP [2], commonality is another important

[1] Application Domain Specific Processor considers multiple algorithms.
[2] Application Specific Instruction-set Processor considers only a single algorithms

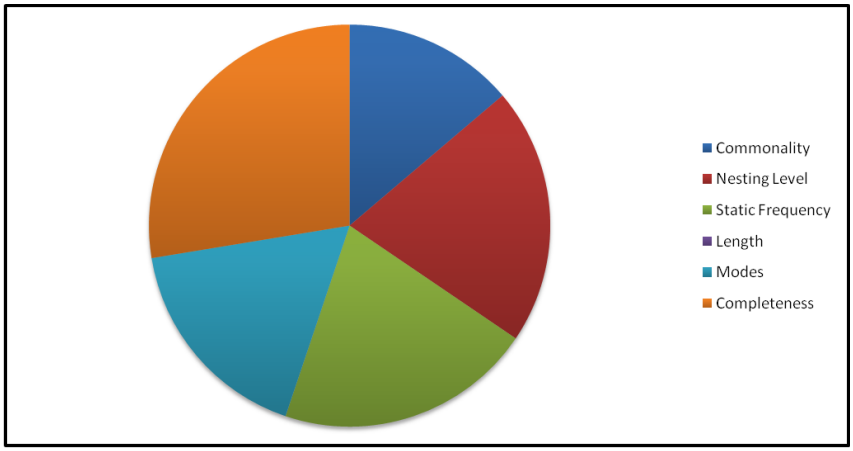
property to consider in the selection of an appropriate set of operations. Table 6.1 ranks these parameters according to their importance.

Table 6.1: Rankings of operations specifications of the considered target applications.

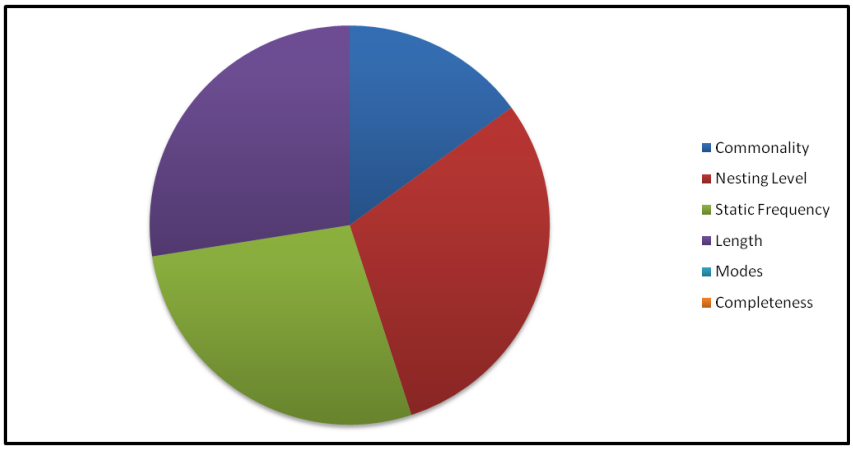
Library Generation	**Custom Instruction Generation**	**Execution Performance Improvement**
(1) Size of the operation (2) Execution frequency (3) Commonality in the algorithms	(1) Execution frequency (2) Commonality in the algorithms (3) Hardware cost (4) Size of the operation	(1) Execution frequency (2) Commonality in the algorithms

The next target application in this demonstration is *library function generation*. In this case, long sequences of operations are preferred since they can increase code re-usability. Therefore, the length of the instruction sequence is considered as a key property. In addition, libraries are designed for consumption by many algorithms. Therefore, commonality among algorithms is also critical when selecting the dedicated operations. Furthermore, execution frequency is important, since, if the selected operation occurs frequently in the algorithm, it will further increase re-usability. It also will shorten the program code. According to this discussion, weighting priorities therefore need to be assigned based on the priority ranking in Table 6.1. Thus, differently from the previous target application, the length of the instruction sequence gets higher priority while the number of modes gets almost no value.

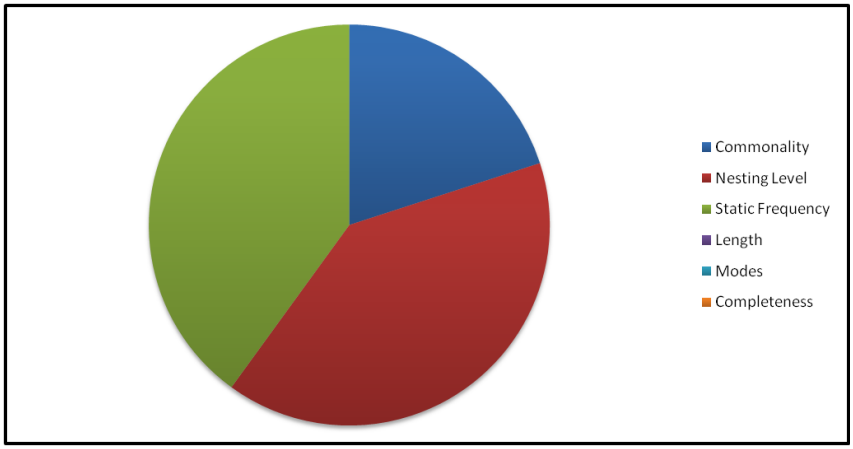
The third target application is *improving execution performance* of multiple algorithms. We performed this by identifying common “hot spots” in the algorithms. A “hot spot” is known as an expensive code segment/operation in which most processor time is spent at runtime. These are typically identified using profiling tools in a dynamic environment for different data sets. We have performed this case study to determine the accuracy of our methodology along with the weighting system. In this case, the key properties of the target application are, obviously, execution frequency and commonality (to retrieve common hot-spots). Therefore, in the weighting allocation, higher values for these two factors need to be assigned while other parameter weights are set to zero. The ranking of these specifications are displayed in Table 6.1. Here, similar priorities for execution frequency parameters and commonality are given. Thus, the execution frequency is emphasised in the resultant operations.



(a) Approximate proportions of weightings for custom instruction design



(b) Approximate proportions of weightings for library function



(c) Approximate proportions of weightings for "hot-spot" identification

Figure 6.1: Example weights for properties of operations, targeting different applications

Based on the above discussion and Table 6.1, appropriate weightings are assigned for the selected properties in the three application areas. Figure 6.1 shows the approximate proportions

each parameter gets in each target application. According to the diagram, the execution frequency and the commonality are praised in all the target applications. This is because these two factors significantly increase the effectiveness of the expected outcome for all the algorithms.

6.1.2 Analysing algorithms to measure properties of operations

In order to determine the criticalness of each operation in the target application, the algorithms need to be analysed in terms of the above mentioned properties. From the selected five properties, execution frequency has already been measured during our static analysis presented in Chapter 3. Therefore, analyses based on the other properties are presented below. Analysis of these new properties of operations are performed in a static environment. Thus, static analysis strategies, specially used for valid operation generation (Section 4.4.1) and operations matching (Section 4.4.2) are used in this property analysis tasks. Therefore, once all the analyses are integrated together as a single framework, each of them will be compatible with the others. This integration will significantly reduce the time and complexity costs of the whole process.

1. Number of Modes

   In computer architecture, the input/output ports of an operation are known as its modes. Since, in our analysis, operations are denoted as a sequence of assembly instructions, their unique operands are the input/outputs of the operation and hence their modes. (Here operands can be *in* or *out* or *in-out* modes, where an "in" mode indicates giving an input, an "out" mode indicates an output, and an "in-out" mode facilitates both "in" and "out" modes to the system.)

   Determination of the modes is performed by analysing each operation's operand list. When an operation is considered, the number of unique operands is identified to count the total number of modes. As described earlier, such analysis is performed in the semantic analysis step described in detail in Section 4.4.2. The semantic matching is performed to determine the similarity between two operations. When this is carried out, the list of operands is analysed to capture the data flow between each assembly instruction of the sequence. Based on the type and values of operands, different symbols are substituted. By counting the number of different symbols assigned for a sequence, its number of modes can be easily determined.

2. Commonality Factor

The commonality factor considers in how many objective algorithms a particular operation exists. As emphasised in the above section, in all target applications, commonality is considered as a key property. This can be determined by matching unique sequences of operations with those of other algorithms. When the two operations of two algorithms are matched, including their semantic meaning, the commonality factor is increased. This task is described in detail in Section 3.8.

3. Size of the operation

   Here, the size of an operation is obtained as the number of atomic operations (assembly instructions). Therefore, when the valid operations are generated during the static analysis (Section 4.4.1), their sizes can be easily determined.

6.1.3 Calculating weights and priorities

The final set of operations to represent the algorithms in a target application are obtained by ranking them based on the priority values assigned to each property. However, in order to compare the degree of criticalness between the operations, a total priority score (*Priority factor*) is calculated as follows, once the weights for the properties are allocated.

1. Parameter factor

   The parameter factor is calculated to determine the criticalness of a given operation towards a particular property compared to other operations. The comparison of an operation is made with the operation which received the highest score for a particular parameter. For example, if the weight allocated for *static frequency* is 0.3 and the highest *static frequency* value scored by any operation is 1000, then the parameter factor of the value 300 scored operation can be calculated as $(0.3/1000) * 300 = 0.09$.

2. Priority factor

   The priority factor of each operation is calculated to discover its overall criticalness towards the target application. Here, the parameter factors obtained for all considered performance properties are added to generate the total value.

Once the final candidate operations list is generated, the operations can be ranked according to their *priority factor* values. Thus the ranking of the operations depends on the weights

allocated by the properties and their individual scores. In this way, the user can calibrate the weighting system appropriately, based on the target application to obtain the best set of operations. The weighting systems are often heuristic since they are subjective to the target applications and other requirements. Therefore, the ranking process can produce the best results for a given application when it is correctly calibrated, based on the given specifications as explained above. The following case studies discuss the accuracy of our framework including the weighting system with demonstrative examples.

6.2 Case study 1: Generating Library Functions

This experiment demonstrates the usage of our weighting system to identify special-purpose operations targeting an application area. Here, a set of operations which show specific characteristics are selected to serve a set of cryptographic algorithms to improve the re-usability of their implementation. The special-purpose operations are selected based on the qualities required to generate a library function common to many objective algorithms.

6.2.1 Software re-usability

One of biggest challenges the IT world today faces is designing reusable software. Software refactoring, component-based architectural design and framework design are some of effective solutions to handle the demand. In an object-oriented environment, code reuse can be achieved by code refactoring. For example, a common property of member classes is moved to their super class. Thus, the property is inherited automatically by the parent class to the member classes without repeating the code. Library generation is another method to re-use software code effectively by implementing common functions in a shared location. In addition, these libraries can be used by non-object-oriented languages like C, where refactoring is not applicable. Another important usage of library functions is that they can be modified easily without affecting any dependent programs. When the requirements of a software system changes, developing new software from scratch is inefficient due to time and cost expenses. Therefore, the most effective option is to modify the existing system according to new functionalities. These changes can be achieved by slight changes of libraries and by refactoring functions, which are used by many dependent programs.

However, even though library software code reuse is beneficial in many ways, generation

of such libraries is not trivial due to difficulties of identifying suitable code segments. Therefore, we use our weighting system to identify suitable operations to be implemented as library functions. These functions thus will be accessed and used by multiple algorithms.

6.2.2 Experimental set-up

For this application, we have used the cryptographic algorithms listed in Table 2.2, introduced in Section 2.7.2. The implementations of these algorithms were taken from the open source polarSSL library. According to the table, the selected algorithms consist of many common functions executed inside common cypher modes such as *CBC* and *CTR* and hash functions such as *MD5* and *SHA1*.

The task of our weighting system was to identify functions often executed and shared by multiple algorithms. In addition, as emphasised earlier, the resultant operations need to be long, since short operations do not make significant contributions to code re-usability. Thus, length of the sequence, its commonality and its execution frequency are the key properties of those that will obtain higher weights. Since there are four property factors to determine the weights, it is important to investigate their effect on the expected outcome prior to assigning the specific weights. In this case, the length of a sequence plays a key role because short sequences should be prevented. In order to obtain frequently occurring operations, considerable weight needs to be assigned for both the *static frequency* and *nesting level* parameters. Likewise, a reasonable weight for the *commonality factor* is needed to emphasise the common usage of the operation. However, even though the weights for each parameter can be roughly assigned, based on such examinations, these weights might needed to be calibrated furthermore to obtain the best set of critical operations accurately.

6.2.3 Resultant characteristic operations

In the experiment, all cryptographic algorithms were analysed according to the procedure of the framework as described previously. First, all the valid operations in the considered set of algorithms are generated. Then they were analysed with respect to different properties to select suitable operations for library generation. As described in the previous section, appropriate weights were assigned for the chosen four parameters. Then *parameter factors* and hence the *priority factor* for each operation were calculated as described in Section 6.1.3. Table 6.2 illustrates the weights and maximum parameter values scored by all operations for all the

analyses in this case study. Based on the calculated priority factors, the operations are sorted in descending order. Tables 6.3 and 6.4 list the top operations obtained according to the calibrated performance parameter weights.

According to the results, the top three operations of the list are only two instructions long. However, our aim was to obtain longer sequences, which are more suitable to implement as library functions. These short sequences obtained their top position in the list because they have scored extremely high values in other parameters: *static frequency* (out of 1508), *nesting level* (out of 21.0), and *commonality* (out of 8). Thus, these operations have overtaken the priority reduction of their short lengths by other key parameters.

The next operation of the list is an operation of 30 instructions length. It contains code to perform 32-bit integer manipulation (Big endian) in hash functions. "Endian" is a method of storing bytes of data in the memory of a computer system. This is achieved in two different ways, known as "Big Endian" and "Little Endian". In Big Endian systems, the *most significant byte* of the word is stored in the smallest memory address while the *least significant byte* is stored in the largest address. In contrast, in Little Endian systems, the *least significant byte* of the word is stored in the smallest memory address, while the *most significant byte* is stored in the largest. These operations are commonly used in cryptographic hash functions in order to manipulate a given plain text or cypher. Table 6.5 provides detail about the algorithms which utilise these operations. The corresponding high level source code of the implementation of this operation is shown in Figure 6.2. The corresponding assembly code is shown in Figure 6.3.

However, even though the Big Endian assembly code only consists of 16 instructions, the resultant sequence was 30 instructions long. The reason for this is that in the algorithm these operations are implemented sequentially, one after the other. Hence, they produce long sequences which repeat the same instruction sequence, since they all belong to a one basic block. This scenario is explained in Figure 6.4. This also produces different instruction sequences, which repeat the same operation multiple times. This is clearly seen in the resultant operations listed in Tables 6.3 and 6.4 (the similar operations are shown in the same background colour). For instance, the sequences in green belong to single operation of Big Endian manipulation. In these sequences, the same string of instructions is repeated while producing different operations by having different starting indexes. Therefore, once all operations are sorted based on the priority parameters, the final operation selection step needs to be performed to eliminate such repetitive

operations from the final operations list.

Table 6.2: Performance Parameter Factors

Performance Parameter	Weights	Maximum Value Scored
Static Frequency	0.4	1508
Nesting Level	0.4	21
Commonality	0.3	8
No. of Modes	0.0	16
Sequence Length	0.5	30

```
42
43 /*
44  * 32-bit integer manipulation macros (big endian)
45  */
46 #ifndef GET_UINT32_BE
47 #define GET_UINT32_BE(n,b,i)                                    \
48 {                                                               \
49     (n) = ( (uint32_t) (b)[(i)    ] << 24 )                    \
50         | ( (uint32_t) (b)[(i) + 1] << 16 )                    \
51         | ( (uint32_t) (b)[(i) + 2] <<  8 )                    \
52         | ( (uint32_t) (b)[(i) + 3]       );                   \
53 }
54 #endif
55
56 /*
57  * 32-bit integer manipulation macros (big endian)
58  */
59 #ifndef PUT_UINT32_BE
60 #define PUT_UINT32_BE(n,b,i)                                    \
61 {                                                               \
62     (b)[(i)    ] = (unsigned char) ( (n) >> 24 );               \
63     (b)[(i) + 1] = (unsigned char) ( (n) >> 16 );               \
64     (b)[(i) + 2] = (unsigned char) ( (n) >>  8 );               \
65     (b)[(i) + 3] = (unsigned char) ( (n)       );               \
66 }
67 #endif
68
69
```

Figure 6.2: High level implementation of Big and Little Endian manipulations.

Likewise, considering the next top resultant instruction sequences, a few more critical operations were identified. The coloured rows of Tables 6.3 and 6.4 indicate such re-occurring operations found when selecting the final set of operations. Table 6.5 lists the final set of operations selected for implementation as library functions.

6.2.4 Performance gain of the Library Functions

The final performance gain of the application domain by the newly created library functions was measured. The main purpose of a library is code re-usability by multiple algorithms. Figure 6.5 shows the number of calls for each library function, from their objective algorithms in this case.

Table 6.3: Top prioritised sequences - part 1

Rank	Instruction Sequence	Static Frequency	Nest Level	Commonality	No. of Modes	Sequence Length
1	l32i.n l32i.n	1005	11	8	4	2
2	s32i.n l32i.n	1043	8	8	4	2
3	l32i.n l32i.n	969	8	8	3	2
4	l32i.n l8ui slli l32i.n l8ui slli l32i.n l8ui slli or or l32i.n l8ui or s32i.n l32i.n l8ui slli l32i.n l8ui slli l32i.n l8ui slli or or l32i.n l8ui or s32i.n	9	5	5	9	30
5	l32i.n l32i.n extui s8i l32i.n l32i.n extui s8i l32i.n l32i.n srli s8i l32i.n l32i.n s8i l32i.n l32i.n extui s8i l32i.n l32i.n extui s8i l32i.n l32i.n srli s8i l32i.n l32i.n s8i	6	5	5	16	30
6	l32i.n l8ui slli l32i.n l8ui slli l32i.n l8ui slli or or l32i.n l8ui or s32i.n l32i.n l8ui slli l32i.n l8ui slli l32i.n l8ui slli or or l32i.n l8ui or	9	5	5	9	29
7	l8ui slli l32i.n l8ui slli l32i.n l8ui slli or or l32i.n l8ui or s32i.n l32i.n l8ui slli l32i.n l8ui slli l32i.n l8ui slli or or l32i.n l8ui or s32i.n	9	5	5	9	29
8	l32i.n add.n l32i add.n add.n s32i l32i slli l32i extui or l32i add.n s32i l32r l32i l32i xor l32i and l32i xor addi l32i.n add.n l32i add.n add.n s32i l32i	60	4	4	16	30
9	add.n l32i add.n add.n s32i l32i slli l32i extui or l32i add.n s32i l32r l32i l32i xor l32i and l32i xor addi l32i.n add.n l32i add.n add.n s32i l32i slli	60	4	4	16	30

```
mov.n    a3, a1
l32i     a5, a1, 100
l8ui     a5, a5, 2
slli     a5, a5, 8
l32i     a6, a1, 100
l8ui     a6, a6, 1
slli     a6, a6, 16
l32i     a4, a1, 100
l8ui     a4, a4, 0
slli     a4, a4, 24
or       a4, a4, a6
or       a4, a4, a5
l32i     a2, a1, 100
l8ui     a2, a2, 3
or       a2, a2, a4
s32i.n   a2, a3, 0
```

Big Endian

```
mov.n    a9, a1
l32i     a10, a1, 84
l8ui     a10, a10, 3
slli     a10, a10, 24
l32i     a11, a1, 84
l8ui     a11, a11, 2
slli     a11, a11, 16
l32i     a12, a1, 84
l8ui     a12, a12, 1
slli     a12, a12, 8
l32i     a8, a1, 84
l8ui     a8, a8, 0
or       a8, a8, a12
or       a8, a8, a11
or       a8, a8, a10
s32i.n   a8, a9, 0
```

Little Endian

Figure 6.3: Assembly level implementation of Big and Little Endian manipulations.

The linearly increasing lines of the figure confirm the correspondence between the number of calls of the library functions when the problem size become larger. Thus, according to these results, we can conclude that the cryptographic algorithms are significantly benefited by the new library functions. They improve algorithms' performance in terms of code re-usability and hence the complexity of their implementation. Thus, our weighting system is capable of successfully choosing critical operations for a particular application based on expected characteristics defined by different performance parameters.

6.3 Case study 2: Custom Instruction Generation

Developing customisable processors is the heart of embedded system design. During the last few decades, electronic devices have dominated the worldwide market. They provide a variety of functionalities and services while expanding technology itself. These devices are closely related with our day-to-day life in modes of mobile phones, cars, cameras, toys, accessories, etc. These functionalities are implemented in different types of computer systems facilitated by different Instruction Set Architectures (ISAs). These architectures are designed to achieve different degrees of trade-off between performance, flexibility and energy consumption. Mainly,

C: sample_tie_crypto_all · T: Debug · Build Active · Run · Profile · Debug · Trace

gen_cry_hash.c | md5.c

```
    ctx->state[0] = 0x67452301;
    ctx->state[1] = 0xEFCDAB89;
    ctx->state[2] = 0x98BADCFE;
    ctx->state[3] = 0x10325476;
    ctx->state[4] = 0xC3D2E1F0;
}

void sha1_process( sha1_context *ctx, const unsigned char data[64] )
{
    uint32_t temp, W[16], A, B, C, D, E;

    GET_UINT32_BE( W[ 0], data,  0 );
    GET_UINT32_BE( W[ 1], data,  4 );
    GET_UINT32_BE( W[ 2], data,  8 );
    GET_UINT32_BE( W[ 3], data, 12 );
    GET_UINT32_BE( W[ 4], data, 16 );
    GET_UINT32_BE( W[ 5], data, 20 );
    GET_UINT32_BE( W[ 6], data, 24 );
    GET_UINT32_BE( W[ 7], data, 28 );
    GET_UINT32_BE( W[ 8], data, 32 );
    GET_UINT32_BE( W[ 9], data, 36 );
    GET_UINT32_BE( W[10], data, 40 );
    GET_UINT32_BE( W[11], data, 44 );
    GET_UINT32_BE( W[12], data, 48 );
    GET_UINT32_BE( W[13], data, 52 );
    GET_UINT32_BE( W[14], data, 56 );
    GET_UINT32_BE( W[15], data, 60 );

#define S(x,n) ((x << n) | ((x & 0xFFFFFFFF) >> (32 - n)))

#define R(t)                                        \
(                                                   \
    temp = W[(t -  3) & 0x0F] ^ W[(t - 8) & 0x0F] ^ \
           W[(t - 14) & 0x0F] ^ W[ t      & 0x0F],  \
    ( W[t & 0x0F] = S(temp,1) )                     \
)

#define P(a,b,c,d,e,x)                              \
{                                                   \
```

XPRES Analysis | Profile Disassembly

Count	Address	Instruction
8,197	6001040d	l8ui a12, a12, 7
8,197	60010410	or a12, a12, a14
0	60010413	s32i.n a12, a13, 0
		GET_UINT32_BE(W[2], data, 8);
0	60010415	addi.n a9, a1, 8
0	60010417	l32i a11, a1, 100
8,197	6001041a	l8ui a11, a11, 10
8,197	6001041d	slli a11, a11, 8
0	60010420	l32i a12, a1, 100
8,197	60010423	l8ui a12, a12, 9
8,197	60010426	slli a12, a12, 16
0	60010429	l32i a10, a1, 100
8,197	6001042c	l8ui a10, a10, 8
8,197	6001042f	slli a10, a10, 24
0	60010432	or a10, a10, a12
0	60010435	or a10, a10, a11
0	60010438	l32i a8, a1, 100
8,197	6001043b	l8ui a8, a8, 11
8,197	6001043e	or a8, a8, a10
0	60010441	s32i.n a8, a9, 0
		GET_UINT32_BE(W[3], data, 12);
0	60010443	addi.n a5, a1, 12
0	60010445	l32i a7, a1, 100
8,197	60010448	l8ui a7, a7, 14
8,197	6001044b	slli a7, a7, 8
0	6001044e	l32i a8, a1, 100
8,197	60010451	l8ui a8, a8, 13
8,197	60010454	slli a8, a8, 16
0	60010457	l32i a6, a1, 100
8,197	6001045a	l8ui a6, a6, 12
8,197	6001045d	slli a6, a6, 24
0	60010460	or a6, a6, a8
0	60010463	or a6, a6, a7
0	60010466	l32i a4, a1, 100
8,197	60010469	l8ui a4, a4, 15
8,197	6001046c	or a4, a4, a6
0	6001046f	s32i.n a4, a5, 0
		GET_UINT32_BE(W[4], data, 16);
0	60010471	addi a15, a1, 16
0	60010474	l32i a3, a1, 100
8,197	60010477	l8ui a3, a3, 18
8,197	6001047a	slli a3, a3, 8
0	6001047d	l32i a4, a1, 100
8,197	60010480	l8ui a4, a4, 17
8,197	60010483	slli a4, a4, 16
0	60010486	l32i a2, a1, 100
8,197	60010489	l8ui a2, a2, 16

Figure 6.4: Big Endian implementation and corresponding assembly code.

they can be categorised into two groups of architectures known as general-purpose processors and application specific processors.

General purpose processors are considered to be the most flexible processors. They allow various software applications to run on the processor with minimum hardware mapping concerns. Therefore, a huge number of applications have been developed, targeting these architectures. Basically, the general-purpose processors are developed based on the Von Neumann computing model. It consists of a memory to save programming data (instructions and data), a control unit, and an arithmetic and logic unit. In the execution, the instructions are read sequentially and executed in five steps: instruction fetch, decoding, read operands, read from memory, execute, write results. The five cycles of execution of this model form a significant performance overhead and hence, energy consumption overhead, even after the process is improved by various techniques such as pipelining, super-scalar execution, register renaming, etc [Bobda, 2007].

Application specific processors, on the other hand, favour a specific application domain

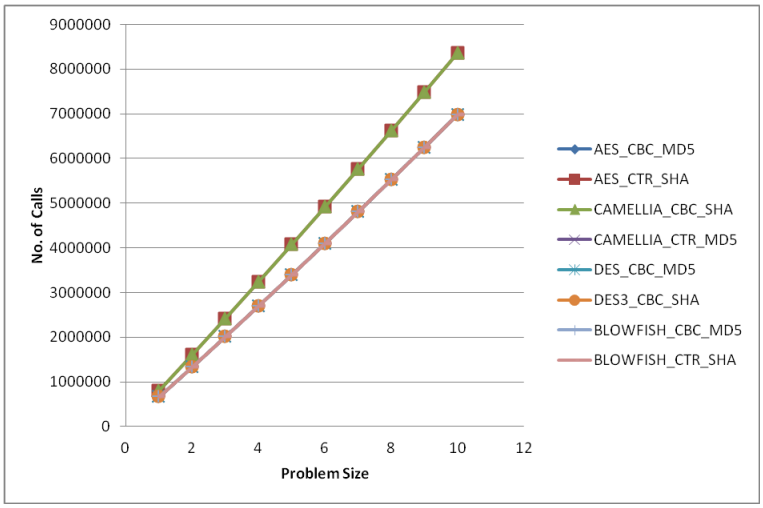


Figure 6.5: Number of calls to chosen cryptographic operations.

by improving their performance for that domain. These types of architecture can be further categorised into three groups: Application Specific Integrated Circuits (ASICs), Application-Specific Instruction-set Processors (ASIP), and Application Domain-Specific Processors (ADSPs). An ASIC is a circuit designed to perform a specific task, such as a processor in a washing machine. The architecture designed for the target applications reduces energy consumption by using a hard-wired functional unit with a software component to set run-time configurable parameters. However, the energy savings of the ASIC are at the cost of the flexibility of the processor by limiting its programmability. Therefore, one system designed at targeting a particular application cannot be re-used. Hence, hardware design needed to be done again and again for different functionalities or applications. The time and cost involved in the whole ASIC design process, including Non-Recurring Engineering (NRE) costs, high masking and testing costs, and manufacturing and integrating costs, make ASIC solutions not always feasible.

ASIP and ADSP processors consist of basic instructions along with customisable instructions. A custom instruction is designed for a special-purpose operation that is local and critical to the target application/s. For instance, Multiply ACcumulate (MAC) is a typical operation executed frequently in signal processing applications. When this operation is executed in a general-purpose processor, the intermediate result of the addition will be written to and read from the memory. Instead, a special hardware circuit is implemented to perform the addition immediately after multiplication. This modification avoids writing and reading the intermediate results and hence a considerable amount of time and energy is saved. An ASIP ISA is designed for tailoring one application, while an ADSP targets a domain of applications. During the past

years, development of the ADSP design research area grew. This is due to positive trade-offs obtained between performance and energy consumption of ADSPs compared to other available architectures while serving multiple applications at a time.

However in terms of optimisation, an ASIP can be more optimised. This is because the architecture is designed for only one application, whereas in an ADSP all the applications in the domain should be benefited by the design [Amold and Corporaal, 2001]. Therefore, executing other applications on this kind of specific processor can be inefficient [Fanucci et al., 2006]. Thus, when designing ADSPs it is essential to include an optimised set of customisable instructions that can be used by all target applications.

Table 6.4: Top prioritised sequences - part 2

Rank	Instruction Sequence	Static Frequency	Nest Level	Commonality	No. of Modes	Sequence Length
10	l32i add.n add.n s32i l32i slli l32i extui or l32i add.n s32i l32r l32i l32i xor l32i and l32i xor addi l32i.n add.n l32i add.n add.n s32i l32i slli l32i	60	4	4	16	30
11	l32i.n l32i.n extui s8i l32i.n l32i.n extui s8i l32i.n l32i.n srli s8i l32i.n l32i.n s8i l32i.n l32i.n extui s8i l32i.n l32i.n extui s8i s8i l32i.n l32i.n srli l32i.n l32i.n	6	5	5	16	29
12	l32i.n extui s8i l32i.n l32i.n extui s8i l32i.n l32i.n srli s8i l32i.n l32i.n s8i l32i.n l32i.n extui s8i l32i.n l32i.n extui s8i l32i.n l32i.n srli s8i l32i.n l32i.n s8i	6	5	5	16	29
13	or l32i add.n s32i l32r l32i l32i xor l32i and l32i xor addi l32i.n add.n l32i add.n add.n s32i l32i slli l32i extui or l32i add.n s32i l32r l32i l32i	56	4	4	16	30
17	l32r l32i l32i xor l32i and l32i xor addi l32i.n add.n l32i add.n add.n s32i l32i slli l32i extui or l32i add.n s32i l32r l32i l32i xor l32i and l32i	56	4	4	16	30
19	l32i l8ui slli l32i l8ui slli l32i l8ui slli l32i l8ui or or or s32i.n addi l32i l8ui slli l32i l8ui slli l32i l8ui slli l32i l8ui or or or	48	4	4	10	30
20	l8ui slli l32i l8ui slli l32i l8ui slli l32i l8ui or or or s32i.n addi l32i l8ui slli l32i l8ui slli l32i l8ui slli l32i l8ui or or or s32i.n	48	4	4	10	30

Table 6.5: Final set of Operations to implement library functions

Instruction Sequence	Operation	Common Algorithms
l32i.n l8ui slli l32i.n l8ui slli l32i.n l8ui slli or or l32i.n l8ui or s32i.n	GET_UNIT32_BE (Big endian manipulation)	aesCBC md5, blowfishCBC md5, blowfishCTR sha, des3CBC sha, desCBC md5
l32i.n l32i.n extui s8i l32i.n l32i.n extui s8i l32i.n l32i.n srli s8i l32i.n l32i.n s8i l32i.n l32i.n extui s8i l32i.n l32i.n extui s8i l32i.n l32i.n srli s8i l32i.n l32i.n s8i	PUT_UNIT32_LE (Little endian manipulation)	aesCBC md5, blowfishCBC md5, blowfishCTR sha, des3CBC sha, desCBC md5
l32i l32i xor l32i and l32i xor addi l32i.n add.n l32i add.n add.n s32i l32i slli l32i extui or l32i add.n s32i	P (32 bit integer calculation)	aesCBC md5, blowfishCBC md5, camelliaCTR md5, desCBC md5
l32i l8ui slli l32i l8ui slli l32i l8ui slli l32i l8ui or or or s32i.n	GET_UINT32_LE (Little endian manipulation)	aesCBC md5, blowfishCBC md5, camelliaCTR md5, desCBC md5

Table 6.6: Performance Parameter Factors for ADSP design

Performance Parameter	Weights	Maximum Value Scored
Static Frequency	0.4	1508
Nest Level	0.4	21
Commonality	0.3	8
No. of Modes	0.2	16
Sequence Length	0.2	30

6.3.1 Selection of Custom Instructions for an ADSP

This case study was performed for the same set of cryptographic algorithms listed in Table 2.2. The purpose of this experiment was selecting the most suitable set of operations to be implemented as custom instructions of an ADSP for cryptographic algorithms. First, the algorithms were implemented in C/C++ and we obtained their assembly code for the Xtensa processor [Tensilica, 2014]. Using the high level and assembly level codes, the analyses of the framework were performed as described earlier.

First, all possible operations were expressed with different analysis measurements. Then the operations were ranked based on customised weights according to the ADSP design requirements. As discussed earlier, the execution frequency and the commonality among algorithms were key properties of a potential custom operation, since they are meant by most of its target applications to execute frequently. Therefore, larger weights for static frequency, nesting level and commonality were assigned. When designing instructions for a processor, it is important to understand the architecture of the target processor. It defines the specifications of its instructions such as the number of operands. Therefore, a suitable weight for the performance factor of *number of modes* also needed to be allocated.

In existing custom instruction generation methodologies, the number of modes is used as a key factor to cut-off the candidate operation list. It defines key features of the target architecture, such as supporting either MISO (Multiple-Input Single-Output) or MIMO (Multiple-Input Multiple-Output) [Galuzzi and Bertels, 2011] operations. However, it is proven that MIMO operations with more than 2 inputs and 1 output can achieve significant speedups compared to operations with up to 2 inputs and 1 output [Galuzzi and Bertels, 2011]. Nevertheless, since the number of inputs of an instruction can be increased only up to a certain level under the architectural constraints, suitable limitations are needed to be applied on potential operations.

In our case study, based on the Xtensa processor architecture (custom TIE instructions only allow three input/outputs), the mode performance factor of each operation was calculated. In this case, the operations consisting of 3 modes got high weights compared to fewer or larger mode values. However, this can lead to resulting short instruction sequences. Therefore, the total weight of the modes performance factor was calibrated to a smaller value compared to the frequency and commonality factors.

The remaining performance factor of the sequence length was given a zero weight. As the effect of the length of the operation needed to be ignored because even lengthy operations can be implemented as one or more beneficial custom instructions. Table 6.6 lists weights allocated for this application. Table 6.7 shows the top operations obtained from the operation ranking according to the above weights.

6.3.2 Custom instruction generation experiment

In order to examine the performance improvements of the above selected operations, they were implemented as custom instructions. The experiment was carried out by implementing them in a simulation environment facilitated by Tensilica's Xtensa customisable processor [Tensilica, 2014]. Xtensa allows designing custom instructions and attaching them to the core ISA as instruction extensions. These new processor configurations with the new ISA are automatically supported by the software tool set comprised of a new compiler and linker for the new ISA. These custom instructions are known as "TIE" instructions in this environment. Here, the limitation of the number of operands per instruction is three, similar to common RISC architectures.

The new ISA with the new custom instructions is simulated by executing the objective algorithms. The profiled data is used to measure the performance difference between the original and the new ISA. Mainly three performance parameters were considered: cycle count, instruction count and energy consumption. In order to measure the performance of the custom instructions, the operations in the target applications were replaced with corresponding instructions. However, to replace the C/C++ level implementation by custom instructions, the implemented custom instructions needed to be represented by a single or multiple algorithm level statement/s. Therefore, in the experimental design, the custom instruction selection was needed to be guided by one other parameter. It should denote the correspondence of assembly level instruction sequences with the algorithm level implementation.

Therefore, to do that, a new parameter was introduced called “completeness”. It indicates whether or not the assembly sequence is representing a complete operation in the high level algorithm implementation. To perform this automatically, a tool was developed to get assembly code with its corresponding high level source code. For each sequence “0” or “1” is attached based on its completeness. Then, for all the algorithms and their operations, this analysis was performed to indicate each operation’s completeness. Once all the unique operations were generated, they were ranked with the previously allocated weights. For the new parameter a higher weighting value was allocated so that the operations which have “1” get a higher priority than others. Therefore these operations came to the top of the ranking order. Table 6.8 lists operations resulting from this new ranking with the new performance parameter.

Table 6.7: Top priotorised sequences from ADSP design experiment

Rank	Instruction Sequence	Static Frequency	Nest Level	Commonality	No. of Modes	Sequence Length
1	slli l32i	1195	18	8	4	2
2	l32i.n l32i.n	1031	21	8	4	2
3	slli l32i	1508	10	8	4	2
4	l32i.n xor	1150	16	8	3	2
5	l32i slli	1256	8	8	3	2
6	l32i slli l32i	1160	8	8	4	3
7	l32i.n l32i.n	907	12.5	8	3	2
8	l32i.n l32i.n add.n addi l32i.n l32i.n add.n addi l8ui l32i.n l32i.n add.n l8ui xor s8i movi l32i.n l32i.n add.n add.n movi l32i.n l32i.n add.n add.n l8ui	8	16	8	8	30
10	extui or	803	13	8	3	2
11	addi l32i.n	1012	9	8	3	2
13	add.n add.n	616	16	8	3	2
15	or s32i.n	480	18	8	3	2
17	addx4 l32i.n	467	18	8	3	2
18	l32i xor	992	8	8	4	2
20	l32i add.n	864	10	8	4	2

6.3.3 Performance gain of the custom instructions

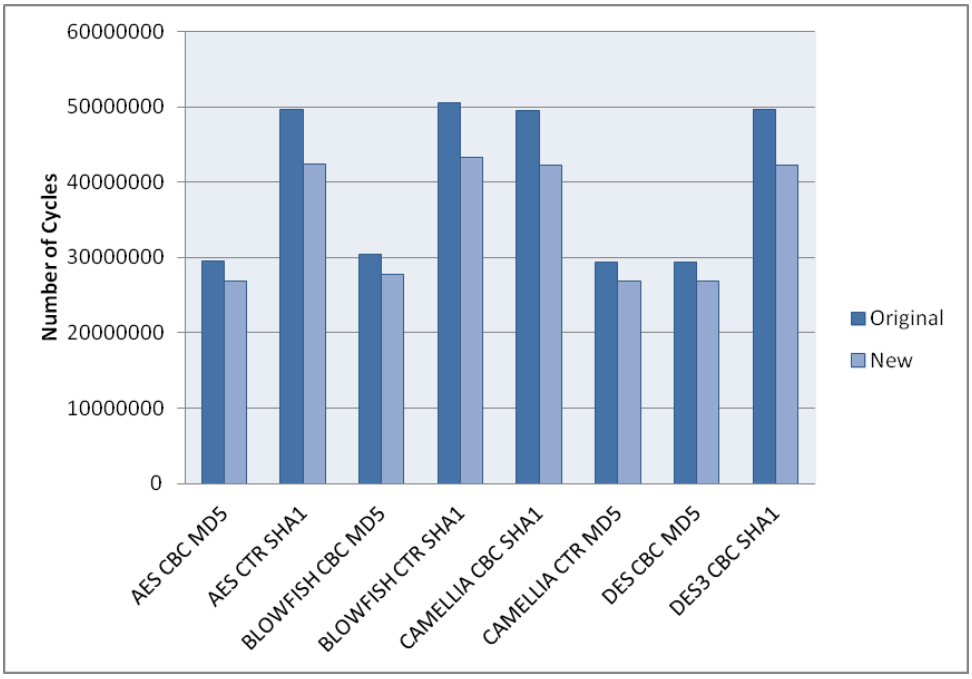
The resultant operations of this experiment were implemented in the Xtensa processor [Tensilica, 2014]. We measured the performance difference between the original and the new ISA by executing the target cryptographic algorithms. Table 6.9 summarises these performance measurements. Figures 6.6a, 6.6b and 6.7a illustrate further comparisons of the results in terms of different parameters. Figure 6.7b shows the total performance improvement obtained by the custom instructions. Thus, the performance improvements obtained in this experiment show that our weighting system is capable of selecting suitable operations for this application domain. It further emphasises the capability of our system to divert the selection of the operations according to the target application.

Table 6.8: Top prioritised sequences obtained with modified parameters

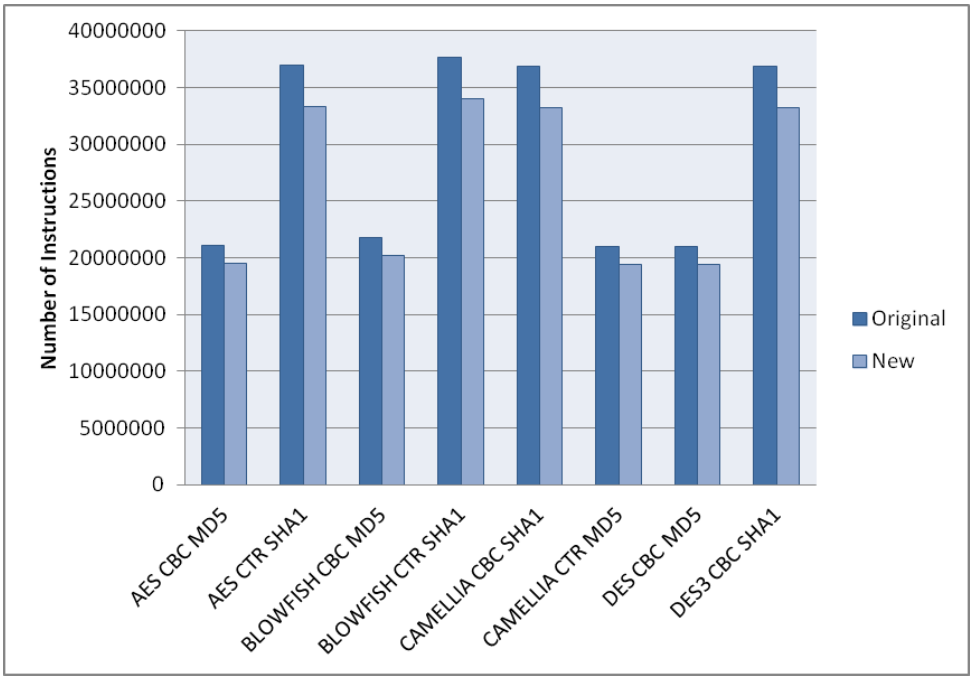
Rank	Instruction Sequence	Completeness	Static Frequency	Nest Level	Commonality	No. of Modes	Sequence Length
1	entry s32i.n	1	351	8	8	3	2
2	movi.n s32i.n	1	324	14.5	8	3	2
3	entry s32i.n s32i.n	1	255	8	8	4	3
4	entry s32i.n s32i.n s32i.n	1	173	8	8	5	4
5	l32i.n addi.n s32i.n	1	167	15.5	8	3	3
6	l32i.n s32i.n	1	83	9	8	3	2
7	l32i l32i xor l32i and l32i xor addi.n l32i.n add.n l32i add.n add.n s32i l32i slli l32i srli or l32i add.n s32i	1	64	8	8	12	22
8	entry s32i.n s32i.n s32i.n s32i.n	1	51	8	8	6	2
9	l32i.n l32i.n s32i.n	1	49	8	8	4	3
10	l32i.n movi.n s32i.n	1	41	8	8	4	3
11	entry s32i.n s32i.n s32i.n s32i.n s32i.n	1	36	8	8	7	6
12	entry s32i.n s32i.n s32i.n s32i.n s32i.n s32i.n	1	28	8	8	7	6

Table 6.9: Evaluation results comparison of custom instructions generated for all cryptographic algorithms

Algorithm	**Original**			**New**			**Performance Gain (%)**		
	Cycles	Instructions	Energy (μJ)	Cycles	Instructions	Energy (μJ)	Cycles	Instructions	Energy (μJ)
AES CBC MD5	29548357	21093663	2542273395	26933415	19528025	2329822523	8.85	7.425	8.36
AES CTR SHA1	49651366	36951761	4243879969	42356036	33312293	3608667613	14.69	9.85	14.97
BLOWFISH CBC MD5	30430068	21781449	2618517046	27806708	20207433	2408988852	8.62	7.23	8.00
BLOWFISH CTR SHA1	50532759	37639240	4321905426	43237429	33999772	3723050704	14.44	9.70	13.86
CAMELLIA CBC SHA1	49547598	36871350	4235030811	42251043	33231182	3636131475	14.73	9.87	14.14
CAMELLIA CTR MD5	29432704	21004245	2530632049	26809664	19430421	2321070607	8.91	7.49	8.28
DES CBC MD5	29441407	21010145	2531373272	26818020	19436107	2321784511	8.91	7.49	8.28
DES3 CBC SHA1	49589163	36901389	4238562483	42293254	33261714	3639717182	14.71	9.86	14.13



(a) Processor cycle comparison of original and new ISA



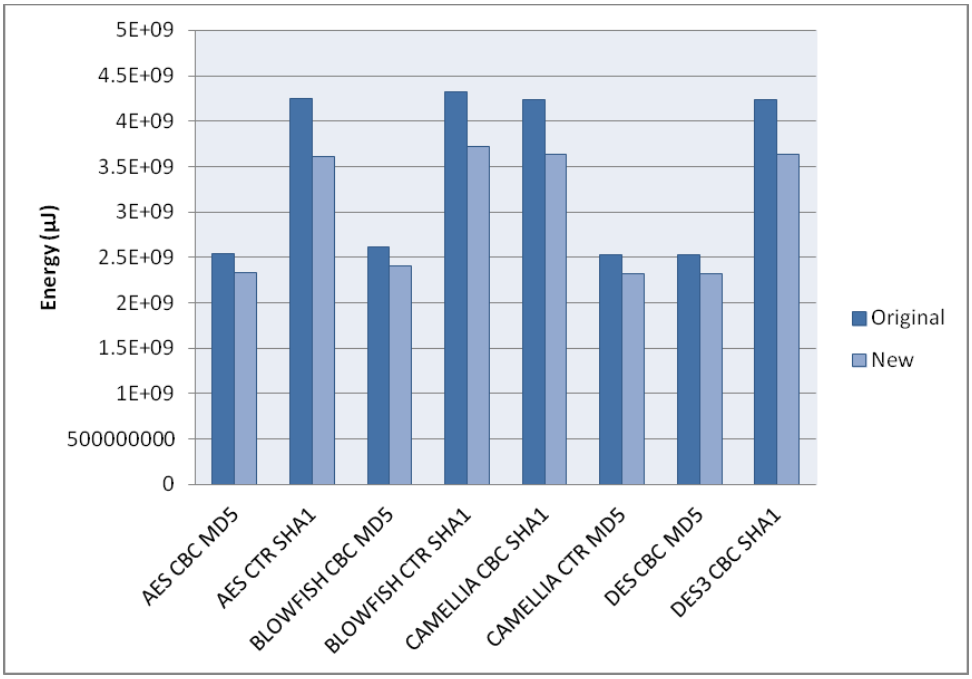
(b) Instructions comparison of original and new ISA

Figure 6.6: Processor cycle and instruction count variations of the original ISA vs the new ISA for the cryptographic algorithms

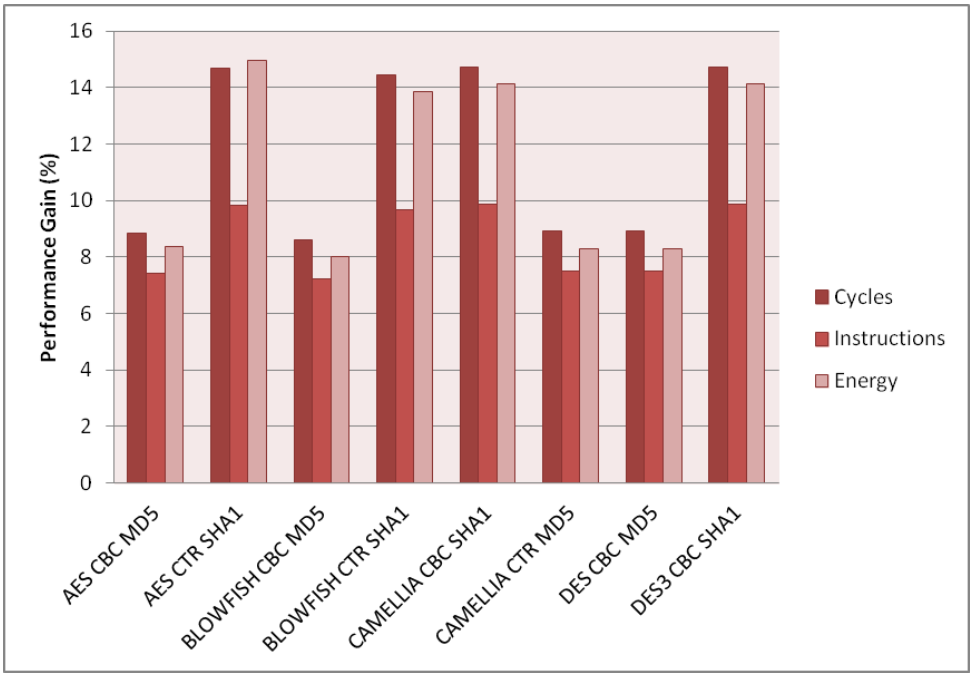
In order to confirm the results of our previous experiment, two more experiments were carried out. Here, the custom instructions were generated targeting only four algorithms from the cryptographic algorithm set previously considered at a time. The algorithms were split into two groups, one with the MD5 hash function and the other with the SHA1 hash function. If the weighting system identified the suitable operations for custom instructions correctly, the targeted algorithm set must show higher performance improvements than the other set, since they share expensive similar hash functionalities (in both cases).

In the experiments, candidate operation lists were generated for the above two algorithm sets. Then the operations were ranked according to their priority factors calculated based on the assigned weights. The operation list was then re-ranked to select the most suitable set of operations to be implemented as custom instructions. Tables 6.10 and 6.11 illustrate the results obtained for these two case studies. Variations of these results are illustrated in the Figures 6.8 to 6.11.

According to Figures 6.8 and 6.9, the performances of the new ISA have favoured the algorithms that used MD5 hash function. Different to the performance variations in Figures 6.6 and 6.7 where all the eight algorithms showed significant performance gains, here the algorithm set, which used the SHA1 hash function, has not obtained any performance improvement. This implies that the selected special-purpose operations belong to the MD5 hash function that was used by the other set of algorithms. Similarly, the performance variations in Figures 6.10 and 6.11 show significant performance improvements only for the set of algorithms that employ the SHA1 hash function. Thus, in both cases, common and frequently executed operations in the two hash functions are identified successfully due to their highly expensive computations. This experiment thus confirms the accuracy of our weighting system by identifying special-purpose operations common to the two considered sets of cryptographic algorithms.



(a) Energy comparison of original and new ISA.

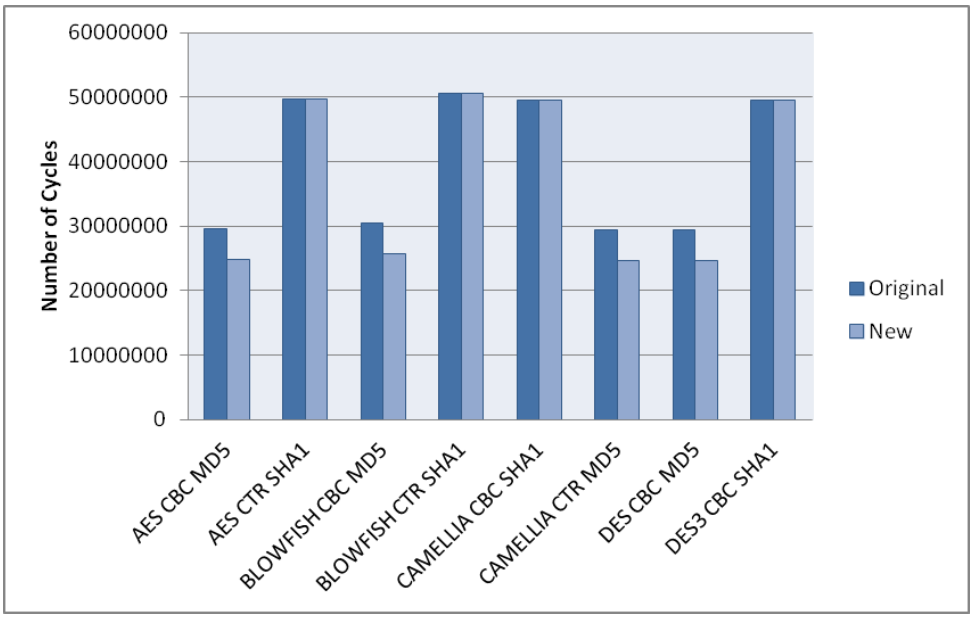


(b) Performance Gain comparison of original and new ISA (%).

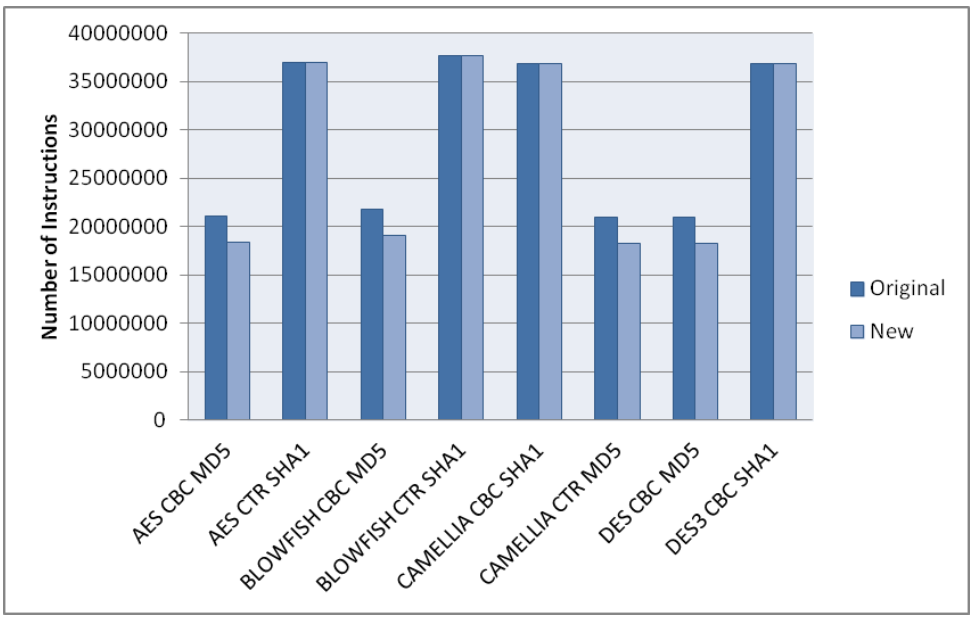
Figure 6.7: Energy and performance gain of the original ISA vs the new ISA for the cryptographic algorithms.

Table 6.10: Evaluation results comparison of custom instructions generated for MD5 hash based algorithms

Algorithm	**Original**			**New**			**Performance Gain (%)**		
	Cycles	Instructions	Energy (μJ)	Cycles	Instructions	Energy (μJ)	Cycles	Instructions	Energy (μJ)
AES CBC MD5	29548357	21093663	2542273395	*same*	*same*	*same*	0.0	0.0	0.0
AES CTR SHA1	49651366	36951761	4243879969	40208305	32099051	3462589135	19.02	13.13	18.41
BLOWFISH CBC MD5	30430068	21781449	2618517046	*same*	*same*	*same*	0.0	0.0	0.0
BLOWFISH CTR SHA1	50532759	37639240	4321905426	41089814	32786616	3540732367	18.69	12.89	18.07
CAMELLIA CBC SHA1	49547598	36871350	4235030811	40103054	32017780	3453600892	19.062	13.16	18.45
CAMELLIA CTR MD5	29432704	21004245	2530632049	*same*	*same*	*same*	0.0	0.0	0.0
DES CBC MD5	29441407	21010145	2531373272	*same*	*same*	*same*	0.0	0.0	0.0
DES3 CBC SHA1	49589163	36901389	4238562483	40144950	32048087	3457167848	19.04	13.15	18.44

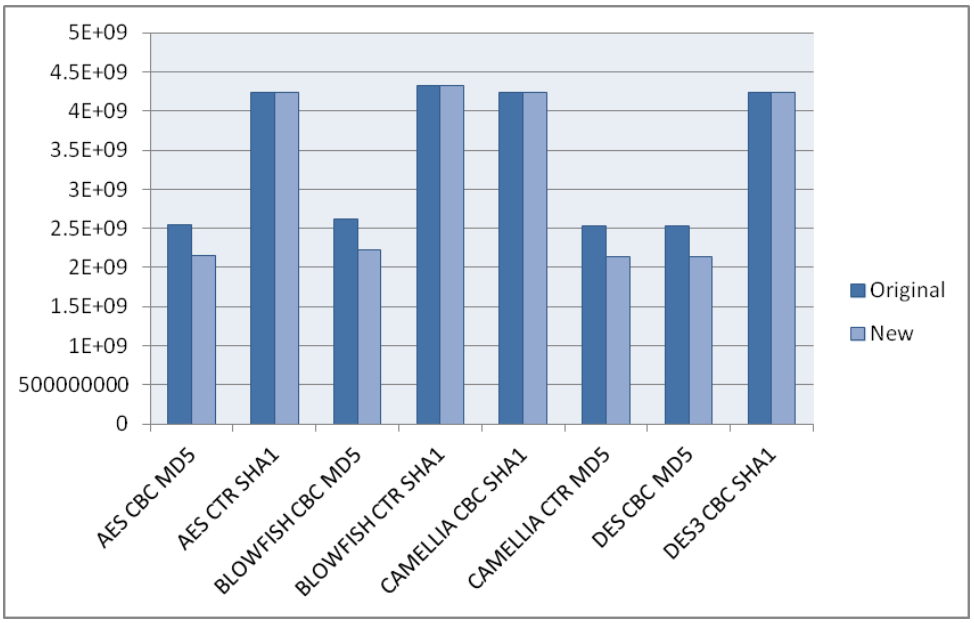


(a) Processor cycle comparison of original and new ISA

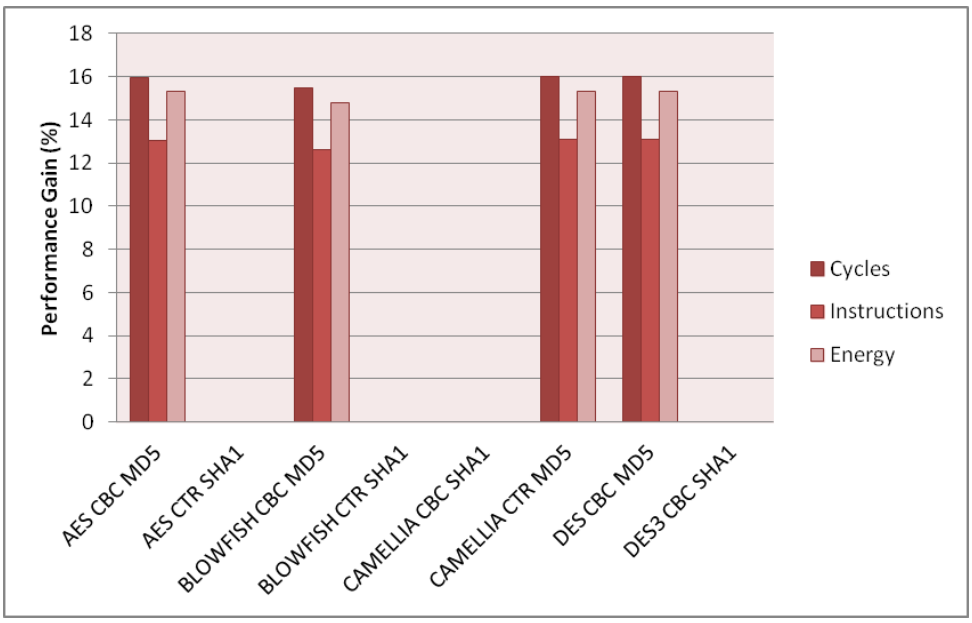


(b) Instructions comparison of original and new ISA

Figure 6.8: Processor cycle and instruction count variations of the original ISA vs the new ISA for the MD5 based algorithms



(a) Energy comparison of original and new ISA

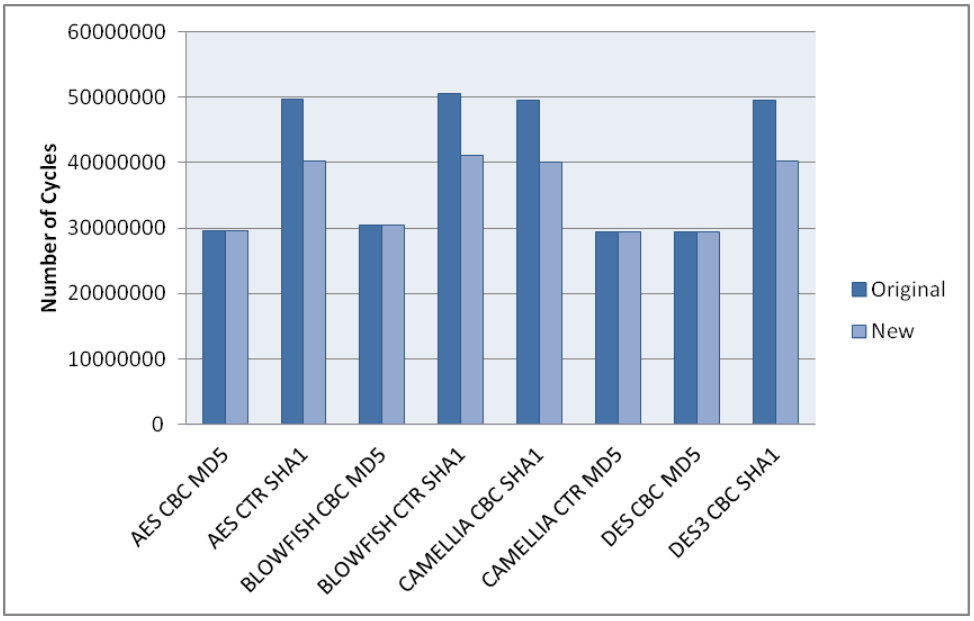


(b) Performance Gain comparison of original and new ISA (%)

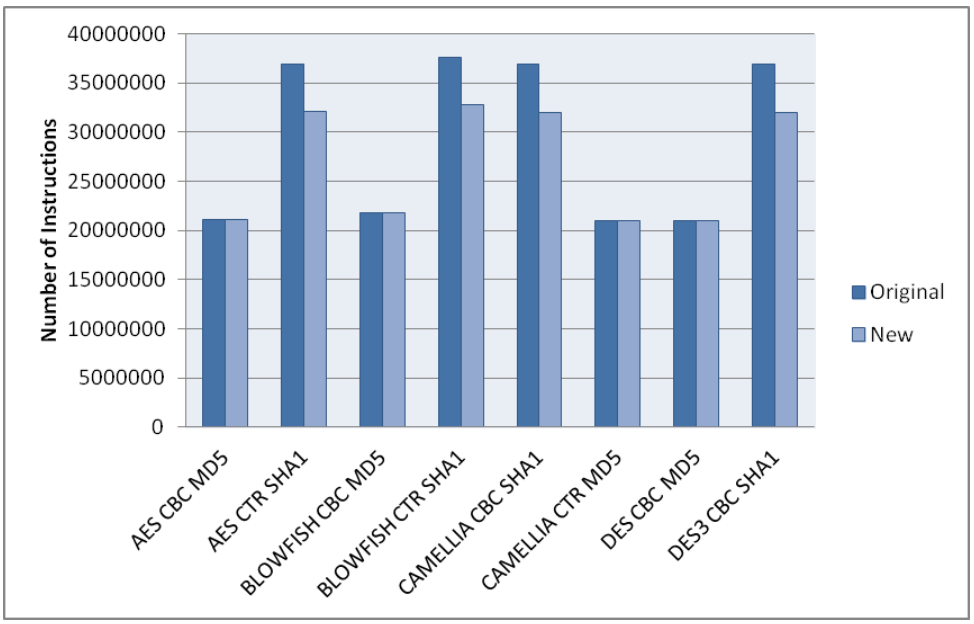
Figure 6.9: Energy and performance gain of the original ISA vs the new ISA for the MD5 based algorithms

Table 6.11: Evaluation results comparison of custom instructions generated for SHA1 hash based algorithms

Algorithm	**Original**			**New**			**Performance Gain (%)**		
	Cycles	Instructions	Energy (μJ)	Cycles	Instructions	Energy (μJ)	Cycles	Instructions	Energy (μJ)
AES CBC MD5	29548357	21093663	2542273395	24834383	18347259	2152748322	15.95	13.02	15.32
AES CTR SHA1	49651366	36951761	4243879969	*same*	*same*	*same*	0.0	0.0	0.0
BLOWFISH CBC MD5	30430068	21781449	2618517046	25716214	19035119	2231065381	15.49	12.61	14.80
BLOWFISH CTR SHA1	50532759	37639240	4321905426	*same*	*same*	*same*	0.0	0.0	0.0
CAMELLIA CBC SHA1	49547598	36871350	4235030811	*same*	*same*	*same*	0.0	0.0	0.0
CAMELLIA CTR MD5	29432704	21004245	2530632049	24719308	18258164	2142872547	16.01	13.07	15.32
DES CBC MD5	29441407	21010145	2531373272	24727102	18263502	2143540339	16.01	13.07	15.32
DES3 CBC SHA1	49589163	36901389	4238562483	*same*	*same*	*same*	0.0	0.0	0.0



(a) Processor cycle comparison of original and new ISA



(b) Comparison of the number of instructions executed by original and new ISA

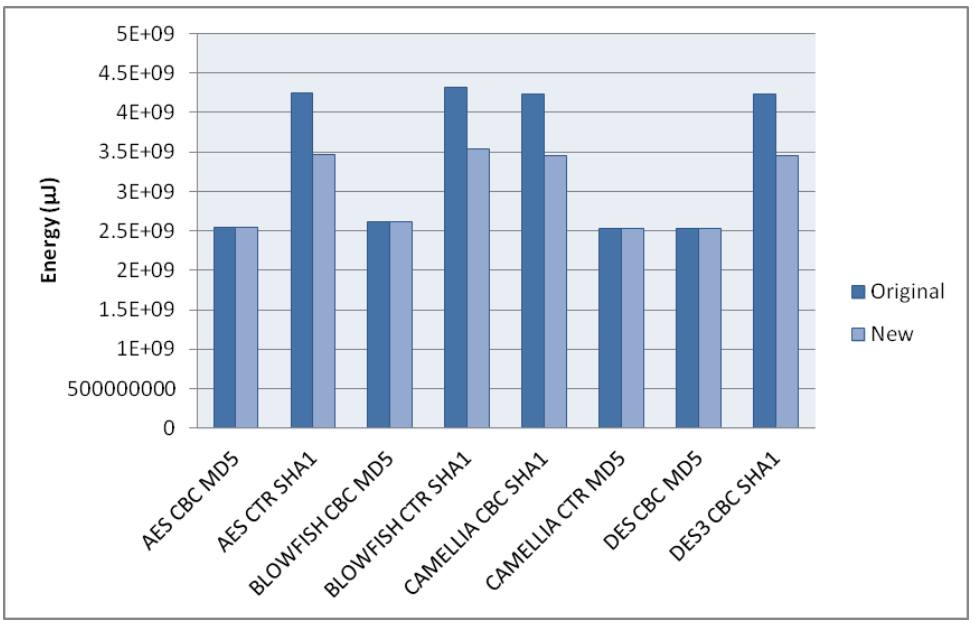
Figure 6.10: Processor cycle and instruction count variations of the original ISA vs the new ISA for the SHA1 based algorithms

We can conclude that the operations identified from our identification framework were truly beneficial for implementation as custom instructions. It is confirmed that our framework is capable of identifying suitable operations when the requirements of the target applications are expressed by their parameter weights.

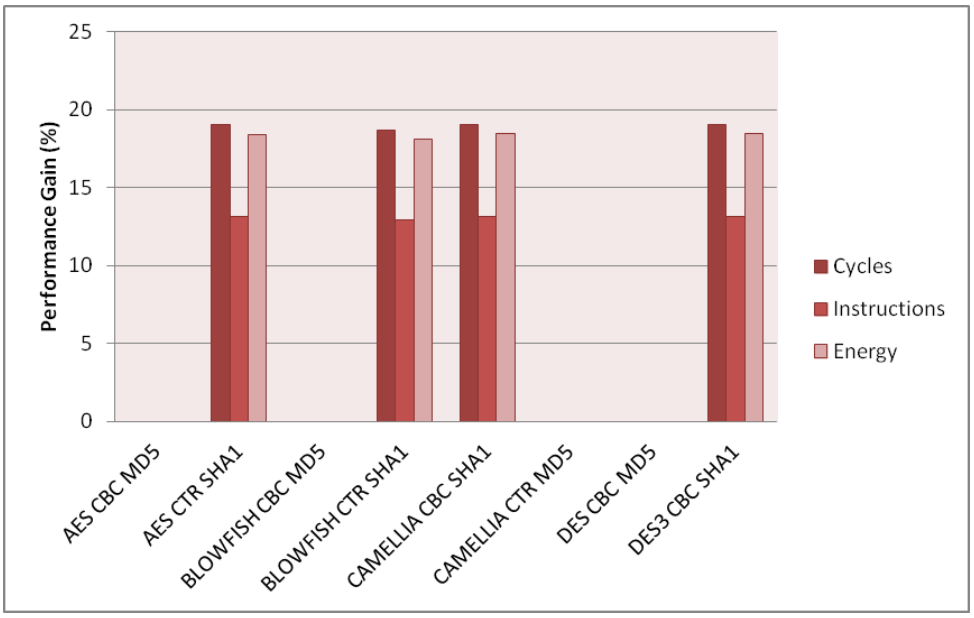
6.4 Case Study 3: Finding "Hot Spots" of a Set of Slgorithms

This case study is performed to the identify "hot spots" of a set of three *sequence alignment* algorithms introduced in Section 2.7.3. A hot spot is an operation that is executed frequently in an algorithm's runtime. They are critical to the algorithm since they consume a major part of processor executions of the whole algorithm. Typically they are found tracking the runtime behaviour of algorithms for suitable data sets using profiling tools. Here, we identify them by using our static analysis strategies for a set of multiple algorithms. The accuracy of our resultant operations is evaluated using dynamic experiments.

As described previously, the steps involved in the framework were performed automatically to obtain the final set of characteristic operations. First, the algorithms were implemented in C/C++ and their assembly instruction code generated targeting the Tensilica's Xtensa Xplorer [Tensilica, 2014] configurable processor. From the assembly source code all possible atomic instruction sequences were generated. The sequences were then analysed to determine their criticalness towards the given target application by the prioritised properties.



(a) Energy comparison of original and new ISA



(b) Performance Gain comparison of original and new ISA (%)

Figure 6.11: Energy and performance gain of the original ISA vs the new ISA for the SHA1 based algorithms

Table 6.12: Top characteristic operations resulting from the analysis of *sequence alignment* algorithms

Op. Name	Priority Factor	Instruction Sequence	Com. Factor	Nest Level	Static Fre.
A	0.921429	movi.n s32i	3	3	174
B	0.899072	s32i l32i	3	3	165
C	0.87775	movi.n s32i.n	3	3	155
D	0.847841	l32i l32i	3	5	93
E	0.786429	s32i.n l32i.n	3	3	116
F	0.779204	movi.n s32i l32i	3	4	88
G	0.774253	s32i movi.n l32i	3	7	12
H	0.767053	s32i.n s32i.n	3	6	33
I	0.76344	l32i addx4	3	6.5	19
J	0.758842	l32i.n l32i.n	3	3	104
K	0.734565	l32i.n addi.n	3	5	43
L	0.713875	l32i.n addi.n s32i.n	3	5	34
M	0.71289	addi.n s32i.n	3	4.5	46
N	0.71188	l32i addx4 l32i.n	3	6	9
O	0.711248	add.n l8ui	3	6	8
P	0.70665	addi movi.n	3	6	6
Q	0.705944	s32i l32i l32i	3	5	32
R	0.704984	l32i.n l32r	3	6	6
S	0.693818	movi.n s32i.n movi.n	3	5	26
T	0.693186	movi.n movi.n	3	5	25
U	0.682323	movi.n s32i.n movi.n s32i.n	3	5	21
V	0.675427	movi.n l32i.n	3	5	18
W	0.659992	l32i.n movi.n	3	3	61
X	0.656732	addx4 l32i.n	3	4	34
Y	0.655394	movi.n s32i.n l32i.n	3	3	59
Z	0.650772	l32r l32i.n	3	5	8

In this case study a set of operations to improve execution performance of multiple algorithms was selected. The target application required selecting the executionally expensive ("hot-spots") of the given algorithms. Therefore, frequency and commonality were the key properties to consider to select the most profitable set of operations. Hence, in the operation selection phase, *parameter factors* of static frequency, nesting level and commonality were given high weighting values while other parameter weights were set to zero. The distribution of these

weights is shown in the pie chart in Figure 6.1c. Thus, similar weights for the two frequency measurements were assigned, while the commonality was given a slightly lesser value. This was done to highlight the importance of execution frequency compared to commonality. Using these values the *priority factors* of all unique operations of the algorithms were calculated. Then, the final set of characteristic operations, critical in terms of their execution frequency were selected by ranking the candidate operations list. Table 6.12 lists some top operations resulting from this experiment.

Evaluation of these results was performed in a dynamic environment. In this evaluation, the run-time execution frequency of the top eight operations was counted to determine their execution frequency in a real execution environment. The problem size of the data set was gradually increased to see the execution variance. The results obtained in this experiment for all three algorithms are presented in Figures 6.12 to 6.14.

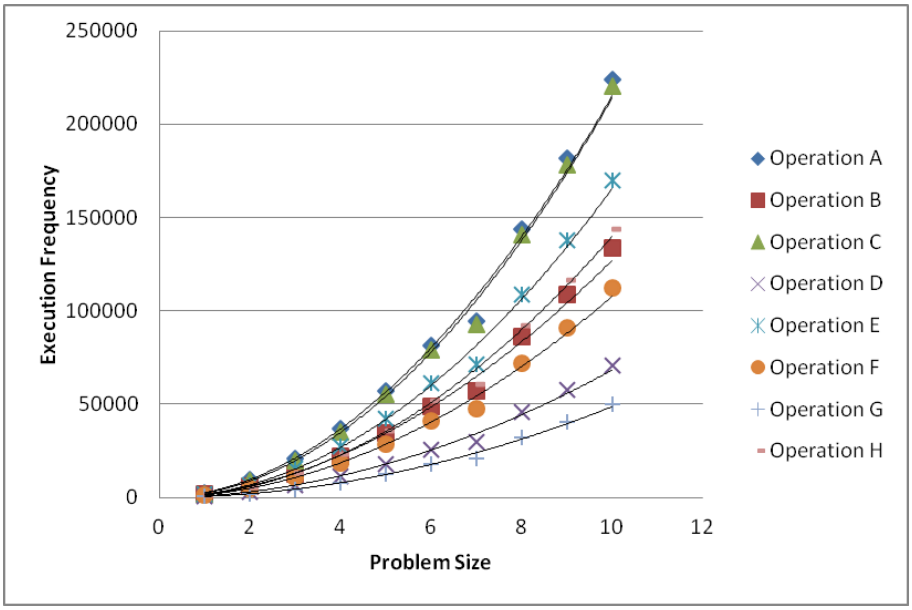


Figure 6.12: Execution frequency of characteristic operations of the *SmithWaterman* algorithm as per Table 6.12

According to these figures, the execution frequency of our identified characteristic operations increased rapidly with gradual changes of input data. This kind of behaviour is consistent with the nature of sequence alignment algorithms, which are highly complex and time consuming. In addition, when the execution frequency variations of individual operations are considered, the SmithWaterman and the NeedlemanWunsch algorithms show similar variations compared to the CV tree Algorithm. As discussed in Section 2.7.3, this can be explained by the fundamental relations between global and local alignment strategies in dynamic programing

methods which are deployed by both algorithms. Finally, based on these results we can conclude that our framework is indeed capable of successfully identifying characteristic operations common to multiple algorithms, and special relations between algorithms, when the specific properties of target applications are defined.

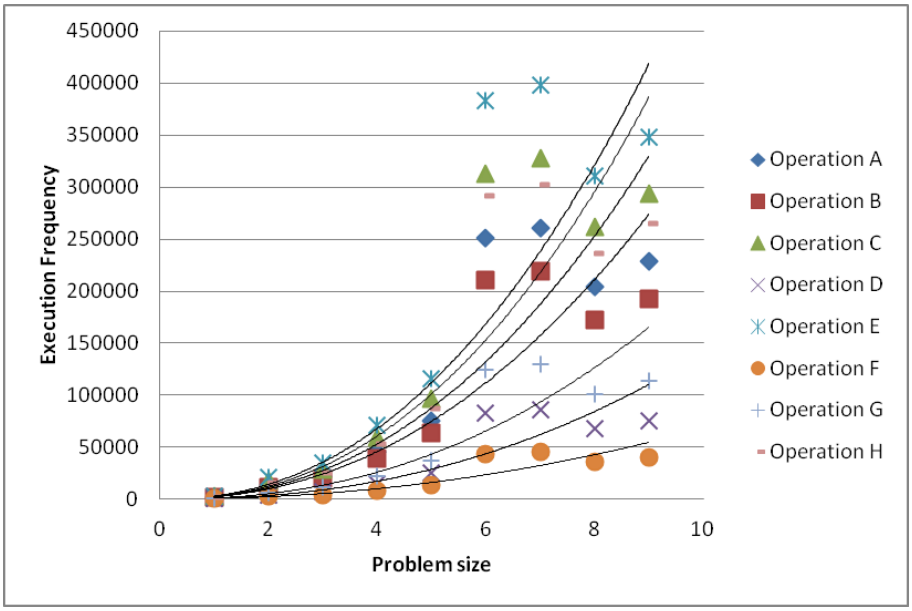


Figure 6.13: Execution frequency of characteristic operations of the *NeedlemanWunsch* algorithm as per Table 6.12

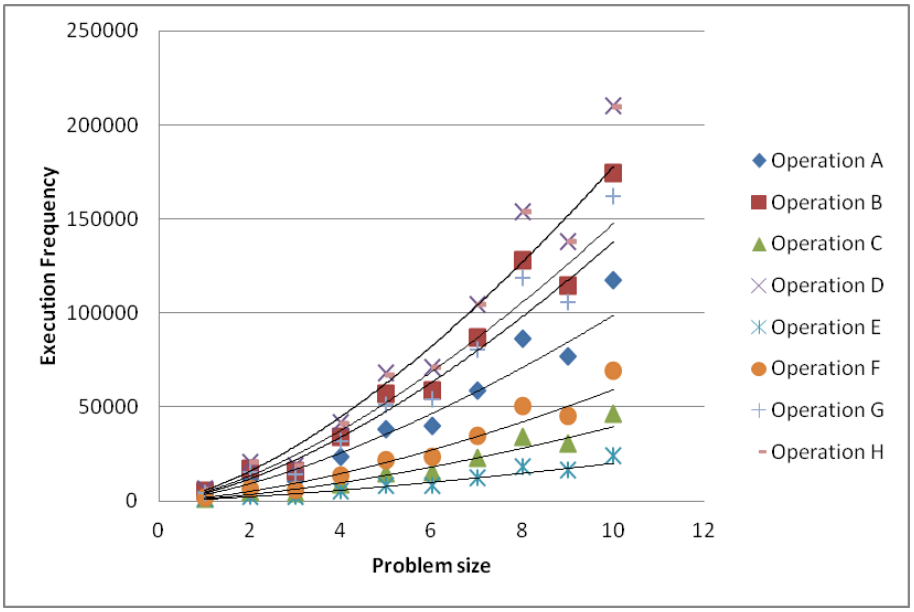


Figure 6.14: Execution frequency of characteristic operations of the *CV Tree* algorithm as per Table 6.12

Chapter 7

Conclusions

The presence of similar operations in multiple algorithms is useful for improving various performance factors of the objective algorithms all at once. However, extensive analyses are needed to identify such relationships between algorithms' operations since they are not trivial to find, especially in large problem domains. In contrast to existing manual and single algorithm targeted analysis approaches, this thesis presents an automated framework to identify similar operations between multiple algorithms. Its customisable process facilitates the selection of operations specific to given applications. The special-purpose operations selected targeting different applications can be used to improve the expected performance parameters of the considered algorithms. The development, usage and results of this static analysis-based framework was discussed and demonstrated using multiple case studies in this thesis. The accuracy of the results obtained by these case studies was further evaluated by appropriate dynamic experiments to confirm their usefulness. This chapter summarises the contributions of this research work and presents the final conclusions.

7.1 Summary of Contributions

The main outcome of this thesis is a general framework for identifying similarities between multiple algorithms, in terms of their shared functionalities. When a set of algorithms is given, the framework is capable of identifying critical operations common to the objective algorithms, considering given specifications for a target application domain. It also can identify clusters of similar algorithms which utilise similar characteristic operations. The resultant clusters can be fed into the system again to identify further specific operations for the custom set of algorithms.

The framework mainly consists of three phases which are interconnected with each other through intermediate inputs and outputs. These three phases address the three obstacles mentioned in Section 1.3, identified in the development process of our framework. The Overview of the framework with breif introduction of these phases are presented in Chapter 3. They are further discussed in detail in Chapters 4, 5 and 6.

7.1.1 Static based analysis strategies

As evidenced by the literature, many application areas could exploit significant performance improvements by finding common functionalities of multiple algorithms. However, in many fields the identification process of such operations is performed manually. They are often based on the known background details of the objective algorithms. The existing automated approaches, which use dynamic analyses, lack flexibility due to dependencies towards their target algorithms. On the other hand, many automated static analysis-based analysis approaches show good results, while maintaining applicability over different algorithms. The main drawback of all these methodologies is that most of them are built to handle only one algorithm at a time. Even though some approaches emphasise the importance of selecting common operations for multiple algorithms in the operations identification process, general procedures have yet to emerge.

When identifying similar operations between several algorithms, the characteristics of the operations considered to find the similarities are important. Since these operations define the dynamic behaviour of an algorithm, they must be "critical" operations to the algorithm in its run-time execution. In addition, as per the case studies presented in the previous chapters, the execution frequency of an operation is the key property for selecting the most beneficial set of operations targeting all three considered application areas of our experiments (Sections 6.2, 6.3 and 6.4). Therefore, the best operations selected to represent algorithms are always the most frequently executed operations. However, the dynamic execution behaviour of an algorithm is difficult to determine by static analysis, since its run-time execution details cannot be analysed in a static environment. Therefore, unlike other existing analysis methodologies, we deployed two complementary strategies to determine the static frequencies and nesting levels of operations. They help in determining the criticalness of the operations in terms of the dynamic execution frequency as explained in Chapter 4.

We performed the analysis in a static environment by deploying shallow syntactic matching

instead of deep semantic analyses based on graphs. The core operation matching phase is performed as a problem of syntactic text matching with corresponding data flow semantics. This helps in reducing the complexity and the execution speed of the typically NP hard problem while obtaining accurate results. Also, in our approach, all valid operations found in an algorithm are considered as candidate operations to be selected as the final operations set. Thus, operation pruning is carried out in the very last stage of the process. This prevents losing potentially good operations in the early stages of the process, allowing them to be selected by other property analyses in later stages for different target applications.

Thus, these static-based syntactic analysis strategies provided solutions for the first challenge mentioned in Section 1.3. The applications of these strategies were demonstrated in detail in Chapter 4. Then, dynamic evaluation experiments confirmed the importance of the selected operations at run time. In addition, these complementary analyses can be used as an independent strategy to identify frequently executed operations of different types of algorithms.

7.1.2 Classification method

The second phase of this research work is the algorithm clustering approach presented in detail in Chapter 5. In the related literature, there are many clustering methodologies which classify algorithms into groups considering their functionality to solve a specific computation or a particular problem. These approaches are limited to specific kinds of algorithms. Also they do not conduct comprehensive comparisons that can be performed between algorithms in terms of all existing core operations. To our knowledge, our clustering method is the first to find similarities of algorithms, based on their characteristic operations. Our clustering approach is not specialised for any particular operation/s or types of algorithm. Instead, it considers characteristic operations which are frequently executed by the algorithms at run time. As shown in the experiments in Chapter 5, this quality enables it to be used in many different groups of algorithms.

This phase is especially important to our core framework when a large number of algorithms needs to be considered. In such a situation, finding similar operations can be difficult with the large number of algorithms and diverse nature of their operations/functionalities. Therefore, it will be hard to find suitable combinations of the algorithms that may all benefit from the same run time environment. This can be determined easily by our clustering method, since it is capable of producing closely related algorithm groups according to a user defined degree of

similarity. Thus, this phase provided solutions for the second challenge discussed in Section 1.3.

In addition, this phase also can be used as an independent classification approach to categorise different types of algorithms in terms of their common operations. Although in our case studies we have used our static analysis strategies to identify characteristic operations, in a different system the classification can be deployed once the characteristic operations have already been found, using any other available analysis method. As shown in the experiments, in fields like ADSP design, our clustering can be used to find the best combination of available algorithms to design custom processors. Likewise, the clustering approach will serve many different application areas where clusters of algorithms, which share common operations, are needed.

7.1.3 Weighting system

The third phase of the framework introduces a heuristic weighting system to calibrate the selection of operations common to multiple algorithms targeting a given application. In contrast to existing algorithm analysis methodologies, which find a candidate set of operations targeting a particular application; here we support different types of applications. The flexibility of the selection process for suitable operations is obtained by a weighting system.

It allows prioritising different properties of the operations for given specifications of a particular application. The necessary property parameters and their priority rankings are determined according to these specifications. The numerical weightings of each property parameter are then decided. The priorities of all operations are calculated, considering their individual scores, and hence operations are ranked to select the best set of dedicated operations. This system facilitates the generalisation of our whole process for different application domains. We have demonstrated the versatility of this system and hence, our framework by case studies carried out in Chapters 6. In addition to being a major part of our framework, this weighting system can be used as an independent approach for specialising operations towards different applications. This provides insights into specialising operations of algorithms towards a given specification. Thus, as described in Chapter 6, the weighting system has successfully addressed the third obstacle identified in Section 1.3.

7.1.4 Concluding remarks

Integration of the above three phases forms our final framework which is capable of performing all the above phases using an automated tool chain. The system takes a set of given algorithms as input and produces a list of operations, which are the most promising operations to use to support a particular kind of application. Thus, we have successfully addressed all problems outlined in Section 1.3.

7.2 Limitations and Future Work

Even though the framework was demonstrated as an automated tool chain built around off-the-shelf software tools, it can be improved as a single tool in the future, where all the intermediate steps can be performed in a single run. Also, it uses the software tools of the Xtensa workspace to map an assembly level implementation with the high level one. This limits our current tool chain to a particular architecture and a high level source language like C/C++. This could be avoided by developing an independent parser in future work to read and map assembly and high level codes with each other. Furthermore a formal method to represent our framework will be developed in the future.

References

Allen, F. E. (1970). Control flow analysis. *SIGPLAN*, 5(7):1–19.

Amold, M. and Corporaal, H. (2001). Designing domain-specific processors. In *Proceedings of the Ninth International Symposium on Hardware/Software Codesign, 2001*, pages 61–66.

Arnold, M. and Corporaal, H. (1999). Automatic detection of recurring operation patterns. In *Seventh International Workshop on Hardware/Software Codesign, 1999*, pages 22–26.

Atasu, K., Pozzi, L., and Ienne, P. (2003). Automatic application-specific instruction-set extensions under microarchitectural constraints. In *Proceedings of the Design Automation Conference*, pages 256–261.

Bakker, P. (2014). Polassl library.

Barry, T., Solz, T., Reising, J., and Williamson, D. (1994). The simultaneous use of three machine speech recognition systems to increase recognition accuracy. In *Proceedings of the IEEE 1994 National Aerospace and Electronics Conference, 1994.*, pages 667–671 vol.2.

Bobda, C. (2007). *Introduction to Reconfigurable Computing: Architectures, Algorithms, and Applications*. Springer Publishing Company, Incorporated, 1st edition.

Bojoi, R., Neacsu, M. G., and Tenconi, A. (2012). Analysis and survey of multi-phase power electronic converter topologies for the more electric aircraft applications. In *International Symposium on Power Electronics, Electrical Drives, Automation and Motion (SPEEDAM), 2012*, pages 440–445.

Chaikalis, C., Salimi-Khaligh, M., Panayotopoulos, N., and Noras, J. (2000). Reconfiguration between soft output Viterbi and log maximum a posteriori decoding algorithms. In *First International Conference 3G Mobile Communication Technologies, 2000*, pages 316–320.

Charras, C. and Lecroq, T. (2004). *Handbook of Exact String Matching Algorithms*. King's College Publications.

Checconi, F., Petrini, F., Willcock, J., Lumsdaine, A., Choudhury, A. R., and Sabharwal, Y. (2012). Breaking the speed and scalability barriers for graph exploration on distributed-memory machines. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, SC '12, pages 13:1–13:12, Los Alamitos, CA, USA. IEEE Computer Society Press.

Choi, H., Kim, J., Yoon, C., Park, I., Hwang, S., and Kyung, C. (1999). Synthesis of application specific instructions for embedded DSP software. *IEEE Transactions on Computers*, 48(6):603–614.

Clark, N., Tang, W., and Mahlke, S. (2002). Automatically generating custom instruction set extensions. In *Workshop on Application-Specific Processors (WASP)*, pages 94–101.

Clark, N., Zhong, H., and Mahlke, S. (2003). Processor acceleration through automated instruction set customization. In *Proceedings of the 36th Annual IEEE/ACM International Symposium on Microarchitecture*, MICRO 36, Washington, DC, USA. IEEE Computer Society.

Clark, N., Zhong, H., and Mahlke, S. (2005). Automated custom instruction generation for domain-specific processor acceleration. *IEEE Transactions on Computers*, 54(10):1258–1270.

Cong, J., Fan, Y., Han, G., and Z., Z. (2004). Application-specific instruction generation for configurable processor architectures. In *Proceedings of ACM International Symposium on Field-Programmable Gate Arrays*, pages 183–189.

De Pierro, A. (1993). On the relation between the isra and the em algorithm for positron emission tomography. *IEEE Transactions on Medical Imaging*, 12(2):328–333.

Enright, M. (2010). An analysis of advanced optimization algorithms for multiuser cooperative navigation. In *Position Location and Navigation Symposium (PLANS), 2010 IEEE/ION*, pages 966–971.

Ernst, M. D. (2003). Static and dynamic analysis: Synergy and duality. In *WODA 2003: ICSE Workshop on Dynamic Analysis*, pages 24–27, Portland, OR.

Eusse, J., Williams, C., and Leupers, R. (2013). Coex: A novel profiling-based algorithm/architecture co-exploration for asip design. In *8th International Workshop on Reconfigurable and Communication-Centric Systems-on-Chip (ReCoSoC), 2013*, pages 1–8.

Fanucci, L., Cassiano, M., Saponara, S., Kammler, D., Witte, E., and Schliebusch, O. (2006). ASIP design and synthesis for non linear filtering in image processing. In *Proceedings of Design, Automation and Test in Europe, 2006. DATE '06*, volume 2, pages 1–6.

Fischer, M. J. and Paterson, M. S. (1974). String-matching and other products. Technical report, Massachusetts Institute of Technology, Cambridge, MA, USA.

Galuzzi, C. and Bertels, K. (2011). The instruction-set extension problem: A survey. *ACM Transactions on Reconfigurable Technology and Systems*, 4(2):18:1–18:28.

Greenberg, H. J. and Loh, L. W. T. (1974). An analysis of cut-off rules for optimization algorithms. *IEEE Transactions on Systems, Man and Cybernetics*, SMC-4(1):108–112.

Gries, M. (2004). Methods for evaluating and covering the design space during early design development. *Integr. VLSI J.*, 38(2):131–183.

Halambi, A., Shrivastava, A., Biswas, P., Dutt, N., and Nicolau, A. (2002). An efficient compiler technique for code size reduction using reduced bit-width ISAs. In *Proceedings Design, Automation and Test in Europe Conference and Exhibition, 2002*, pages 402–408.

Hoffmann, A., Fiedler, F., Nohl, A., and Parupalli, S. (2005). A methodology and tooling enabling application specific processor design. In *18th International Conference on VLSI Design, 2005*, pages 399–404.

Hubert, H. and Stabernack, B. (2009). Profiling-based hardware/software co-exploration for the design of video coding architectures. *IEEE Transactions on Circuits and Systems for Video Technology*, 19(11):1680–1691.

Jain, M., Balakrishnan, M., and Kumar, A. (2001). ASIP design methodologies: survey and issues. In *Fourteenth International Conference on VLSI Design, 2001*, pages 76 –81.

Jin, G. and Tierney, B. L. (2003). System capability effects on algorithms for network bandwidth measurement. In *Proceedings of the 3rd ACM SIGCOMM Conference on Internet Measurement*, IMC '03, pages 27–38, New York, NY, USA. ACM.

Kaufmann, P., Englehart, K., and Platzner, M. (2010). Fluctuating emg signals: Investigating long-term effects of pattern matching algorithms. In *Annual InternationalConference of the IEEE Engineering in Medicine and Biology Society (EMBC), 2010*, pages 6357–6360.

Kim, H.-E., Yoon, J.-S., Hwang, K.-D., Kim, Y.-J., Park, J.-S., and Kim, L.-S. (2012). A reconfigurable heterogeneous multimedia processor for ic-stacking on si-interposer. *IEEE Transactions on Circuits and Systems for Video Technology*, 22(4):589–604.

Kun, Y., Chun, Z., and Zhihua, W. (2006). Application Specific Processor Design For H.264 Baseline Profile Bit-Stream Decoding. In *8th International Conference on Signal Processing, 2006*, volume 1.

Kwok, Y.-K. and Ahmad, I. (1998). Benchmarking the task graph scheduling algorithms. In *Proceedings of the First Merged International Parallel Processing Symposium and Symposium on Parallel and Distributed Processing*, pages 531 –537.

Levitin, A. (2007). *Introduction to the Design and Analysis of Algorithms*. Addison-Wesley, second edition. ISBN 0-321-36413-9.

Liem, C., May, T., and Paulin, P. (1994). Instruction-set matching and selection for DSP and ASIP code generation. In *European Design and Test Conference, 1994*, pages 31 –37.

Lin, H. and Fei, Y. (2010). Exploring custom instruction synthesis for application-specific instruction set processors with multiple design objectives. In *Proceedings of the 16th ACM/IEEE International Symposium on Low Power Electronics and Design*, ISLPED '10, pages 141–146, New York, NY, USA.

Lloyd, S. P. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, pages 129–137.

MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations.

Mbaye, M., Belanger, N., Savaria, Y., and Pierre, S. (2005). Application specific instruction-set processor generation for video processing based on loop optimization. In *IEEE International Symposium on Circuits and Systems, 2005*, pages 3515–3518.

Mehrotra, M., Goel, A., Agarwal, N., Hima Bindu, M., and Sharma, B. (2009). Algorithm classification using worst-case execution time. In *2nd IEEE International Conference on Computer Science and Information Technology*, pages 286 –290.

Menezes, A. J., Van Oorschot, P. C., and Vanstone, S. A. (1996). *Handbook of applied cryptography*. CRC press.

Mili, L., Kamek, U., and Joci, A. (1987). Analysis of filter bank transfer functions. *IEEE Proceedings Electronic Circuits and Systems*, 134(1):23–30.

Mitchell, B. and Mancoridis, S. (2001). Comparing the decompositions produced by software clustering algorithms using similarity measurements. In *IEEE International Conference on Software Maintenance, 2001*, pages 744–753.

Mount, D. W. (2004). Sequence and genome analysis. *New York: Cold Spring*.

Needleman, S. B. and Wunsch, C. D. (1970). A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology*, 48(3):443 – 453.

Nery, A. S., Nedjah, N., Franca, F. M., Jozwiak, L., and Corporaal, H. (2014). Automatic complex instruction identification for efficient application mapping onto ASIPs. In *IEEE 5th Latin American Symposium on Circuits and Systems (LASCAS), 2014*, pages 1–4.

Nohl, A., Greive, V., Braun, G., Hoffman, A., Leupers, R., Schliebusch, O., and Meyr, H. (2003). Instruction encoding synthesis for architecture exploration using hierarchical processor models. In *Proceedings Design Automation Conference, 2003*, pages 262–267.

Ong, Y.-S., Lim, M.-H., Zhu, N., and Wong, K.-W. (2006). Classification of adaptive memetic algorithms: a comparative study. *IEEE Transactions on Systems, Man, and Cybernetics, Part B: Cybernetics*, 36(1):141 –152.

Pelegri-Llopart, E. and Graham, S. L. (1988). Optimal code generation for expression trees: An application burs theory. In *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '88, pages 294–308, New York, NY, USA. ACM.

Price, D. B. (1982). Statistical analysis of piloted simulation of real time trajectory optimization algorithms. In *American Control Conference, 1982*, pages 1105–1106.

Qadeer, W., Hameed, R., Shacham, O., Venkatesan, P., Kozyrakis, C., and Horowitz, M. A. (2013). Convolution engine: Balancing efficiency of flexibility in specialized computing. *SIGARCH Computer Architecture News*, 41(3):24–35.

Rashid, M., Picard, D., and Pottier, B. (2008). Application analysis for parallel processing. In *11th EUROMICRO Conference on Digital System Design Architectures, Methods and Tools, 2008*, pages 633–640.

Riedl, R. and Richter, L. (1996). Classification of load distribution algorithms. In *Proceedings of the Fourth Euromicro Workshop on Parallel and Distributed Processing*, pages 404 –413.

Rinard, M. and Diniz, P. (1996). Commutativity analysis: a technique for automatically parallelizing pointer-based computations. In *The 10th International Parallel Processing Symposium, 1996*, pages 14–22.

Sang, S., Li, X., and Ye, Y. (2005). Automatic instruction generation for application specific co-processor. In *6th International Conference On ASIC, 2005*, volume 2, pages 934–938.

Sedgewick, R. and Flajolet, P. (2013). *An Introduction to the Analysis of Algorithms*. Addison-Wesley, 2nd edition.

Shirai, K., Ikenaga, T., and Kitabatake, H. (1990). Design system for special purpose processor executing algorithms described by higher level language. In *Proceedings of Third Annual IEEE ASIC Seminar and Exhibit, 1990*, pages P7/1.1 –P7/1.4.

Sistanizadeh, K. (1990). Analysis and performance evaluation studies of high bit rate digital subscriber lines (HDSL) using QAM and PAM schemes with ideal decision feedback equalization (DFE) within a carrier serving area (CSA). In *IEEE Global Telecommunications Conference and Exhibition, 1990*, pages 1172–1176 vol.2.

Smith, T. and M.S., W. (1981). Identification of common molecular subsequences. *Molecular Biology*, 147:195–197.

Sreedharan, D. and Akoglu, A. (2008). A hybrid processing element based reconfigurable architecture for hashing algorithms. In *IEEE International Symposium on Parallel and Distributed Processing, 2008*, pages 1–8.

Srinivasan, U., Chen, P.-S., Diao, Q., Lim, C.-C., Li, E., Chen, Y., Ju, R., and Zhang, Y. (2005). Characterization and analysis of HMMER and SVM-RFE parallel bioinformatics applications. In *Proceedings of the IEEE International Workload Characterization Symposium, 2005*, pages 87–98.

Stelling, P. and Oklobdzija, V. (1997). Implementing multiply-accumulate operation in multiplication time. In *13th IEEE Symposium on Computer Arithmetic, 1997*, pages 99–106.

Sugita, K., Naemura, T., and Harashima, H. (2003). Performance evaluation of programmable graphics hardware for image filtering and stereo matching. In *Proceedings of the ACM Symposium on Virtual Reality Software and Technology*, VRST '03, pages 176–183, New York, NY, USA. ACM.

Sun, F., Raghunathan, A., Ravi, S., and Jha, N. K. (2004). Custom-instruction synthesis for extensible-processor platforms. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 23:216–228.

Tarjan, R. (1973). Testing flow graph reducibility. In *Proceedings of the fifth annual ACM symposium on Theory of computing*, STOC '73, pages 96–107, New York, NY, USA. ACM.

Tensilica (accessed in July 2014). Xtensa customizable processors. http://www.tensilica.com/products/xtensa-customizable.htm.

Thomas, A. and Olukotun, K. (2005). An application analysis framework for polymorphic chip multiprocessors. In *19th IEEE International Parallel and Distributed Processing Symposium, 2005*.

Trabelsi, Z. and Hamdy, R. (2010). Implementation of a DNA-based anomaly identification system utilizing associative string processor (ASP). In *IEEE/ACS International Conference on Computer Systems and Applications (AICCSA), 2010*, pages 1 –8.

Wei, Y., Wang, G., and Hsieh, J. (2005). Relation Between the Filtered Backprojection Algorithm and the Backprojection Algorithm in CT. *IEEE Signal Processing Letters*, 12(9):633–636.

Woodward, J. and Swan, J. (2010). Why classifying search algorithms is essential. In *IEEE International Conference on Progress in Informatics and Computing (PIC)*, volume 1, pages 285 –289.

Xian-feng, H., Yu-bao, Y., and Lu, X. (2010). Hybrid pattern-matching algorithm based on bm-kmp algorithm. In *Advanced Computer Theory and Engineering (ICACTE), 2010 3rd International Conference on*, volume 5, pages V5–310–V5–313.

Xu, R. and Wunsch, D., I. (2005). Survey of clustering algorithms. *IEEE Transactions on Neural Networks*, 16(3):645 –678.

Yang, S., Lin, C., Hung, C., Wu, J., and Wang, Y. (2007). Application-specific instruction generation for SOC processors. In *IEEE International Symposium on Circuits and Systems, 2007*, pages 3752–3755.

Yu, Z.-G., Chu, K., Li, C., Anh, V., Zhou, L.-Q., and Wang, R. (2010). Whole-proteome phylogeny of large dsDNA viruses and parvoviruses through a composition vector method related to dynamical language model. *BMC Evolutionary Biology*, 10(1).

Zampirolli, F. D. A. and Lotufo, R. D. A. (2000). Classification of the distance transformation algorithms under the mathematical morphology approach. In *Proceedings XIII Brazilian Symposium on Computer Graphics and Image Processing*, pages 292 –299.

Zhao, K. and Bian, J. (2010). Peeling algorithm for custom instruction identification. In *IEEE Asia Pacific Conference on Circuits and Systems (APCCAS), 2010*, pages 720–723.

Zhao, K., Bian, J., Dong, S., Song, Y., and Goto, S. (2008). Automated specific instruction customization methodology for multimedia processor acceleration. In *Proceedings of the 9th international symposium on Quality Electronic Design*, pages 321–324, Washington, DC, USA. IEEE Computer Society.

Zhou, B., Wu, G.-f., Xu, Y., Li, J.-g., and Liu, Y. (2013). A visual dataflow model for the process flow of remote sensing products. In *Proceedings of the 9th International Conference on Intelligent Computing Theories*, ICIC'13, pages 482–489, Berlin, Heidelberg. Springer-Verlag.

Zou, G. and Liu, X. (2005). An efficient approach to custom instruction set generation. In *11th IEEE International Conference on Embedded and Real-Time Computing Systems and Applications, 2005. Proceedings*, pages 547–550.